



MIPI CSI/DSI IP

IP Version: v4.2.0

User Guide

FPGA-IPUG-02321-1.2

August 2026

Disclaimers

Lattice makes no warranty, representation, or guarantee regarding the accuracy of information contained in this document or the suitability of its products for any particular purpose. All information herein is provided AS IS, with all faults, and all associated risk is the responsibility entirely of the Buyer. The information provided herein is for informational purposes only and may contain technical inaccuracies or omissions, and may be otherwise rendered inaccurate for many reasons, and Lattice assumes no obligation to update or otherwise correct or revise this information. Products sold by Lattice have been subject to limited testing and it is the Buyer's responsibility to independently determine the suitability of any products and to test and verify the same. LATTICE PRODUCTS AND SERVICES ARE NOT DESIGNED, MANUFACTURED, OR TESTED FOR USE IN LIFE OR SAFETY CRITICAL SYSTEMS, HAZARDOUS ENVIRONMENTS, OR ANY OTHER ENVIRONMENTS REQUIRING FAIL-SAFE PERFORMANCE, INCLUDING ANY APPLICATION IN WHICH THE FAILURE OF THE PRODUCT OR SERVICE COULD LEAD TO DEATH, PERSONAL INJURY, SEVERE PROPERTY DAMAGE OR ENVIRONMENTAL HARM (COLLECTIVELY, "HIGH-RISK USES"). FURTHER, BUYER MUST TAKE PRUDENT STEPS TO PROTECT AGAINST PRODUCT AND SERVICE FAILURES, INCLUDING PROVIDING APPROPRIATE REDUNDANCIES, FAIL-SAFE FEATURES, AND/OR SHUT-DOWN MECHANISMS. LATTICE EXPRESSLY DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY OF FITNESS OF THE PRODUCTS OR SERVICES FOR HIGH-RISK USES. The information provided in this document is proprietary to Lattice Semiconductor, and Lattice reserves the right to make any changes to the information in this document or to any products at any time without notice.

Inclusive Language

This document was created consistent with Lattice Semiconductor's inclusive language policy. In some cases, the language in underlying tools and other items may not yet have been updated. Please refer to Lattice's inclusive language [FAQ 6878](#) for a cross reference of terms. Note in some cases such as register names and state names it has been necessary to continue to utilize older terminology for compatibility.

Contents

Contents	3
Abbreviations in This Document.....	8
1. Introduction	10
1.1. Overview of the IP	10
1.2. Quick Facts	10
1.3. IP Support Summary	11
1.4. Features	11
1.4.1. MIPI CSI/DSI IP on Transmitter Mode	11
1.4.2. MIPI CSI/DSI IP on Receiver Mode	11
1.5. Licensing and Ordering Information	11
1.6. Minimum Device Requirements	12
1.7. Naming Conventions	12
1.7.1. Nomenclature.....	12
1.7.2. Signal Names	12
1.7.3. Attribute Names.....	12
2. Functional Description.....	13
2.1. IP Architecture Overview	13
2.2. Clocking	14
2.2.1. MIPI CSI/DSI Transmitter Clocking Overview	15
2.2.2. MIPI CSI/DSI Receiver Clocking Overview	17
2.3. Reset	18
2.3.1. Reset Overview	18
2.4. User Interfaces	18
2.4.1. Unified Video Streaming Interface (UVSI)	19
2.4.2. MIPI Byte Streaming Interface (MBSI).....	24
2.5. MIPI CSI/DSI Transmitter PHY Block.....	28
2.6. MIPI Timing Parameters.....	28
2.7. MIPI CSI/DSI Transmitter Controller Block	29
2.7.1. MIPI CSI/DSI Transmitter Skew Calibration	29
2.8. MIPI CSI/DSI Transmitter Pixel-to-Byte Block	31
2.8.1. MIPI CSI/DSI Transmitter Blanking Insertion	31
2.9. MIPI CSI/DSI Receiver PHY Block.....	33
2.10. MIPI CSI/DSI Receiver Controller Block	33
2.11. MIPI CSI/DSI Receiver Byte-to-Pixel Block.....	34
2.12. Dynamic Reconfiguration	34
2.12.1. MIPI CSI/DSI Transmitter Dynamic Lane Reconfiguration	34
2.12.2. MIPI CSI/DSI Receiver Dynamic Lane Reconfiguration	34
2.12.3. MIPI CSI/DSI Transmitter Resolution Reconfiguration	35
3. IP Parameter Description.....	36
3.1. General.....	36
3.2. MIPI CSI/DSI TX Protocol Timing Parameters	42
4. Signal Description	44
4.1. MIPI CSI/DSI TX Signal Description	44
4.1.1. Clock Interface.....	44
4.1.2. D-PHY Interface	45
4.1.3. Unified Video Streaming Interface	45
4.1.4. MIPI Byte Streaming Interface	45
4.1.5. Miscellaneous.....	46
4.1.6. AXI4-Lite Subordinate Interface	46
4.2. MIPI CSI/DSI RX Signal Description.....	47
4.2.1. Clock Interface.....	47
4.2.2. D-PHY Interface	47

4.2.3.	Unified Video Streaming Interface	48
4.2.4.	MIPI Byte Streaming Interface	48
4.2.5.	AXI4-Lite Subordinate Interface	49
5.	Register Description	50
5.1.	MIPI CSI/DSI Transmitter Register Description	50
5.1.1.	STATUS_REG (0x000)	50
5.1.2.	CONTROLLER_STATUS_REG (0x020)	50
5.1.3.	CONVERTER_STATUS_REG1 (0x044)	50
5.1.4.	PHY_CONFIG_REG (0x210)	51
5.1.5.	CONFIG_REG0 (0x220)	51
5.1.6.	CONFIG_REG1 (0x240)	52
5.1.7.	CONFIG_REG2 (0x244)	52
5.2.	MIPI CSI/DSI Receiver Register Description	53
5.2.1.	STATUS_REG (0x000)	53
5.2.2.	CONTROLLER_STATUS_REG (0x020)	54
5.2.3.	CONVERTER_STATUS_REG (0x040)	54
5.2.4.	CONFIG_REG (0x200)	54
5.2.5.	PHY_CONFIG_REG (0x210)	55
6.	Designing with the IP	56
6.1.	Generating and Instantiating the IP	56
6.1.1.	Generated Files and File Structure	58
6.2.	Design Implementation	59
6.3.	Timing Constraints	59
6.4.	Physical Constraints	60
6.5.	Specifying the Strategy	61
6.6.	Running Functional Simulation	61
6.6.1.	Simulation Results	62
7.	Example Design	64
7.1.	Example Design Supported Configuration	64
7.2.	Overview of Example Design and Features	65
7.3.	Example Design Overview	66
7.3.1.	Example Design System	66
7.4.	Data Flow	70
7.4.1.	Main Example Design System	70
7.4.2.	RISC-V Subsystem	71
7.5.	Clocking Scheme	71
7.6.	Reset Scheme	71
7.6.1.	Reset Overview	71
7.7.	Software	72
7.7.1.	RISC-V Bare Metal Program	72
7.8.	Signal Description	73
7.9.	Example Design Hardware Setup	73
7.9.1.	Locating the Example Design Files	73
7.9.2.	Opening the Lattice Propel Builder Project	73
7.9.3.	Regenerating the IP and Recompiling the Lattice Radiant Software Project	74
7.9.4.	Hardware Board Setup	74
7.9.5.	Running the Hardware	75
7.9.6.	Setting Up the Demo GUI Application	77
8.	Debugging	81
8.1.	Debug Methods	81
8.1.1.	PHY Clock Status in Transmitter Mode	81
8.1.2.	Generated byte_clk_o Does Not Toggle in Receiver Mode	81
8.1.3.	Data Appears Corrupted in Receiver Mode	81
8.1.4.	Missing Packets in Receiver Mode	81

8.2.	Debug Tools.....	81
8.2.1.	Reveal Analyzer	81
9.	Design Considerations	83
9.1.	Limitations for MIPI CSI/DSI Transmitter	83
9.2.	Limitations for MIPI CSI/DSI Receiver	83
9.3.	Horizontal and Vertical Blanking Period Insertion Tuning	84
	Appendix A. Resource Utilization	90
	Appendix B. Raspberry Pi OS Setup	96
	Raspberry Pi 5 OS Software Setup	96
	Setting Up the MIPI CSI Display in the Raspberry Pi 5 OS.....	96
	Troubleshooting the Raspberry Pi 5 Camera Software Preview Issue.....	97
	References.....	98
	Technical Support Assistance	99
	Revision History.....	100

Figures

Figure 1.1.	MIPI CSI/DSI IP Overview	10
Figure 2.1.	MIPI CSI/DSI IP Transmitter Core Block Diagram	13
Figure 2.2.	MIPI CSI/DSI IP Receiver Core Block Diagram	13
Figure 2.3.	MIPI CSI/DSI IP Transmitter Clock Domain Block Diagram (UVSI)	14
Figure 2.4.	MIPI CSI/DSI IP Transmitter Clock Domain Block Diagram (MBSI).....	14
Figure 2.5.	MIPI CSI/DSI IP Receiver Clock Domain Block Diagram (UVSI)	15
Figure 2.6.	MIPI CSI/DSI IP Receiver Clock Domain Block Diagram (MBSI).....	15
Figure 2.7.	Edge Clock Sharing Port Connection.....	17
Figure 2.8.	MIPI CSI/DSI IP Transmitter Reset Sequence Waveform.....	18
Figure 2.9.	MIPI CSI/DSI IP Receiver Reset Sequence Waveform	18
Figure 2.10.	UVSI to Video Frame Timing Mapping	19
Figure 2.11.	Unified Video Streaming Timing Diagram.....	20
Figure 2.12.	TDATA Pixel Mapping for Sample RGB565 Configuration	21
Figure 2.13.	TDATA Pixel Mapping for Sample RGB888 Configuration	21
Figure 2.14.	TDATA Pixel Mapping for Sample RAW10 Configuration	22
Figure 2.15.	TDATA Pixel Mapping for Sample RAW12 Configuration	22
Figure 2.16.	UVSI TX Timing Diagram with Blanking Interval	23
Figure 2.17.	UVSI TX Timing Diagram without Blanking Interval.....	23
Figure 2.18.	UVSI RX Timing Diagram without TREADY De-assertion.....	24
Figure 2.19.	UVSI RX Timing Diagram with TREADY De-assertion	24
Figure 2.20.	MBSI TX Timing Diagram for 32-bit Passthrough Mode	25
Figure 2.21.	MBSI TX Timing Diagram for 64-bit Passthrough Mode	25
Figure 2.22.	MBSI TX Timing Diagram for 32-bit Passthrough Mode with Backpressure.....	26
Figure 2.23.	MBSI RX Timing Diagram for 32-bit Passthrough Mode	26
Figure 2.24.	MBSI RX Timing Diagram for 64-bit Passthrough Mode	27
Figure 2.25.	MBSI RX Timing Diagram for 64-bit MPPT Passthrough Mode.....	27
Figure 2.26.	MBSI RX Timing Diagram for 32-bit Passthrough Mode with TREADY Deassertion	28
Figure 2.27.	MIPI Timing Parameters	28
Figure 2.28.	Initial Skew Calibration Sequence.....	30
Figure 2.29.	Periodic Skew Calibration Sequence.....	30
Figure 2.30.	LP-11 State After HS Burst Transmission	32
Figure 6.1.	Module/IP Block Wizard	56
Figure 6.2.	MIPI CSI/DSI IP Receiver Mode Configuration	57
Figure 6.3.	MIPI CSI/DSI IP Transmitter Mode Configuration	57
Figure 6.4.	Check Generated Result	58
Figure 6.5.	Example Evaluation Top Setup	59

Figure 6.6. Example Evaluation Project Settings	60
Figure 6.7. Defining MIPI D-PHY Port Pins Using MIPI_DPHY IO_TYPE.....	60
Figure 6.8. Simulation Wizard.....	61
Figure 6.9. Add and Reorder Source	62
Figure 6.10. Simulation Waveform	62
Figure 6.11. Simulation Result	63
Figure 6.12. Simulation Result with Error	63
Figure 6.13. Additional Timing Information.....	63
Figure 7.1. Example Design System Diagram.....	66
Figure 7.2. MIPI Subsystem Diagram	67
Figure 7.3. I2C Subsystem Diagram	68
Figure 7.4. RISC-V Subsystem Diagram.....	69
Figure 7.5. Reset Sequence of the Example Design.....	72
Figure 7.6. Hardware Setup	74
Figure 7.7. Selecting Bitstream in Radiant Programmer	76
Figure 7.8. LEDs Light Up After Bitstream Programming.....	76
Figure 7.9. Demo GUI Setup	78
Figure 7.10. Color Bar CFA – BGGR for Pattern Subsystem to TX Mode Configuration Example	79
Figure 7.11. Sensor RAW Mode for RX to TX Retransmit Mode.....	79
Figure 7.12. Color Bar, CFA – RGGB for Pattern Subsystem to TX Mode	80
Figure 7.13. Color Bar, CFA – BGGR for Pattern Subsystem to TX Mode	80
Figure 7.14. Checkerboard for Pattern Subsystem to TX Mode	80
Figure 9.1. Minimum Vertical Frame Blanking between Frame End and the Next Frame Start HS Burst Short Packet (Number of Vertical Blanking Lines Per Frame = 0)	85
Figure 9.2. Horizontal Line Blanking between Line End and the Next Line Start HS Burst Short Packet (Number of Horizontal Blanking Bytes in Byte Clock Domain = 559)	86
Figure 9.3. Horizontal Line Blanking between Line End and the Next Line Start HS Burst Short Packet (Number of Horizontal Blanking Bytes in Byte Clock Domain = 133)	87
Figure 9.4. Minimum Horizontal Line Blanking between Line End and the Next Line Start HS Burst Short Packet (Number of Horizontal Blanking Bytes in Byte Clock Domain = 0).....	88
Figure 9.5. Vertical Frame Blanking between Frame End and the Next Frame Start HS Burst Short Packet (Number of Vertical Blanking Lines Per Frame = 133)	89
Figure B.1. Camera Sensor Detected in the Raspberry Pi 5 OS.....	96
Figure B.2. Timeout Error in the Raspberry Pi 5 OS.....	97

Tables

Table 1.1. Summary of the MIPI CSI/DSI IP.....	10
Table 1.2. MIPI CSI/DSI IP Support Readiness	11
Table 2.1. MIPI CSI/DSI Transmitter Operating Frequency Details and Requirements	15
Table 2.2. MIPI CSI/DSI Receiver Operating Frequency Details and Requirements	17
Table 2.3. User Interfaces and Supported Protocols	18
Table 2.4. MIPI CSI/DSI Transmitter Word Count Support Limitation for MBSI and UVS1	29
Table 2.5. MIPI CSI/DSI Transmitter Pixel-to-Byte Supported Configurations	31
Table 2.6. Example of Expected Frame Rate Calculation	32
Table 2.7. MIPI CSI/DSI Receiver Word Count Support Limitation	33
Table 2.8. MIPI CSI/DSI Receiver Byte-to-Pixel Supported Configurations.....	34
Table 3.1. General Attributes	36
Table 3.2. MIPI CSI/DSI TX Protocol Timing Parameters – General Attributes	42
Table 4.1. Clock and Reset Ports.....	44
Table 4.2. D-PHY Interface Ports	45

Table 4.3. Unified Video Streaming Interface Ports	45
Table 4.4. MIPI Byte Streaming Interface Ports.....	45
Table 4.5. Miscellaneous Ports	46
Table 4.6. AXI4-Lite Subordinate Interface Ports	46
Table 4.7. Clock and Reset Ports ¹	47
Table 4.8. D-PHY Interface Ports	47
Table 4.9. Unified Video Streaming Interface Ports	48
Table 4.10. MIPI Byte Streaming Interface Ports.....	48
Table 4.11. AXI4-Lite Subordinate Interface Ports	49
Table 5.1. Summary of MIPI CSI/DSI Transmitter Registers	50
Table 5.2. STATUS_REG (0x000) Register	50
Table 5.2. STATUS_REG (0x000) Register	50
Table 5.3. CONVERTER_STATUS_REG1 (0x044) Register.....	50
Table 5.4. PHY_CONFIG_REG (0x210) Register.....	51
Table 5.5. CONFIG_REG0 (0x220) Register	51
Table 5.6. CONFIG_REG1 (0x240) Register	52
Table 5.7. CONFIG_REG2 (0x244) Register	52
Table 5.8. Summary of MIPI CSI/DSI Receiver Registers	53
Table 5.9. Soft D-PHY STATUS_REG (0x000) Register	53
Table 5.10. Hard D-PHY STATUS_REG (0x000) Register.....	53
Table 5.11. CONTROLLER_STATUS_REG (0x020) Register	54
Table 5.12. CONVERTER_STATUS_REG (0x040) Register.....	54
Table 5.13. CONFIG_REG (0x200) Register	54
Table 5.14. Soft D-PHY PHY_CONFIG_REG (0x210) Register	55
Table 5.15. Hard D-PHY PHY_CONFIG_REG (0x210) Register	55
Table 6.1. Generated File List	58
Table 7.1. MIPI CSI/DSI IP Configuration Supported by the Example Design	64
Table 7.2. Clocking Scheme in the Example Design.....	71
Table 7.3. Reset Scheme in the Example Design	71
Table 7.4. Main API	72
Table 7.5. Primary I/O Interface Signals for the mipi_interopt_ed_top Module	73
Table 7.6. SW1 DIP Switch Settings	75
Table A.1. MIPI CSI/DSI Transmitter Resource Utilization for the LFCPNX-100-9LFG672C Device ¹	90
Table A.2. MIPI CSI/DSI Receiver Resource Utilization for the LFCPNX-100-9LFG672C Device ¹	91
Table A.3. MIPI CSI/DSI Transmitter Resource Utilization for the LIFCL-40-CABGA400 Device ¹	93
Table A.4. MIPI CSI/DSI Receiver Resource Utilization for the LIFCL-40-CABGA400 Device ¹	94

Abbreviations in This Document

A list of abbreviations used in this document.

Abbreviation	Definition
AHB	Advanced High-performance Bus
APB	Advanced Peripheral Bus
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
AXI4-Lite	Advanced eXtensible Interface 4-Lite
AXI4-Stream	Advanced eXtensible Interface 4-Stream
B2P	Byte-to-Pixel
BGGR	Bayer Color Filter Array Pattern (Blue-Green-Green-Red)
BPC	Bits per Color
BPP	Bits per Pixel
CFA	Color Filter Array
CIL	Camera Interface Layer
CPP	Color per Pixel
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
CSI-2	Camera Serial Interface-2
CSR	Control and Status Register
D-PHY	Digital PHY (Physical Layer)
DDR	Double Data Rate (Serializer/Deserializer)
DIP	Dual In-line Package
DSI-2	Display Serial Interface-2
EBR	Embedded Block Random Access Memory (RAM)
ECC	Error Correction Code
EoT	End of Transmission
FAQ	Frequently Asked Questions
FFC/FPC	Flexible Flat Cable/Flexible Printed Circuit
FHD	Full High Definition
FIFO	First In First Out
FPGA	Field-Programmable Gate Array
FS	Frame Start
FSM	Finite State Machine
GBRG	Bayer Color Filter Array Pattern (Green-Blue-Red-Green)
GPIO	General Purpose Input/Output
GRBG	Bayer Color Filter Array Pattern (Green-Red-Blue-Green)
GUI	Graphical User Interface
HDL	Hardware Description Language
HDMI	High-Definition Multimedia Interface
HS	High-Speed
I2C	Inter-Integrated Circuit
IBIS	I/O Buffer Information Specification
IDDRX4	Input Double Data Rate ×4
IoT	Internet of Things
IP	Intellectual Property
IP-XACT	IP Packaging Standard (IEEE 1685)
JTAG	Joint Test Action Group (debug interface)

Abbreviation	Definition
LE	Line End
LED	Light Emitting Diode
LMMI	Lattice Memory Mapped Interface
LP	Low Power
LS	Line Start
LSE	Lattice Synthesis Engine
LUT	Look-up Table
MAP	Mapping Stage During FPGA Implementation
MBSI	MIPI Byte Streaming Interface
MC	Microcontroller
MEM	Memory Initialization File
MIPI	Mobile Industry Processor Interface
MPPT	Multi-Packet Per Transaction
ODDRX4	Output Double Data Rate x4
OS	Operating System
P2B	Pixel-to-Byte
PCIe	Peripheral Component Interconnect Express
PDC	Physical Design Constraint
PHY	Physical Layer
PLL	Phase-Locked Loop
PPC	Pixel per Clock
PPI	PHY-Protocol Interface
RAW10	10-bit Raw Bayer Format
RAW12	12-bit Raw Bayer Format
RGB565	16-bit RGB Color Format (5-6-5)
RGB888	24-bit RGB Color Format (8-8-8)
RISC-V	Reduced Instruction Set Computer V
RGGB	Bayer Color Filter Array Pattern (Red-Green-Green-Blue)
ROM	Read-Only Memory
RTL	Register Transfer Level
RX	Receiver
SCL	Serial Clock Line
SDA	Serial Data Line
SDC	Synopsys Design Constraints
SOF	Start of Frame
SoT	Start of Transmission
TX	Transmitter
UART	Universal Asynchronous Receiver/Transmitter
UI	Unit Interval
ULPS	Ultra Low Power State
USB	Universal Serial Bus
UVSI	Unified Video Streaming Interface
VC	Virtual Channel
VCD	Value Change Dump
VCX	Virtual Channel Extension
WC	Word Count

1. Introduction

The Lattice™ MIPI® CSI/DSI IP provides a flexible solution for transmission and reception of high-speed video data over a Mobile Industry Processor Interface (MIPI)-compatible D-PHY interface in FPGA designs. MIPI enables high-speed, low-power video transmission between components such as cameras and displays, and is widely used in smartphones, automotive systems, industrial equipment, and Internet of Things (IoT) devices.

1.1. Overview of the IP

The Lattice MIPI CSI/DSI IP transmitter supports the Camera Serial Interface-2 (CSI-2) protocol while the receiver supports CSI-2 and Display Serial Interface-2 (DSI-2) protocols. The IP operates in both high-speed and low power modes and supports up to 1,500 Mbps per lane with Soft D-PHY implementation and up to 2,500 Mbps per lane with Hard D-PHY implementation.

In transmitter mode, video data received from the upstream pipeline can arrive through either the Unified Video Streaming Interface (UVSI) or the MIPI Byte Streaming Interface (MBSI).

When sourced from UVSI, the pixel-based video data is first converted into byte-aligned format by the Pixel-to-Byte (P2B) block. In contrast, data received through MBSI is already byte-formatted and is routed directly to the controller without P2B conversion. The processed byte stream is then handled by the controller, serialized, and transmitted through the PHY for seamless delivery to the connected receiver.

In receiver mode, incoming video data is received through the PHY, processed by the controller, and converted through the Byte-to-Pixel (B2P) block into UVSI or MBSI packets for seamless integration with downstream video pipelines.

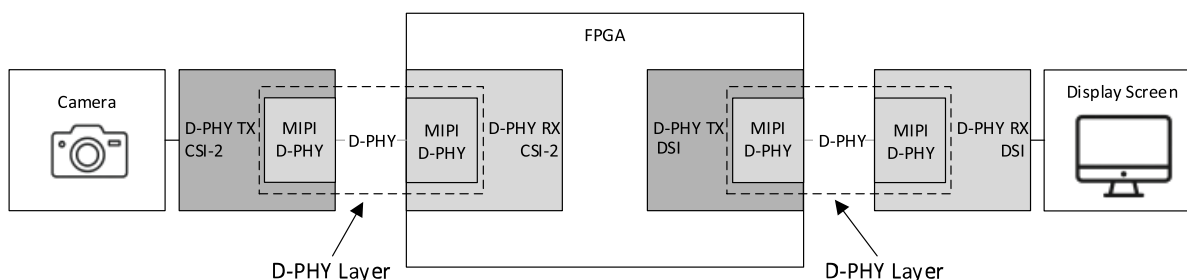


Figure 1.1. MIPI CSI/DSI IP Overview

1.2. Quick Facts

Table 1.1. Summary of the MIPI CSI/DSI IP

IP Requirements	Supported Devices	CrossLink™-NX, Certus™-NX, CertusPro™-NX, MachXO5™-NX
	IP Changes ¹	Refer to the MIPI CSI/DSI IP Release Notes (FPGA-RN-02112) .
Resource Utilization	Supported User Interface	UVSI AXI4-Stream, MBSI AXI4-Stream, AXI4-Lite
	Resources	Refer to the Resource Utilization section.
Design Tool Support	Lattice Implementation	IP Core v4.2.0 – Lattice Radiant™ Software 2026.1
	Synthesis	Synopsys® Synplify Pro for Lattice
	Simulation	Refer to the Lattice Radiant software user guide for the list of supported simulators.

Note:

1. In some instances, the IP may be updated without changes to the user guide. This user guide may reflect an earlier IP version but remains fully compatible with the later IP version. Refer to the IP Release Notes for the latest updates.

1.3. IP Support Summary

Table 1.2. MIPI CSI/DSI IP Support Readiness

Device Family	PHY Direction	PHY Mode	Protocol Mode	Line Rate Per Lane (Mbps)	Radiant Timing Model
CrossLink-NX, Certus-NX, CertusPro-NX, MachXO5-NX	TX	Soft D-PHY	CSI-2	160–1500	Final
		Hard D-PHY ¹		320–2500	Final
	RX	Soft D-PHY	CSI-2 and DSI-2	80–1500	Final
		Hard D-PHY ¹		160–2500	Final

Note:

1. Hard D-PHY is available only for CrossLink-NX devices.

The example design for the MIPI CSI/DSI IP allows for simulation and deployment to development boards for testing. Refer to the [Example Design](#) section for more details on how to run the example design on hardware and simulation.

1.4. Features

Key features of the MIPI CSI/DSI IP include:

- Supports the MIPI D-PHY protocol with 1, 2, or 4 data lanes and 1 clock lane.
- Supports Soft D-PHY bandwidth up to 1.5 Gbps per lane, for a total bandwidth of 6 Gbps. See [Table 1.2](#).
- Supports Hard D-PHY bandwidth up to 2.5 Gbps per lane, for a total bandwidth of 10 Gbps. See [Table 1.2](#).
- Supports continuous and non-continuous MIPI D-PHY clock operation.
- Supports skew calibration.
- Supports multiple color formats:
 - RGB888
 - RGB565
 - RAW10
 - RAW12
- Supports UFSI for pixel-domain transmission and reception.
- Supports MBSI for byte-domain transmission and reception.
- Supports AXI4-Lite interface register access.
- Supports dynamic lane reconfiguration.

1.4.1. MIPI CSI/DSI IP on Transmitter Mode

- Supports the MIPI CSI-2 protocol.
 - Supports generation of Frame Start and Frame End with frame count.
 - Optionally supports the generation of Line Start and Line End with optional Line Count.
- Supports 1, 2, and 4 pixels per clock (PPC).

1.4.2. MIPI CSI/DSI IP on Receiver Mode

- Supports MIPI CSI-2 and DSI-2 protocols.
- Supports all MIPI DSI video modes:
 - Non-burst mode with sync pulses.
 - Non-burst mode with sync events.
 - Burst mode.
- Supports 1, 2, and 4 pixels per clock.

1.5. Licensing and Ordering Information

The MIPI CSI/DSI IP is provided at no additional cost with the Lattice Radiant software.

1.6. Minimum Device Requirements

Refer to the [Resource Utilization](#) section for the minimum required resources to instantiate this IP.

1.7. Naming Conventions

1.7.1. Nomenclature

The nomenclature used in this document is based on Verilog HDL.

1.7.2. Signal Names

Signal names that end with:

- `_n` are active low signals (asserted when value is logic 0)
- `_i` are input signals
- `_o` are output signals
- `_io` are bidirectional signals

1.7.3. Attribute Names

Attribute names in this document are formatted in title case and italicized (*Attribute Name*).

2. Functional Description

2.1. IP Architecture Overview

The following figures show the architecture blocks and data flow in the MIPI CSI/DSI IP. Each block may be used depending on the IP attribute settings.

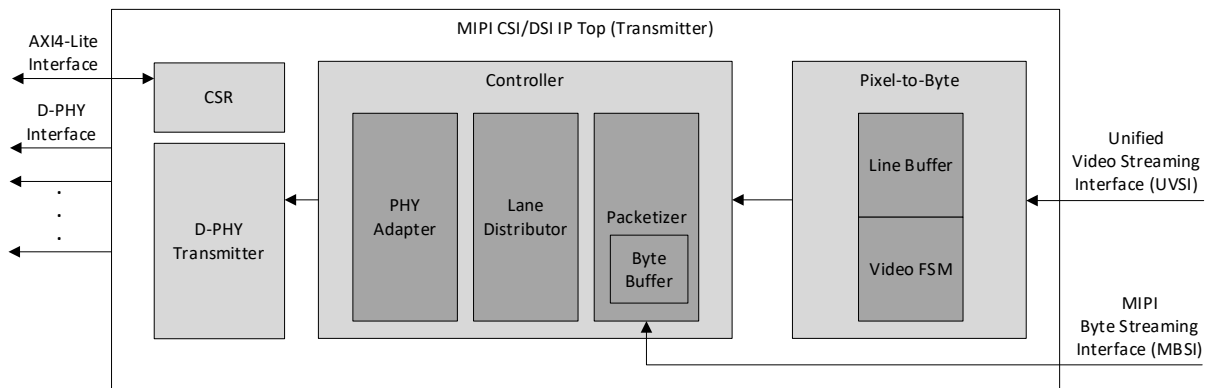


Figure 2.1. MIPI CSI/DSI IP Transmitter Core Block Diagram

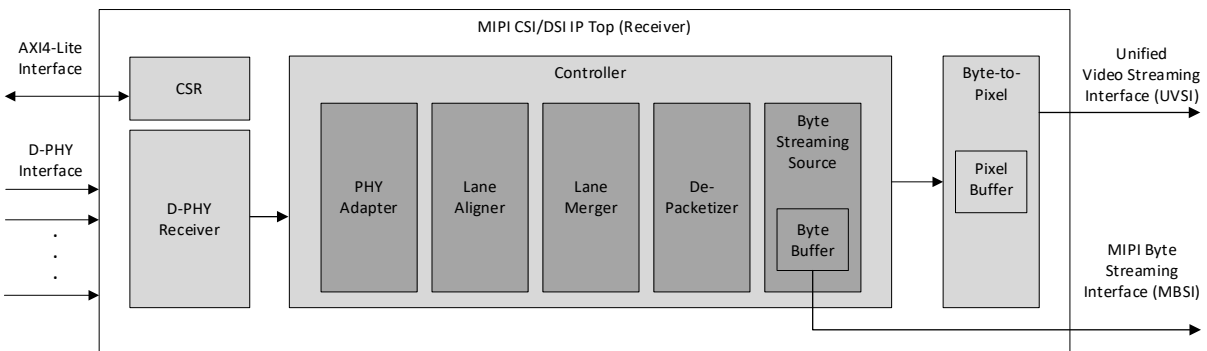


Figure 2.2. MIPI CSI/DSI IP Receiver Core Block Diagram

The MIPI CSI/DSI IP includes the following layers:

- Transmitter
 - Control and Status Register (CSR) block
 - PHY block
 - Controller block
 - Pixel-to-Byte block
 - Video Finite State Machine (FSM)
- Receiver
 - CSR block
 - PHY block
 - Controller block
 - Byte-to-Pixel block

2.2. Clocking

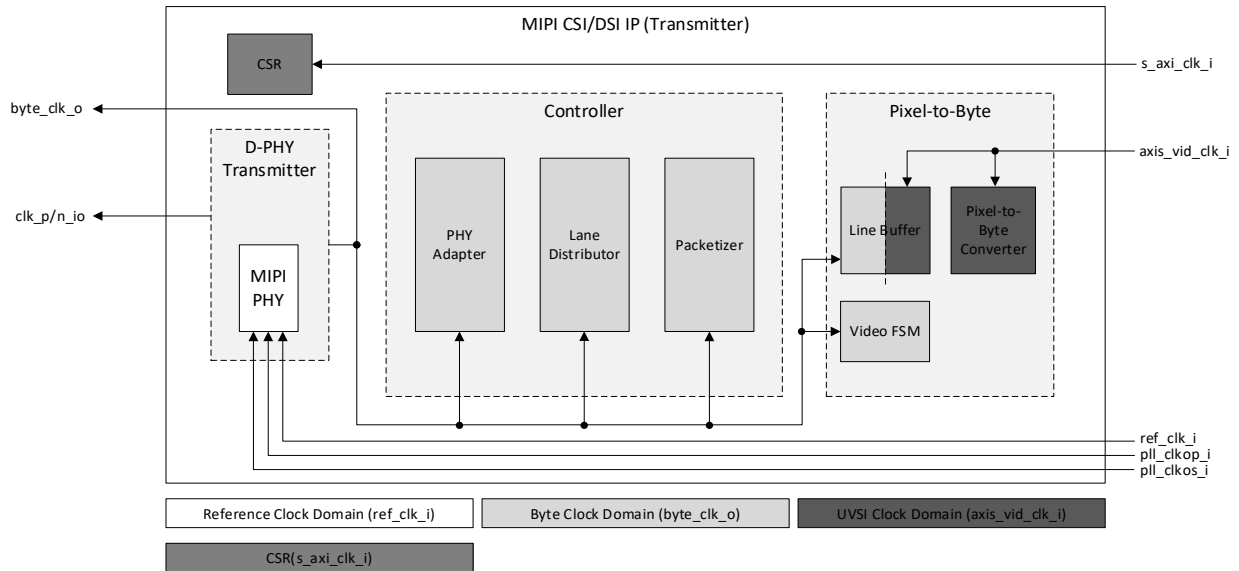


Figure 2.3. MIPI CSI/DSI IP Transmitter Clock Domain Block Diagram (UVSI)

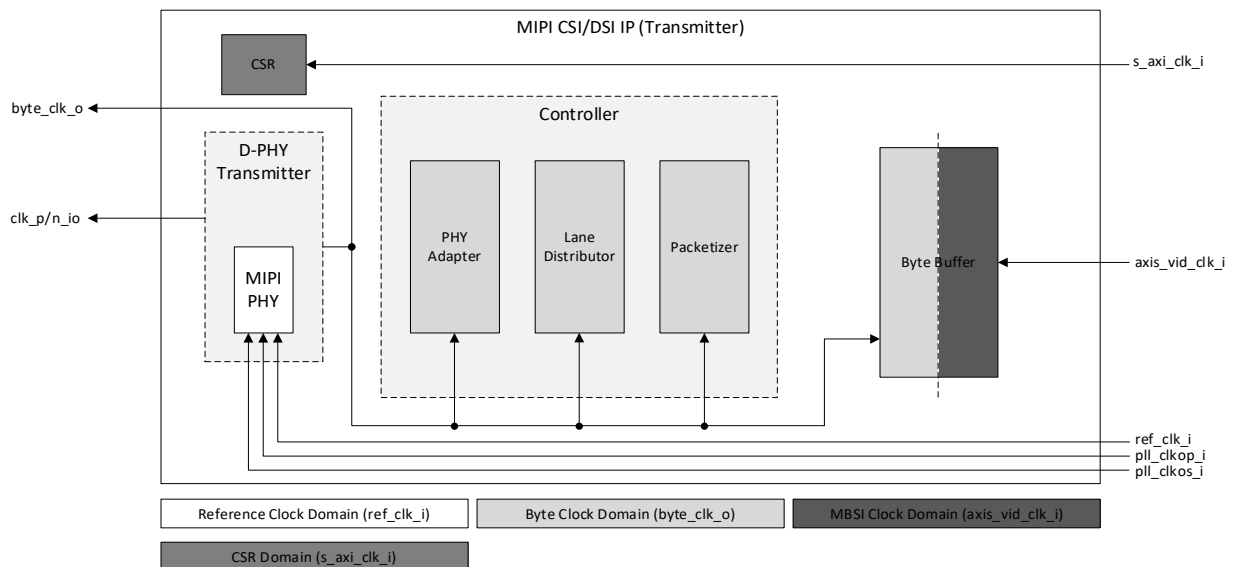


Figure 2.4. MIPI CSI/DSI IP Transmitter Clock Domain Block Diagram (MBSI)

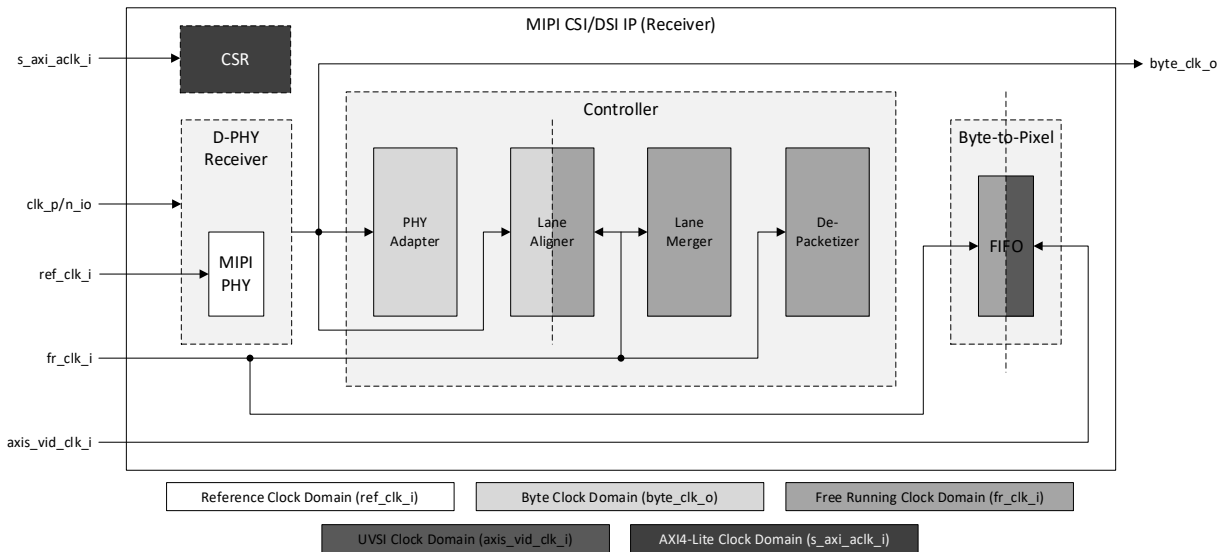


Figure 2.5. MIPI CSI/DSI IP Receiver Clock Domain Block Diagram (UVSI)

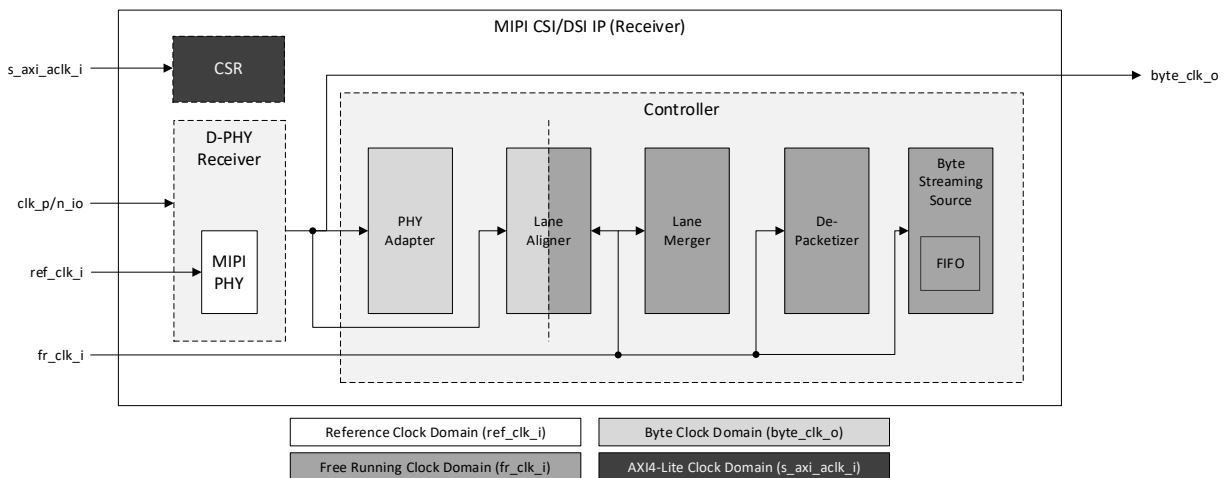


Figure 2.6. MIPI CSI/DSI IP Receiver Clock Domain Block Diagram (MBSI)

2.2.1. MIPI CSI/DSI Transmitter Clocking Overview

Clocks shown in [Figure 2.3](#) and [Figure 2.4](#) are typically generated using an external PLL. For more details of each clock, refer to the [Signal Description](#) section.

Table 2.1. MIPI CSI/DSI Transmitter Operating Frequency Details and Requirements

Clock	Operating Frequency Details and Requirements
ref_clk_i	The operating frequency of this clock can be set using the <i>Reference Clock Frequency (MHz)</i> attribute. Supported values are 60 MHz and up to 200 MHz, depending on the device configuration.
pll_clkop_i	This is the 0-degree phase output of an external PLL. This clock depends on the <i>Line Rate Per Lane (Mbps)</i> . Example: $\text{pll_clkop_i} = (\text{Line Rate Per Lane (Mbps)} / 2)$ For a configured line rate of 1000 Mbps per lane

Clock	Operating Frequency Details and Requirements
	$\text{pll_clkop_i} = (1000/2) = 500 \text{ MHz}$
pll_clkos_i	This clock operates at the same frequency as pll_clkop_i , but 90-degree phase shifted.
$\text{byte_clk_o}^{1,2}$	The frequency of this clock can be calculated as: <i>Actual Byte Clock Frequency (MHz) = Line Rate Per Lane (Mbps)/PHY Data Width</i> . Byte clock is generated by gearing down pll_clkop_i and operates as a divided version of the PLL output clock.
axis_vid_clk_i	When <i>Unified Video Streaming Interface (UVSI)</i> is checked: The axis_vid_clk_i frequency must be selected such that the average input pixel rate does not exceed the effective throughput of the MIPI transmitted over time. If axis_vid_clk_i is higher than the recommended frequency from the formula below, the upstream provides pixel data faster than the IP can transfer the data into the byte domain. To avoid FIFO overflow, sufficient horizontal blanking and/or line buffer depth must be provided. Otherwise, the IP asserts backpressure on the UVSI interface. The operating frequency is recommended to be derived based on the following formula to ensure correct and reliable operation. $\text{axis_vid_clk_i}_{\text{MHz}} = ((\text{Number of D-PHY Lanes} \times \text{PHY Data Width}) / (\text{Number of Pixel per Clock (PPC)} \times \text{BPP}^1)) \times \text{Actual Byte Clock Frequency (MHz)}$ Example 1: <i>Line Rate Per Lane (Mbps)</i> = 1,500 <i>PHY Data Width</i> = 8 <i>Number of D-PHY Lanes</i> = 4 <i>Number of Pixel per Clock (PPC)</i> = 4 <i>Enable RGB888 Color Format</i> = checked (BPP = 24) <i>Actual Byte Clock Frequency (MHz)</i> = 187.5 MHz $\text{axis_vid_clk_i}_{\text{MHz}} = ((4 \times 8) / (4 \times 24)) \times 187.5 = 62.5 \text{ MHz}$ When <i>MIPI Byte Streaming Interface (MBSI)</i> is checked: The operating frequency is recommended to be equal to <i>Actual Byte Clock Frequency (MHz)</i> to ensure correct and reliable operation. When axis_vid_clk_i is higher than the <i>Actual Byte Clock Frequency (MHz)</i>, sufficient horizontal blanking and/or byte buffer depth must be provided to prevent buffer overflow.
s_axi_aclk_i	The maximum operating frequency of this clock is 60 MHz.

Notes:

1. The maximum data rate depends on the family, package, and performance grade of the device. Check the device data sheet for more information.
2. For edge clock sharing ports, refer to the [MIPI CSI/DSI Transmitter Edge Clock Sharing](#) section.

2.2.1.1. MIPI CSI/DSI Transmitter Edge Clock Sharing

This edge clock sharing mode is intended for designs that instantiate multiple Soft D-PHY TX blocks within the same bank and need to conserve edge clock resources, provided all instances operate at the same bit rate. Edge clock is the high-speed per-bank clock network in Lattice FPGAs that drives DDR I/O elements and synchronizes PHY data transitions. The Soft D-PHY TX uses FPGA DDR elements together with edge clock synchronizer and divider resources to generate the edge-aligned transmit clock. When *Edge Clock Mode* is External, the IP does not instantiate the edge clock synchronizer and divider. Instead, the ports required to drive the DDR path such as the byte clock, edge clock sync clock, reset, and readiness indicators, are exposed as top-level inputs. These signals must be connected to the corresponding outputs of another D-PHY TX instance that sets *Edge Clock Mode* as Internal, provides the synchronizer and divider, and serves as the primary clock source.

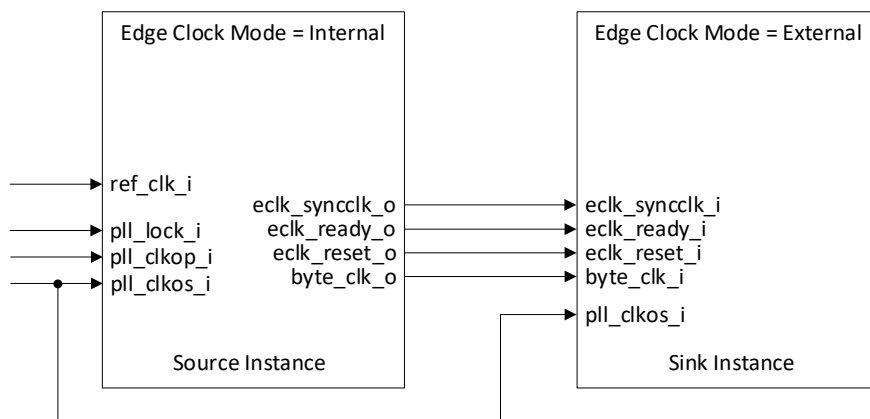


Figure 2.7. Edge Clock Sharing Port Connection

2.2.2. MIPI CSI/DSI Receiver Clocking Overview

Input clocks shown in Figure 2.5 and Figure 2.6 are typically generated using an external PLL while the output clock, byte_clk_o, is generated by an FPGA clock divider element. For more details of each clock, refer to the [Signal Description](#) section.

Table 2.2. MIPI CSI/DSI Receiver Operating Frequency Details and Requirements

Clock	Operating Frequency Details and Requirements
ref_clk_i	The operating frequency of this clock can be set using the <i>Reference Clock Frequency (MHz)</i> attribute. Supported values are 60 MHz and up to 200 MHz, depending on the device configuration.
byte_clk_o	The frequency of this clock can be calculated as: <i>Actual Byte Clock Frequency (MHz) = Line Rate Per Lane (Mbps)/PHY Data Width</i> .
fr_clk_i	The operating frequency of this clock can be set to greater than or equal to <i>Actual Byte Clock Frequency (MHz)</i> , with a minimum value of 60 MHz.
axis_vid_clk_i	The operating frequency must be computed as follows, with minimum of <i>Actual Byte Clock Frequency (MHz)</i>: $\text{axis_vid_clk_i}_{\text{MHz_TEMP}} = ((\text{Number of D-PHY Lanes} \times \text{PHY Data Width}) / (\text{Number of Pixel per Clock (PPC)} \times \text{BPP}^1)) \times \text{Actual Byte Clock Frequency (MHz)}$ If axis_vid_clk_i_MHz_TEMP < Actual Byte Clock Frequency (MHz): - axis_vid_clk_i_MHz ≥ Actual Byte Clock Frequency (MHz) Else: - axis_vid_clk_i_MHz ≥ axis_vid_clk_i_MHz_TEMP Example 1: Line Rate Per Lane (Mbps) = 1,500 PHY Data Width = 8 Number of D-PHY Lanes = 4 Number of Pixel per Clock (PPC) = 4 Enable RGB888 Color Format = checked (BPP = 24) Actual Byte Clock Frequency (MHz) = 187.5 MHz $\text{axis_vid_clk_i}_{\text{MHz_TEMP}} = ((4 \times 8) / (4 \times 24)) \times 187.5 = 62.5 \text{ MHz}$ As 62.5 MHz < 187.5 MHz, axis_vid_clk_i_MHz ≥ 187.5 MHz

Clock	Operating Frequency Details and Requirements
	Example 2: Line Rate Per Lane (Mbps) = 500 PHY Data Width = 8 Number of D-PHY Lanes = 4 Number of Pixel per Clock (PPC) = 1 Enable RAW10 Color Format = checked (BPP = 10) Actual Byte Clock Frequency (MHz) = 62.5 MHz $\text{axis_vid_clk_iMHz_TEMP} = ((4 \times 8) / (1 \times 10)) \times 62.5 = 200 \text{ MHz}$ As 200 MHz > 62.5 MHz, $\text{axis_vid_clk_iMHz} \geq 200 \text{ MHz}$
s_axi_aclk_i	When PHY Mode is Soft D-PHY, the operating frequency of this clock can be up to the fabric frequency limit of 200 MHz. When PHY Mode is Hard D-PHY, the operating frequency of this clock is limited to a maximum of 60 MHz.

2.3. Reset

Resets of the IP are defined in the [Signal Description](#) section.

2.3.1. Reset Overview

When the clocks are generated by an external PLL, it is recommended to keep all associated resets active until the PLL achieves lock and the input clocks are stable as shown in [Figure 2.8](#) and [Figure 2.9](#).

There is no required reset sequence. However, all resets must be asserted together if a reset occurs during an active transaction. This ensures that any residual data inside internal FIFOs or pipelines is completely flushed. Failure to reset all relevant domains simultaneously during an in-transaction reset may leave stale data in the data path, which can lead to an IP hang or unexpected behavior.

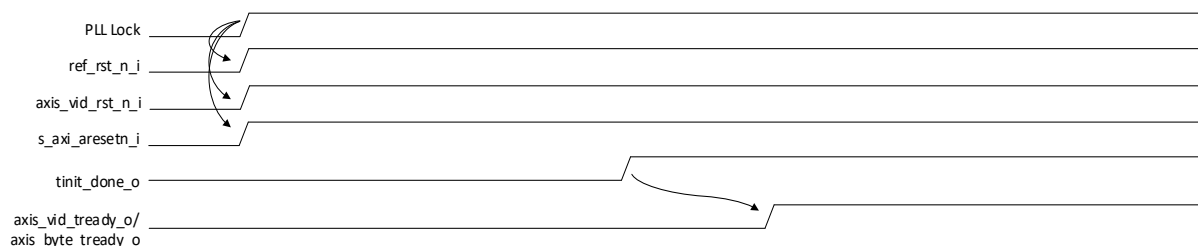


Figure 2.8. MIPI CSI/DSI IP Transmitter Reset Sequence Waveform

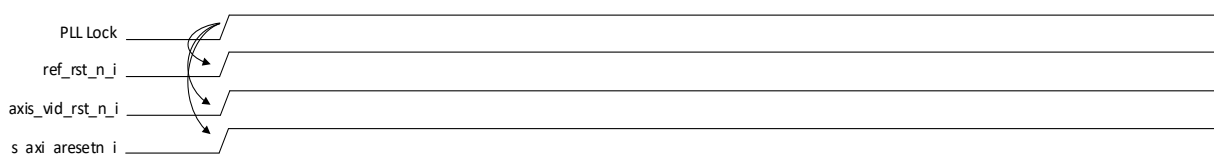


Figure 2.9. MIPI CSI/DSI IP Receiver Reset Sequence Waveform

2.4. User Interfaces

The table below lists the available user interface and protocols used on the MIPI CSI/DSI IP.

Table 2.3. User Interfaces and Supported Protocols

User Interface	Supported Protocols	Description
Device receiver	Unified Video Streaming Interface	This MIPI CSI/DSI transmitter interface accepts video data in the pixel domain and follows the standard

User Interface	Supported Protocols	Description
		AXI4-Stream protocol for transferring pixel information into the transmitter pipeline.
	MIPI Byte Streaming Interface	This MIPI CSI/DSI transmitter interface receives data in byte format and follows the standard AXI4-Stream protocol.
Device transmitter	Unified Video Streaming Interface	This MIPI CSI/DSI receiver interface transmits data in pixel domain and follows the standard AXI4-Stream protocol.
	MIPI Byte Streaming Interface	This MIPI CSI/DSI receiver interface transmits data in byte domain and follows the standard AXI4-Stream protocol.

2.4.1. Unified Video Streaming Interface (UVSI)

The UVSI transfers video data in the pixel domain and complies with the AXI4-Stream protocol. For the complete list of interface signals, refer to the [Signal Description](#) section.

In UVSI, the first transmission of every frame is always the frame boundary, indicated by the assertion of the Start of Frame (SOF) signal. This transition marks the beginning of the valid video-frame data. The data flow strictly follows the AXI4-Stream handshaking requirements, where the source must assert TVALID only when valid data is available, and the sink must assert TREADY only when the sink is ready to accept data. A data beat is transferred only when both TVALID and TREADY are high, ensuring proper flow control and preventing data loss or corruption.

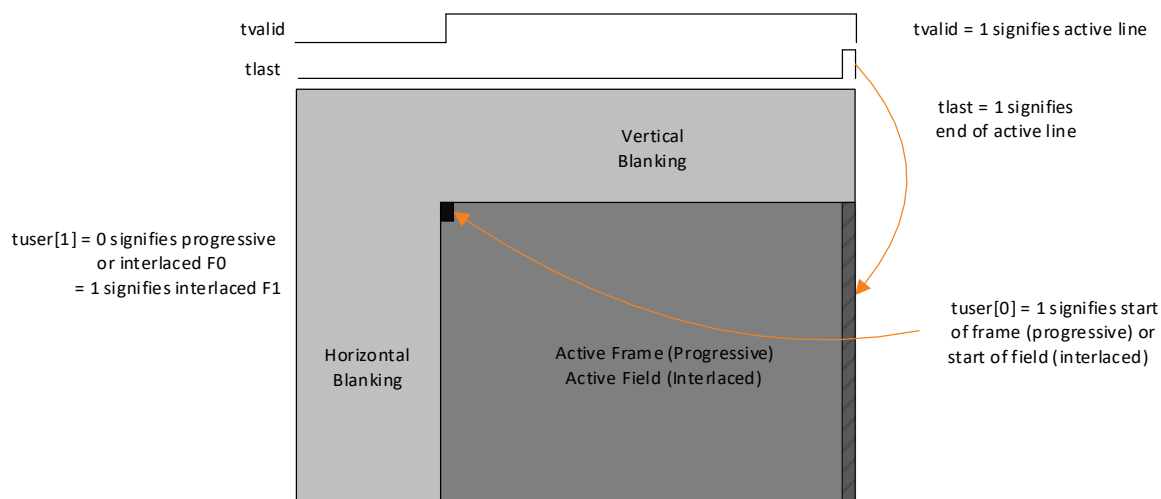


Figure 2.10. UVSI to Video Frame Timing Mapping

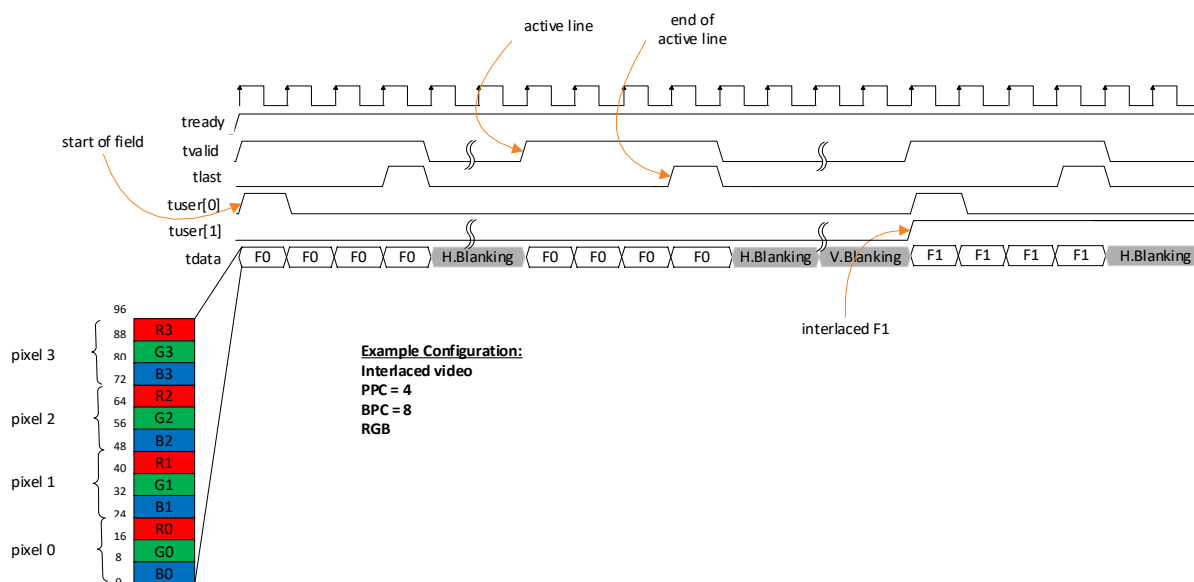


Figure 2.11. Unified Video Streaming Timing Diagram

The fundamental signaling rules of UVSI are consistent for both the TX and RX paths. Data movement is governed by the same AXI4-Stream based handshake, ensuring reliable and flow-controlled transfer between the source and the sink. These common rules ensure predictable behavior whether the IP is transmitting or receiving video frames.

2.4.1.1. TDATA Mapping

Key definitions in TDATA mapping are as follows:

- Colors per Pixel (CPP)—Number of color components per pixel. For example, 1 for RAW, 3 for RGB.
- Bits per Color (BPC)—Configured bit width for each color component.
- Bits per Pixel (BPP)—Total bits per pixel = CPP × BPC.
- TDATA width per pixel—Width of UVSI TDATA allocated for one pixel, rounded up to the nearest byte boundary = $\text{ceil}(\text{BPP}/8) \times 8$.
- TDATA_WIDTH—Total width of the AXI4-Stream TDATA bus for all pixels in a clock cycle = TDATA width per pixel × PPC.

RGB565

- CPP: 3
- BPC: 8 (actual: 5, 6)
- BPP: $3 \times 8 = 24$ bits
- TDATA width per pixel: $\text{ceil}(24/8) \times 8 = 24$ bits

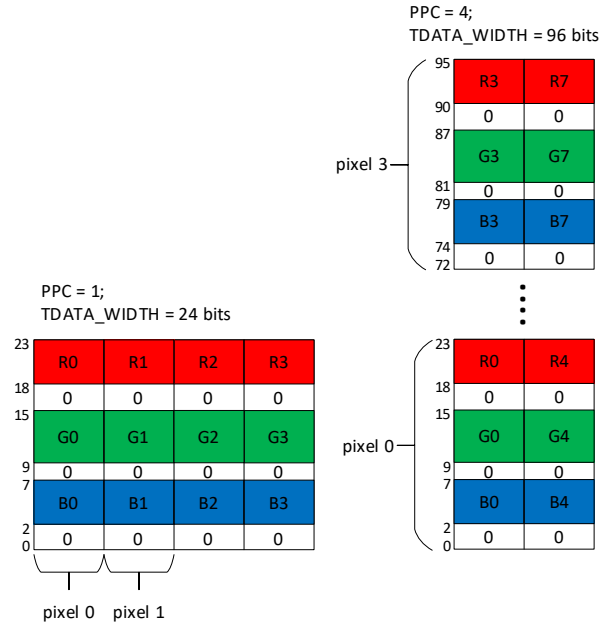


Figure 2.12. TDATA Pixel Mapping for Sample RGB565 Configuration

RGB888

- CPP: 3
- BPC: 8
- BPP: $3 \times 8 = 24$ bits
- TDATA width per pixel: $\text{ceil}(24/8) \times 8 = 24$ bits

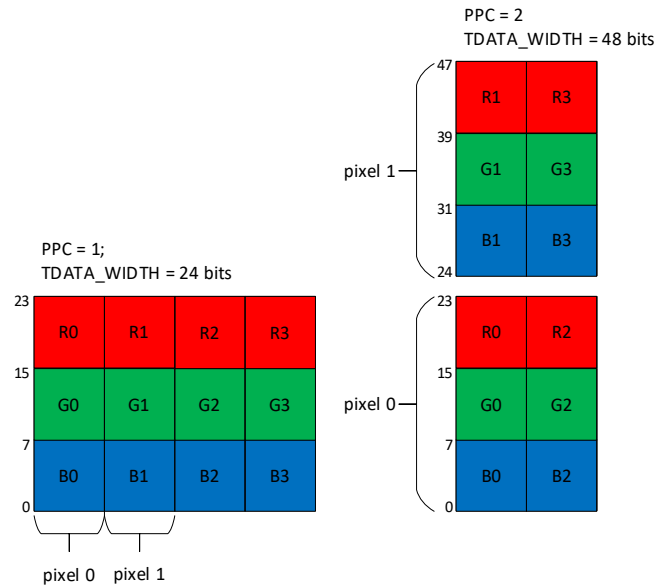


Figure 2.13. TDATA Pixel Mapping for Sample RGB888 Configuration

RAW10

- CPP: 1
- BPC: 10
- BPP: $1 \times 10 = 10$ bits
- TDATA width per pixel: $\text{ceil}(10/8) \times 8 = 16$ bits

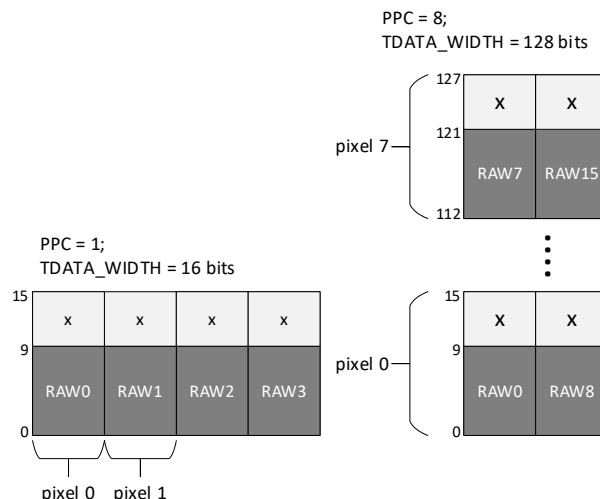


Figure 2.14. TDATA Pixel Mapping for Sample RAW10 Configuration

RAW12

- CPP: 1
- BPC: 12
- BPP: $1 \times 12 = 12$ bits
- TDATA width per pixel: $\text{ceil}(12/8) \times 8 = 16$ bits

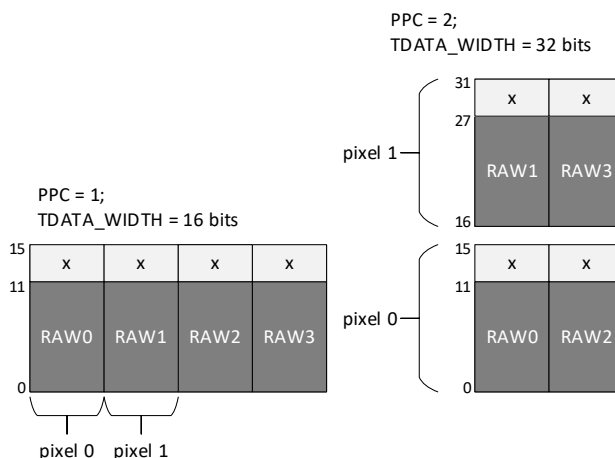


Figure 2.15. TDATA Pixel Mapping for Sample RAW12 Configuration

2.4.1.2. MIPI CSI/DSI Transmitter UVSI

Figure 2.16 illustrates the timing diagram for video frame transmission with blanking interval over UVSI during a TX transaction. In this integrated solution, the design includes a configurable line-buffer architecture, where the buffer depth can be configured according to the video line-packet size to support various resolutions and packet formats. When sufficient horizontal blanking interval is provided between consecutive line packets on UVSI, the IP has sufficient time to complete the D-PHY LP-to-HS mode switching and to flush the stored data from the line buffer, enabling smooth and continuous operation without triggering backpressure.

When horizontal blanking is minimal or absent and line packets arrive back-to-back at UVSI, the IP must simultaneously store incoming data into the line buffer while requesting the D-PHY to transition from low power to high-speed mode. This LP to HS transition is not instantaneous and must meet the D-PHY timing parameters, such as tHS-PREPARE and

tHS-ZERO, which introduce a finite delay. If additional lines continue to arrive during this transition and the buffer reaches the maximum capacity, the IP asserts backpressure on UVSI to prevent overflow and ensure reliable data transfer as shown in [Figure 2.17](#).

You can increase the available buffering margin by adjusting the *Line Buffer Depth* parameter in the IP configuration. A larger *Line Buffer Depth* allows the IP to absorb consecutive line packets without prematurely asserting backpressure.

When backpressure is asserted (*axis_vid_tready_o* = low), the upstream video source is expected to pause pixel transmission immediately until *tready* returns high. The upstream logic must be able to throttle or halt data flow on *tready* deassertion, otherwise, incoming data may be lost or misaligned.

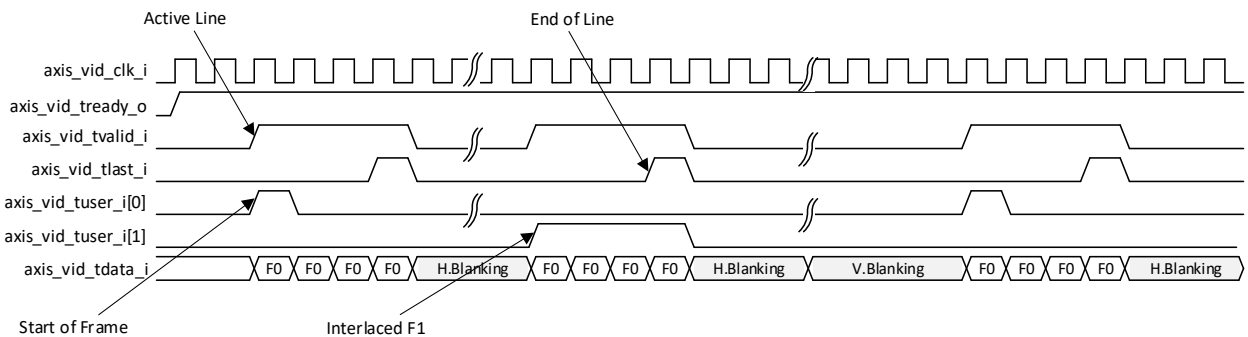


Figure 2.16. UVSI TX Timing Diagram with Blanking Interval

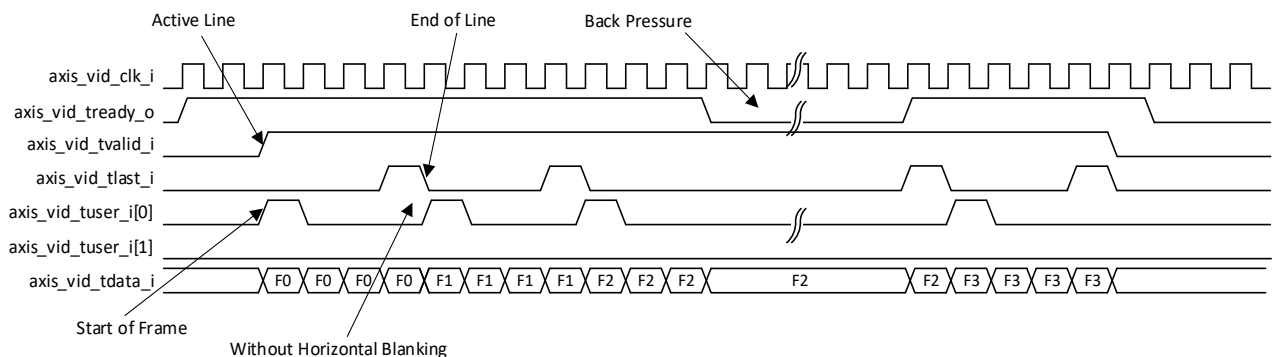


Figure 2.17. UVSI TX Timing Diagram without Blanking Interval

2.4.1.3. MIPI CSI/DSI Receiver UVSI

[Figure 2.18](#) shows the timing diagram for video frame reception over UVSI where *axis_vid_tready_i* stays asserted during an RX transaction.

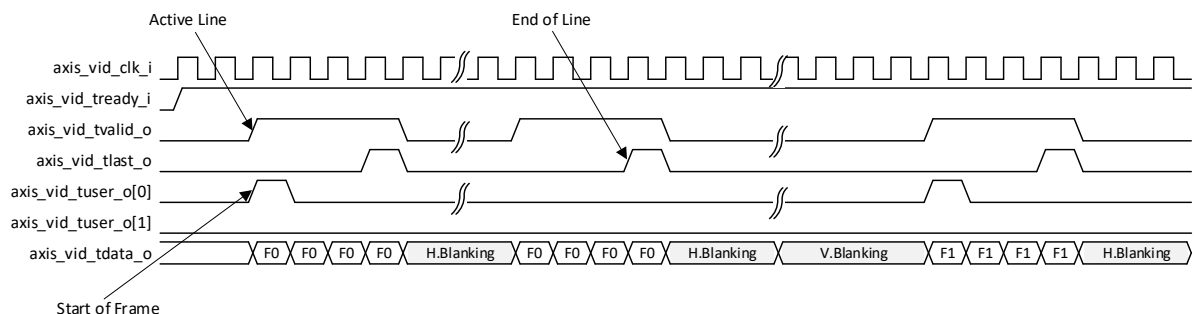


Figure 2.18. UVSI RX Timing Diagram without TREADY De-assertion

When `axis_vid_tready_i` is de-asserted (low) during an active packet transfer, as shown in Figure 2.19, pixels are stored in a pixel buffer. If `axis_vid_tready_i` stays low for a duration greater than or equal to the configured buffer depth (*Pixel Buffer Depth*), the buffer may overflow, which can cause functional errors.

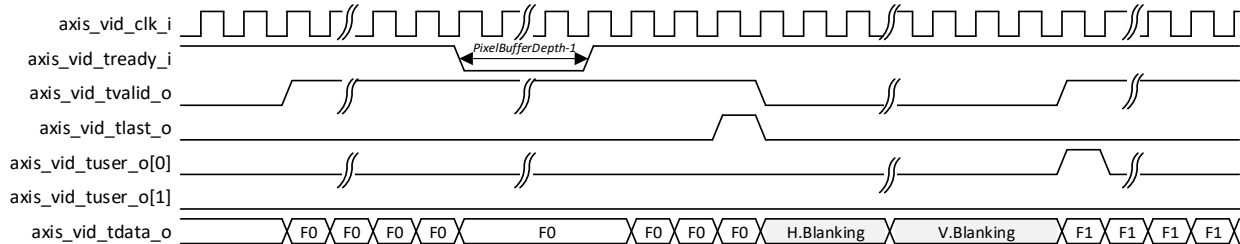


Figure 2.19. UVSI RX Timing Diagram with TREADY De-assertion

2.4.2. MIPI Byte Streaming Interface (MBSI)

2.4.2.1. MIPI CSI/DSI Transmitter MBSI

When *MIPI Byte Streaming Interface* is enabled, the transmitter is designed to forward the formatted MIPI packets supplied by upstream logic, rather than operating on pixel-level video data. Pixel-level processing functions such as pixel-to-byte conversion, video timing generation, and synchronization packet construction are not part of the default data path in this mode. With MBSI in passthrough mode, the converter module such as Pixel-to-Byte is bypassed. The transmitter accepts a byte-oriented AXI4-Stream input, where packet headers, payloads, and checksums are generated externally and driven on TVALID, TREADY, TDATA, TUSER, and TLAST. Data movement strictly follows AXI4-Stream handshaking rules, where TVALID must be asserted by the source only when valid data is present, and TREADY must be asserted by the sink when the sink is ready to accept data. Transfer occurs only when both signals are high.

The transmitter primarily performs lane distribution and PHY interfacing, mapping the incoming packets onto the configured data lanes and driving the packets onto the D-PHY through the standard PHY-Protocol Interface (PPI). This mode is suitable for applications that require transparent CSI-2 packet handling, including CSI-2 bridging, tunneling, stream aggregation, and systems where packet formation is managed externally.

To support advanced use cases, passthrough mode provides optional packet integrity and header handling features that can be individually enabled through configuration parameters. When enabled, the transmitter can recalculate and insert ECC and/or CRC values, allowing controlled modification of packet headers or payloads while still operating in a byte-stream passthrough flow.

In addition, passthrough mode is intended to support Virtual Channel (VC) and Virtual Channel Extension (VCX) usage. This mode enables upstream logic to transmit multi-stream or virtualized MIPI traffic while maintaining correct packet identification across the link.

2.4.2.2. ECC Insertion and Virtual Channel Handling of the MIPI CSI/DSI Transmitter

When ECC insertion is enabled, the transmitter overrides the ECC field of incoming CSI-2 packet headers. In this mode, the Virtual Channel (VC) and Virtual Channel Extension (VCX) information can be provided dynamically through the AXI4-Stream TUSER sideband signals. The transmitter uses this VC/VCX information to update the packet header fields and recalculates the corresponding ECC value before transmitting the packet to the PHY. When ECC insertion is enabled, the incoming packet header must still include the ECC field as part of the AXI4-Stream data format. However, the ECC value is not required to be valid, as the value is ignored and overwritten by the transmitter with a recalculated ECC.

This mechanism is useful when the upstream logic needs to modify VC or VCX values without manually regenerating the MIPI packet header.

Alternatively, if ECC insertion is disabled, the transmitter treats the packet header as fully formed. The incoming packet including the header, VC/VCX fields, and ECC is forwarded unchanged. Any VC or VCX modification must then be performed entirely by the upstream logic before placing the packet on the AXI4-Stream TDATA bus.

2.4.2.3. CRC Insertion of the MIPI CSI/DSI Transmitter

CRC handling is controlled independently of ECC insertion. When CRC insertion is enabled, the transmitter recalculates the long-packet CRC based on the received payload and appends the updated CRC value before packet transmission. This allows payload data to be modified upstream while ensuring packet integrity at the CSI-2 protocol level. When CRC insertion is enabled, the CRC field must still be present in the incoming packet, but the value is treated as don't-care and is overwritten by the transmitter. When CRC insertion is disabled, the transmitter forwards the packet payload and CRC field unmodified, preserving the original packet contents end-to-end. The upstream logic must provide a valid CRC, as the field is forwarded unchanged.

The following figures show the timing diagrams for MIPI packets reception over MBSI where `axis_byte_tready_o` stays asserted during a TX transaction for 32-bit passthrough mode in Figure 2.20 and 64-bit passthrough mode in Figure 2.21.

The full MIPI packet is driven on TDATA. TLAST asserts on the payload MSB footer (CRC[15:8]).

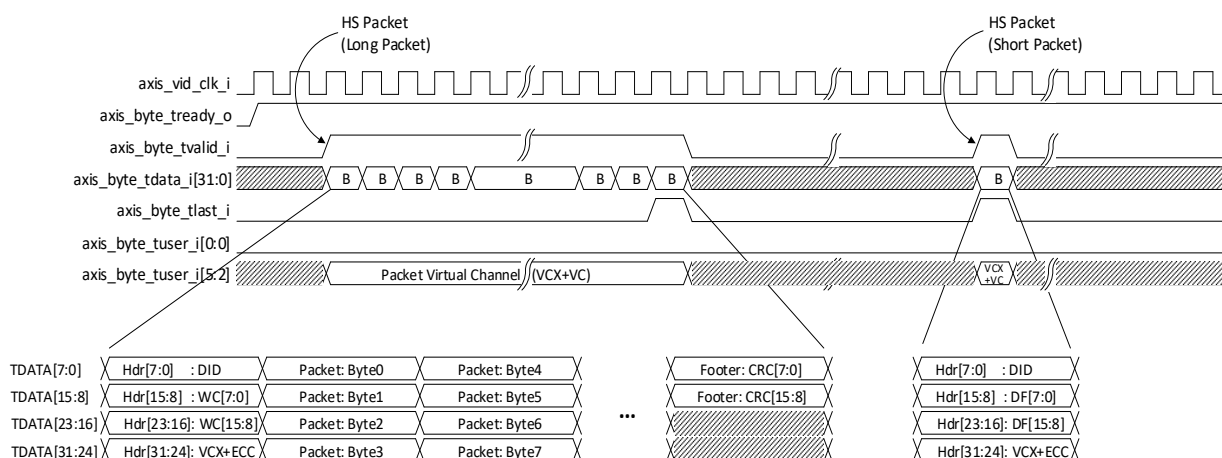


Figure 2.20. MBSI TX Timing Diagram for 32-bit Passthrough Mode

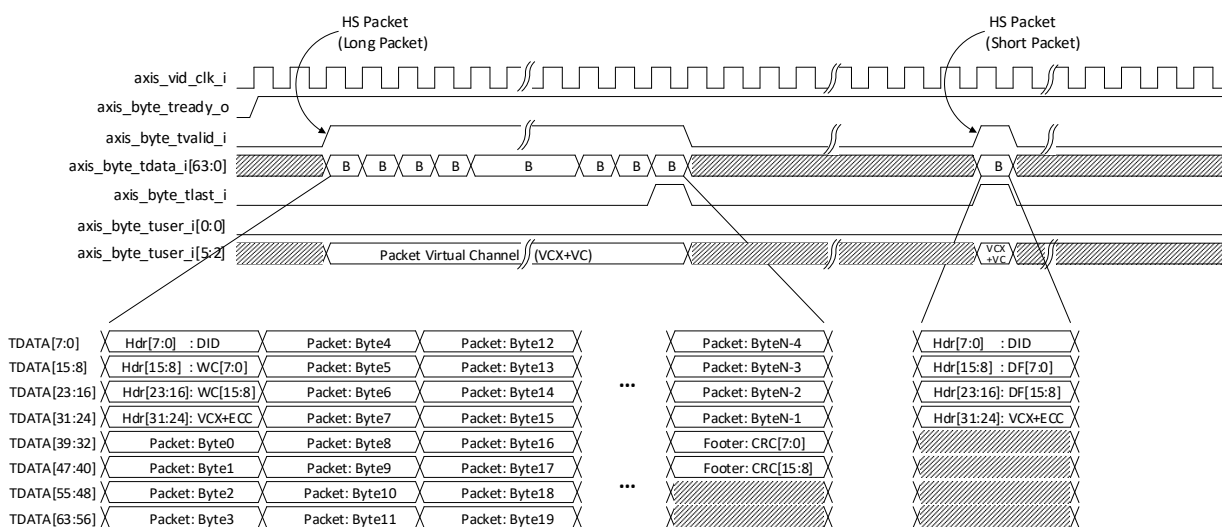


Figure 2.21. MBSI TX Timing Diagram for 64-bit Passthrough Mode

When *MIPI Byte Streaming Interface* is enabled, a byte buffer (dual clock FIFO) is used to hold packets when downstream is not ready (*axis_byte_tready_o* is low). FIFO depth is configurable through the *Byte Buffer Depth* attribute. For multiple back-to-back packet, a full buffer leads to backpressure as shown in Figure 2.22. Byte buffer depth must be configured to store a full long packet. The byte buffer FIFO stores the complete MIPI packet, including the packet header, payload, and CRC bytes. Therefore, the maximum supported payload size is equal to the configured packet size minus 8 bytes.

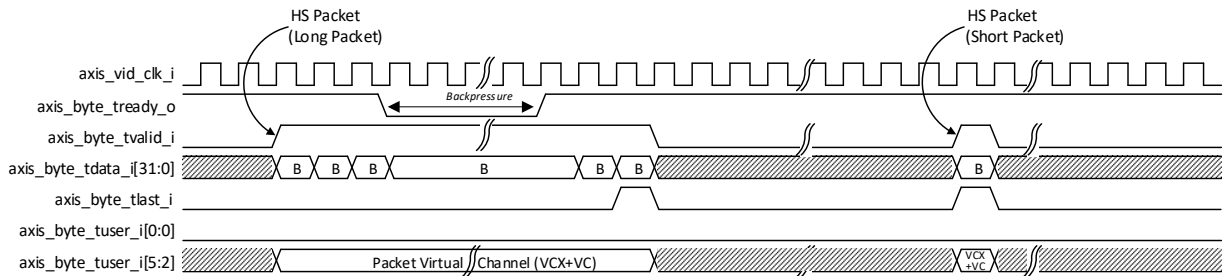


Figure 2.22. MBSI TX Timing Diagram for 32-bit Passthrough Mode with Backpressure

2.4.2.4. MIPI CSI/DSI Receiver MBSI

On the receiver side, when *MIPI Byte Streaming Interface* is enabled, the IP converts all MIPI short and long packets into AXI4-Stream in the byte domain and handles backpressure. This conversion includes every packet type such as NULL and BLANKING packets whenever the packets are present.

The IP maps the internal MIPI packet information (header, payload bytes, footer) to the corresponding AXI4-Stream signals: TVALID, TREADY, TDATA, TUSER, and TLAST. Data movement strictly follows AXI4-Stream handshaking rules, where TVALID must be asserted by the source only when valid data is present, and TREADY must be asserted by the sink when the sink is ready to accept data. Transfer occurs only when both signals are high.

You must implement any handling specific to packet type, such as identifying long or short packets or determining the footer byte position on a 32 or 64-bit TDATA bus.

With MBSI in passthrough mode, converter IPs such as the Byte-to-Pixel IP are bypassed.

Figure 2.23 and Figure 2.24 show the timing diagram for MIPI packets reception over MBSI where *axis_byte_tready_i* stays asserted during an RX transaction for 32-bit passthrough mode and 64-bit passthrough mode respectively.

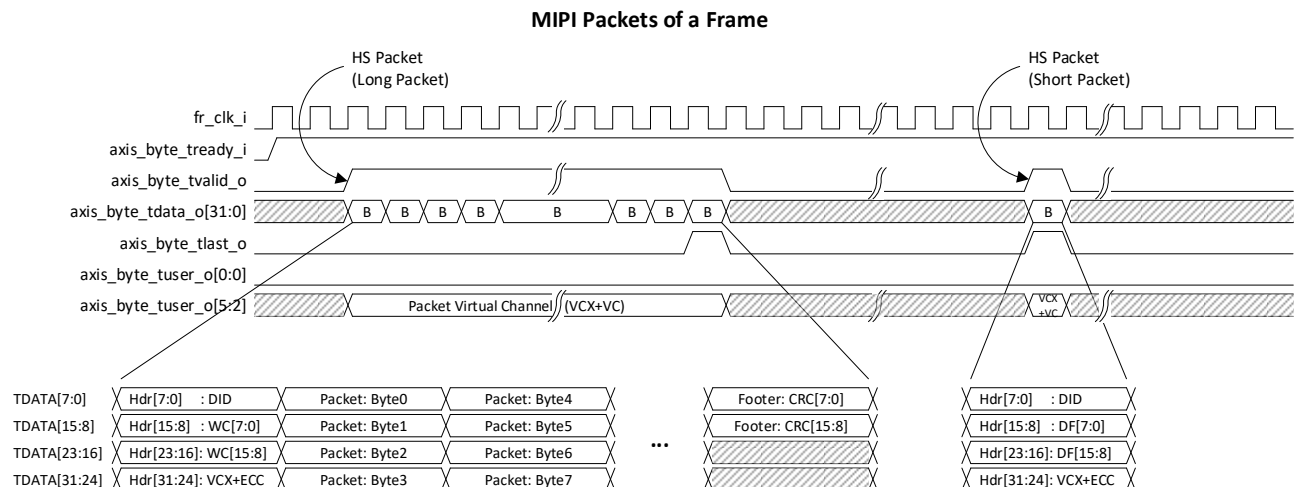


Figure 2.23. MBSI RX Timing Diagram for 32-bit Passthrough Mode

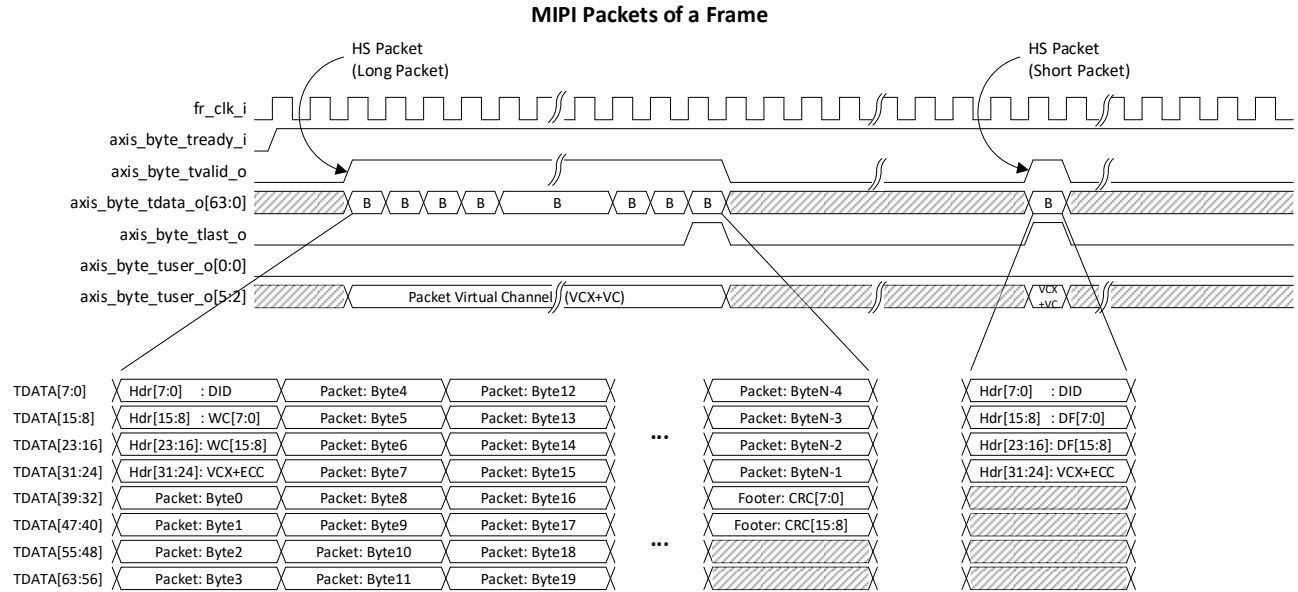


Figure 2.24. MBSI RX Timing Diagram for 64-bit Passthrough Mode

In 64-bit DSI Controller Mode, packets for MPPT may appear in the same clock cycle. [Figure 2.25](#) shows an example where the footer of the first packet coincides with the header bytes of the second packet in the same clock cycle. TLAST asserts on the last beat of packet per HS transaction and asserts exactly once per HS transaction, on the last content beat of the transaction.

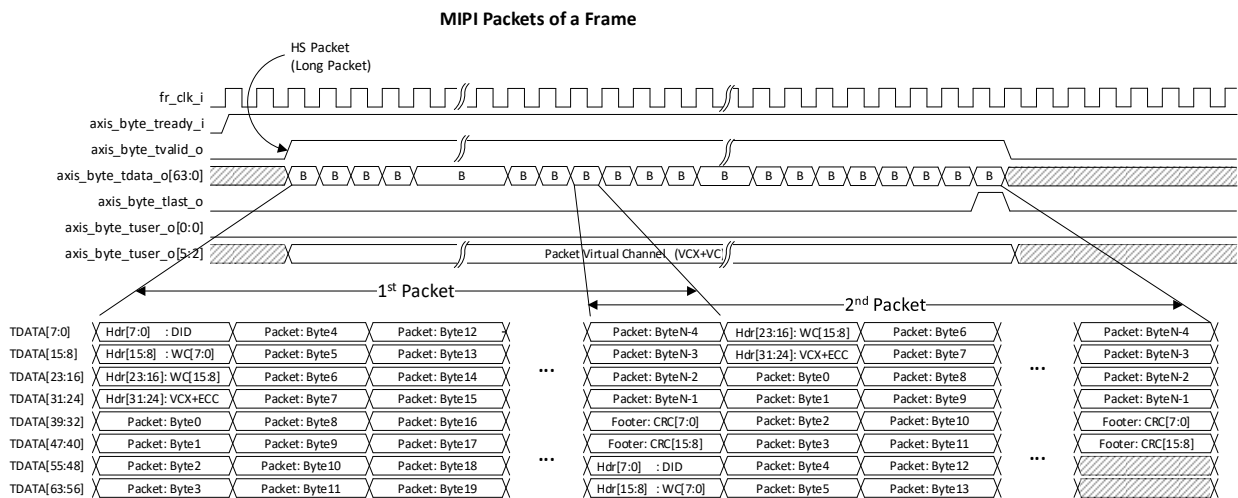


Figure 2.25. MBSI RX Timing Diagram for 64-bit MPPT Passthrough Mode

When *MIPI Byte Streaming Interface* is enabled, a single clock FIFO is used to hold data when downstream is not ready (TREADY is low). FIFO depth is configurable through the *Byte Buffer Depth* attribute. If TREADY remains low for a duration greater than or equal to the configured buffer depth, the buffer can overflow, leading to functional errors.

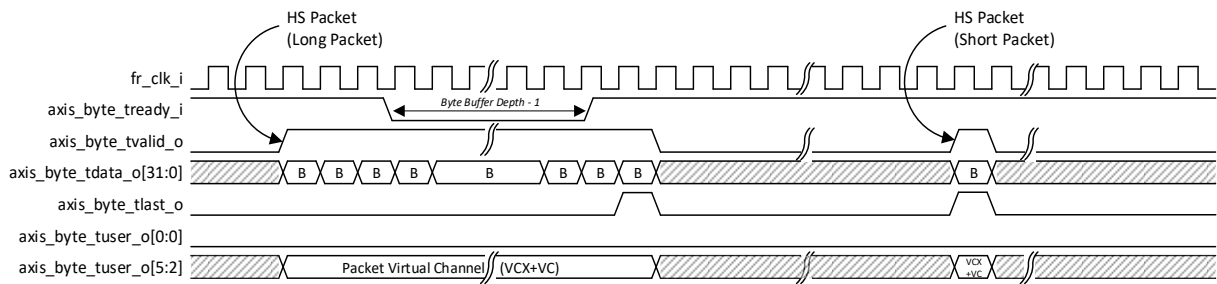


Figure 2.26. MBSI RX Timing Diagram for 32-bit Passthrough Mode with TREADY Deassertion

2.5. MIPI CSI/DSI Transmitter PHY Block

The PHY block (D-PHY transmitter) is the physical interface that drives MIPI D-PHY signals onto differential lanes and converts parallel byte data into serialized high-speed (HS) and low power (LP) signaling as required by the MIPI D-PHY specification. The PHY block supports the Soft D-PHY and Hard D-PHY modes.

This block performs the following key functions:

- Clock generation: Produces the byte clock output from external high-speed clock and ensures proper distribution to the data lanes.
- Start-of-transmission (SoT) signaling: Generates the required HS prepare, HS-zero, and SoT sequences and appends SYNC WORD pattern 0xB8 to incoming HS payload data before transmitting.
- End-of-transmission (EoT) signaling: Drives the required EoT and HS-trail patterns, followed by switching the lane back to LP mode.
- Serialization: Converts the incoming 8-bit parallel data into DDR-based serialized HS output on differential data lanes.
- Low power and high-speed control: Manages LP-to-HS and HS-to-LP transitions based on request inputs, ensuring compliance with MIPI D-PHY timing requirements.
- LP signaling: Drives LP-00 and LP-11 states for control sequences.
- Skew calibration: Drives a skew calibration HS sequence on all data lanes when requested by the controller after completing the power-up initialization (T-INIT) process. The PHY drives the serial link with skew calibration sync pattern followed by calibration pattern for the defined calibration time.

2.6. MIPI Timing Parameters

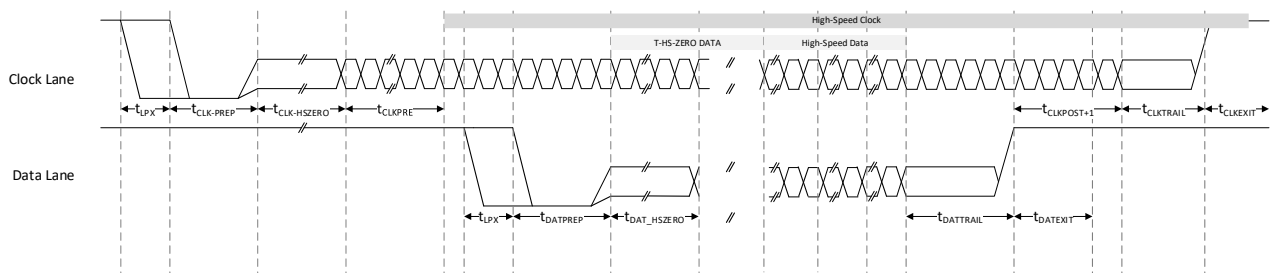


Figure 2.27. MIPI Timing Parameters

Figure 2.27 provides a visual representation of the MIPI D-PHY clock and data lane timing behavior, illustrating the relationship between signal transitions and the defined timing parameters. The timing parameters referenced in the [MIPI CSI/DSI TX Protocol Timing Parameters](#) section are depicted in the diagram using labeled intervals on the clock and data lanes. Each labeled interval corresponds to a specific timing parameter and indicates the location of delays and transition periods relative to the clock and data signals. Variations in timing parameter values are reflected as

changes in the extent or placement of the corresponding waveform segments on the clock lane, the data lane, or both. When viewed together with the timing parameter definitions in the [MIPI CSI/DSI TX Protocol Timing Parameters](#) section, the diagram provides a clear mapping between the configured timing values and the resulting manifestation on the physical interface.

2.7. MIPI CSI/DSI Transmitter Controller Block

The TX controller block prepares outgoing video data and constructs MIPI-compatible packet headers and payloads before sending the data to the PHY for serialization. The controller block handles packet formatting, lane distribution, and all necessary packet-level control required for transmission.

This block performs the following key functions:

- **Packetization:** In UVSI mode, the packetizer takes incoming pixel data and builds MIPI packets by adding the required header and footer information, such as Data Type, Word Count, Frame Number, Line Number, ECC, and CRC. In MBSI mode, the packetizer receives an already formatted packet (including header, payload, and CRC footer), then recalculates and inserts the ECC/CRC values and updates the VC/VCX information in the packet header.
- **Lane distribution:** Splits the outgoing byte stream across multiple lanes based on the configured lane count, performing width expansion, byte mapping, and lane-based data scheduling.
- **Initial skew calibration scheduling:** Triggers a one-time skew calibration pulse on power-up when *Initial Skew Calibration* is enabled and supports runtime re-triggering through CSR when *Enable AXI4-Lite Interface* is enabled. The controller gates the upstream data path until the initial skew calibration transmission is complete.
- **Periodic skew calibration scheduling:** Triggers a periodic skew calibration pulse during vertical blanking intervals to avoid disrupting active video transmission. The periodic calibration feature can be enabled or disabled through runtime CSR configuration.

Table 2.4. MIPI CSI/DSI Transmitter Word Count Support Limitation for MBSI and UVSI

PHY Direction	PHY Mode	Protocol Mode	Controller Mode	Minimum Word Count Limitation (Bytes)
TX	Soft D-PHY	CSI-2	32-bit	32
TX	Hard D-PHY	CSI-2	64-bit	32

2.7.1. MIPI CSI/DSI Transmitter Skew Calibration

Skew calibration is a Hard D-PHY transmitter feature that compensates for inter-lane skew on high-speed serial links. Per the MIPI D-PHY specification, transmission of a skew calibration sequence is required for data rates above 1500 Mbps. Configure the initial and periodic skew calibrations with an acceptable period interval to avoid disruption of throughput. In UVSI mode, configure the calibration period with a value that is lower than the blanking interval between two image frames. The IP supports two calibration modes:

- **Initial skew calibration** — Runs once after PHY power-up and initialization, before any HS data transmission begins. This ensures the receiver can compensate for lane-to-lane skew from the start of operation.
- **Periodic skew calibration** — Runs at Frame End intervals during operation to compensate for skew drift over time.

Initial and periodic skew calibration can be enabled using the following methods:

- Using the GUI:
 - Initial skew calibration is available when *PHY Mode* is set to Hard D-PHY and the *Line Rate per Lane* exceeds 1500.0 Mbps.
 - Periodic skew calibration is available when *PHY Mode* is set to Hard D-PHY, at any supported line rate.
- Using CSR access:
 - When the AXI4-Lite interface is enabled, both initial and periodic skew calibration can be enabled or disabled at runtime via the CSR register at address 0x220 via the CAL_INITIAL_EN and CAL_PERIODIC_EN register spaces, respectively.
 - The calibration duration for both initial and periodic skew calibration can also be configured through the same register location via the T_SKEW_INITIAL and T_SKEW_PERIODIC register spaces, respectively.

- Initial skew calibration can be re-triggered at runtime using a level-write pulse to CSR via the CAL_INITIAL_TRIGGER_EN register space. This operation allows the recalibration after the link has been established.
- When configured through CSR, initial skew calibration and re-trigger are supported at any line rate, provided the *PHY Mode* is set to Hard D-PHY.
- When configured through CSR, periodic skew calibration is supported at any line rate, provided the *PHY Mode* is set to Hard D-PHY.

2.7.1.1. Initial Skew Calibration Operation

When Initial skew calibration is enabled through the GUI, the skew calibration FSM in the TX controller issues a skew calibration pulse to the PHY after PHY initialization is complete. The PHY then transmits a calibration pattern across all active data lanes. When the calibration pattern transmission is complete, the TREADY signal on the IP interface is asserted, indicating that the IP is ready to accept data for HS transmission. Figure 2.28 shows the signal transition during initial skew calibration.

Alternatively, initial skew calibration can be configured at runtime through the CSR register at address 0x220. The CAL_INITIAL_EN field controls whether initial calibration is enabled, and the T_SKEW_INITIAL field specifies the calibration duration in byte clock cycles. The IP also supports re-triggering the initial calibration process via the CAL_INITIAL_TRIGGER_EN field in the same register. This enables recalibration for systems where external devices have varying power-up times or where link conditions may have changed since the initial calibration. Suspend any incoming packets of the interval when initial skew calibration is re-triggered.

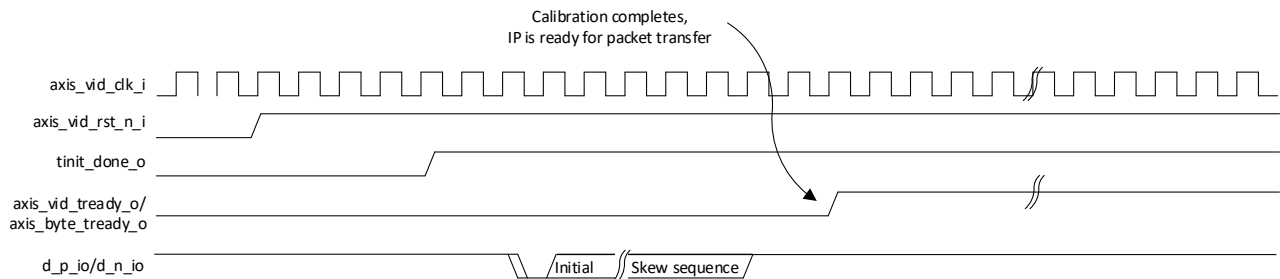


Figure 2.28. Initial Skew Calibration Sequence

2.7.1.2. Periodic Skew Calibration Operation

When Periodic skew calibration is enabled, the TX controller schedules calibration between image frames to avoid disrupting active data transmission. After a Frame End is transmitted, the TX controller waits for the PHY to enter the Stop state, indicating that the link is idle and ready for calibration. The scheduler in the TX controller then issues a skew calibration pulse to the PHY, triggering a periodic calibration pattern across the serial links. Figure 2.29 shows the signal transition during periodic skew calibration.

Periodic skew calibration can also be enabled or disabled at runtime through the CSR register at address 0x220 via the CAL_PERIODIC_EN field. The calibration duration is configured through the T_SKEW_PERIODIC field.

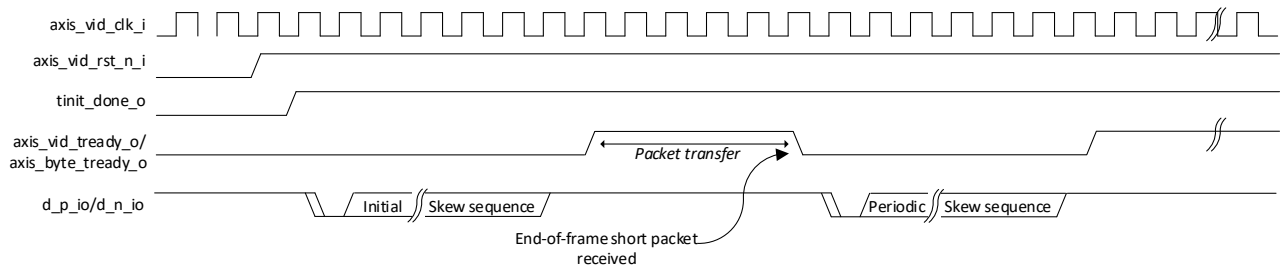


Figure 2.29. Periodic Skew Calibration Sequence

Calibration time calculation:

Byte Clock Period (ns) = PHY Data Width / Line Rate per Lane (Mbps) × 1000

Calibration Time (ns) = T_SKEWCAL_INITIAL × Byte Clock Period (ns)

Example:

Line Rate per Lane = 2500.0 Mbps

PHY Data Width = 16

T_SKEWCAL_INITIAL = 2049 (0x801)

Byte Clock Period = 6.4 ns

Calibration Time = 13113.6 ns

Programming T_SKEW_INITIAL with a value of 2049 (0x801) and triggering via CAL_INITIAL_TRIGGER_EN results in a calibration time of approximately 13113.6 ns across the serial links. Similar formula and calculation are applicable to periodic skew calibration time calculation. Refer to [Table 5.6](#) for more information about registers.

2.8. MIPI CSI/DSI Transmitter Pixel-to-Byte Block

The Pixel-to-Byte module converts incoming pixel data into MIPI-compatible payload bytes and stores the data in the line buffer before forwarding the data to the packetization block through UVSI. This block bridges the video pixel clock domain (axis_vid_clk_i) and byte clock domain (byte_clk_o), using a dual-clock FIFO for buffering, synchronization, and flow-control management.

This block performs the following key functions:

- **Pixel-to-Byte conversion:** Converts incoming pixel data of various color formats into byte-aligned payload structures suitable for MIPI packetization. Implements AXI4-Stream TVALID/TREADY handshaking to manage data flow and prevent FIFO overflow or underflow during mode transitions or clock-rate differences.
- **Line buffer:** Stores pixel information in a byte-aligned format and performs clock-domain bridging between the video-pixel clock and the byte clock using a dual-clock FIFO, ensuring safe data transfer across asynchronous domains while managing backpressure.
- **Video FSM:** The video timing block defines and enforces the video cadence by sequencing synchronization events and active video transfer. This block drives packet generation of sync short packets (Frame Start, Frame End, Line Start, Line End) and active video packets. The video timing FSM operates in CSI-2 mode, selecting the protocol-specific packet schedule and timing rules. In accordance with the CSI-2 specification, the LS (Line Start) and LE (Line End) short packets are optional and may be disabled based on system configuration or video timing requirements using an IP parameter.

Table 2.5. MIPI CSI/DSI Transmitter Pixel-to-Byte Supported Configurations

TX Controller Mode	Color Format	Number of Pixel per Clock (PPC)
32-bit	RGB888, RGB565, RAW10, RAW12	1, 2, 4
64-bit ¹	RGB888, RGB565, RAW10, RAW12	1, 2, 4

Note:

1. 64-bit controller mode is supported only in Hard D-PHY with 4 lanes configuration, avoiding resource consumption in other configurations.

2.8.1. MIPI CSI/DSI Transmitter Blanking Insertion

At the end of line or frame on the HS burst transmission, video FSM inserts additional horizontal line or vertical frame blanking period duration at LP-11 (Stop/Idle) state where both positive and negative differential serial d_p/n_io signals stay high. The horizontal_blanking_cnt_en_o and vertical_blanking_cnt_en_o output signals assert during the corresponding horizontal line or vertical frame blanking period of the LP-11 state.

Refer to [Table 4.5](#) for more details.

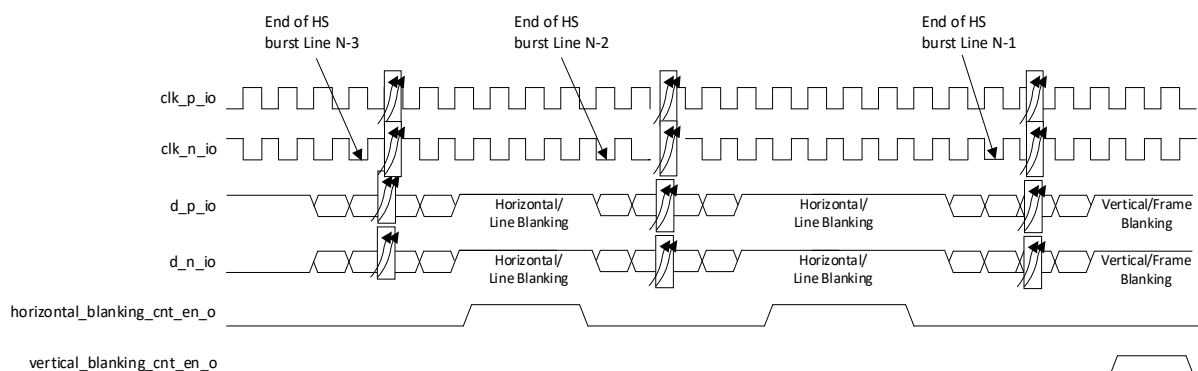


Figure 2.30. LP-11 State After HS Burst Transmission

To achieve a desirable target frame rate, the transmitter is required to dynamically insert horizontal and/or vertical blanking periods to regulate the frame period duration of the video frame output. With additional blanking insertion, the transmitter introduces additional duration at LP-11 state to throttle the serial streaming output and maintain the frame rate to be constant.

As the number of lanes increases with fixed lane rate, the active pixel data is distributed across all lanes thus active data transmission time is reduced. To maintain the same frame rate across different number of lanes, the required horizontal line or vertical frame blanking period duration at LP-11 (Stop/Idle) state needs to be adjusted accordingly.

The following table describes an example of expected frame rate derived with no overhead:

- Active Resolution: 1920 × 1080
- Horizontal Blanking Pixels: 280
- Vertical Blanking Lines: 45
- Total Horizontal Pixels: 2,200
- Total Vertical Lines: 1,125
- Pixel format: RAW10
- Lane rate: 912 Mbps
- Pixels Per Clock: 4
- Byte Clock (Lane rate/PHY Data Width): 114 MHz

Table 2.6. Example of Expected Frame Rate Calculation

Parameter	Number of Lanes			Unit
	1 Lane	2 Lanes	4 Lanes	
Required Pixel Clocks ((Number of Lanes x Lane Rate) / (Bits Per Pixels)/Pixels Per Clock)	22.80	45.60	91.20	MHz
Active Horizontal Bytes (Per Lane)	2,400	1,200	600	Byte Clock Cycle
Horizontal Blanking Bytes (Per Lane)	350	175	88	
Total Horizontal Bytes (Per Lane)	2750	1375	688	
Line Period (Total Horizontal Bytes x Byte Clock Period)	24.123	12.061	6.035	us
Frame Period (Total Lines x Line Period)	27.138	13.569	6.789	ms
Frame Rate Per Second (1/Frame Period)	36.8	73.7	147.3	fps

With fixed horizontal and vertical blanking period across different number of lanes, the frame rate increases proportionally. However, additional overheads (short packet, packet header, CRC, HS to LP state transition, data path

logic delays) further throttle the serial streaming output and the actual frame rate measured is less than the example above.

The horizontal or vertical blanking period required to be inserted in TX can be configured using the following settings from the IP Wizard GUI or CSR registers:

- Number of Vertical Blanking Lines Per Frame (NUM_LINE_V_BLANKING field in CONFIG_REG1 register)
- Number of Horizontal Blanking Bytes in Byte Clock Domain (H_BLANKING_BYTES_TH field in CONFIG_REG1 register)
- Number of Horizontal Active Bytes in Byte Clock Domain (NUM_BYTES_ACTIVE_H field in CONFIG_REG2 register)

Refer to the [Horizontal and Vertical Blanking Period Insertion Tuning](#) section for more details.

2.9. MIPI CSI/DSI Receiver PHY Block

The PHY block (D-PHY receiver) is the physical interface that receives MIPI D-PHY signals from differential pairs and converts the signals to parallel byte data. The PHY block supports the Soft D-PHY and Hard D-PHY modes.

This block performs the following key functions:

- Clock recovery: Generates and distributes byte clock (byte_clk_o) from incoming MIPI clock lane.
- Deserialization: Uses DDR for 8-bit deserialization.
- Word alignment: Detects the 0xB8 SoTp and aligns incoming data.
- Low power and high-speed control: Manages LP-HS transitions.
- Deskew calibration: Ensures data received on multiple lanes is aligned and synchronized. This feature is supported only when *PHY Mode* is set to *Hard D-PHY*.

2.10. MIPI CSI/DSI Receiver Controller Block

The RX controller block processes byte data from the PHY into MIPI packet headers and payloads. The block handles lane alignment, lane merging, and depacketization with error detection and correction.

This block performs the following key functions:

- Lane alignment: Aligns data across multiple lanes using FIFO-based buffering and SoTp detection.
- Lane merge: Combines aligned lane data into a single stream, performs width conversion, byte reordering, and propagates end-of-packet indicators. During width conversion, this block merges the data into 32-bit or 64-bit streams based on lane count. The 64-bit controller mode is for *PHY Data Width* of 16 with 4 data lanes. All other *PHY Data Width* and lane combinations are treated as 32-bit controller mode.
- Depacketization: Extracts headers and payloads. This depacketizer enforces minimum word count requirement based on selected configuration as shown in [Table 2.7](#). This limitation is applicable for supported streaming interfaces.
- Error detection and correction: ECC for D-PHY (1-bit correction, 2-bit detection) and CRC for headers and payloads with dedicated error flags.
- Byte streaming source: Converts internal short and long packets to the AXI4-Stream protocol in byte domain and handles backpressure.

Table 2.7. MIPI CSI/DSI Receiver Word Count Support Limitation

PHY Direction	PHY Mode	Protocol Mode	Controller Mode	Minimum Word Count Limitation (Bytes)
RX	Soft D-PHY or Hard D-PHY	DSI-2	32-bit	14
RX	Soft D-PHY or Hard D-PHY	CSI-2	32-bit	6
RX	Hard D-PHY	DSI-2	64-bit	34
RX	Hard D-PHY	CSI-2	64-bit	10

Note:

1. 64-bit controller mode is only supported in Hard D-PHY with 4 lanes configuration.

2.11. MIPI CSI/DSI Receiver Byte-to-Pixel Block

The Byte-to-Pixel module converts MIPI packet payload bytes into pixel data and outputs the data via UVSI AXI4-Stream. The block bridges the free running clock domain (fr_clk_i) and video pixel clock domain (axis_vid_clk_i), and uses dual clock FIFO for data buffering and flow control.

This block performs the following key functions:

- Byte-to-Pixel conversion: Translates MIPI CSI/DSI packet payload bytes into pixel data according to the configured video format. Payload processing begins after detecting SOF.
- Clock domain bridging: Interfaces between the free running clock domain (fr_clk_i) and the video pixel clock domain (axis_vid_clk_i).
- AXI4-Stream output: Outputs the converted pixel data via UVSI.

Table 2.8. MIPI CSI/DSI Receiver Byte-to-Pixel Supported Configurations

RX Controller Mode	Color Format	Number of Pixel per Clock (PPC)
32-bit	RGB888, RGB565, RAW10, RAW12	1, 2, 4
64-bit	RGB888, RGB565, RAW10, RAW12	1, 2, 4

Note:

1. 64-bit controller mode is only supported in Hard D-PHY with 4 lanes configuration

2.12. Dynamic Reconfiguration

Dynamic lane reconfiguration allows runtime configuration of the active MIPI data lane number without a system reset. This feature is targeted for power management, bandwidth adaptation, and responding to changing display and sensor requirements.

To enable this feature, configure the IP based on the supported IP attributes in the [IP Parameter Description](#) section. Check the additional IP parameters in the Module/IP Block Wizard in the Lattice Radiant software as follows:

- Ensure the **Enable AXI4-Lite Interface** and **Dynamic Reconfiguration** parameters are checked. This is required as registers are accessible only through AXI4-Lite.
- Set the **Number of D-PHY Lanes** parameter to the maximum desired value at compile time to enable dynamic reconfiguration of all possible combinations.

When you configure the PHY_CONFIG_REG (0x210) register via the AXI4-Lite interface, the new lane configuration takes effect when all the current active data lanes are in idle state, which ensures no data corruption occurs.

2.12.1. MIPI CSI/DSI Transmitter Dynamic Lane Reconfiguration

To reconfigure the dynamic lane registers, follow these steps:

1. After sending the whole frame packet, wait for 2ms to allow packet processing to complete.
2. Write to PHY_CONFIG_REG 0x210[2:0] to change the number of active lanes and PHY_CONFIGURATION_REG 0x210[3] to enable the update.
3. For generated IP with x4 lanes, to configure the IP to x1 lanes, write 0x0000_0009 to PHY_CONFIG_REG address (0x210). To configure the IP to x2 lanes, write 0x0000_000A to PHY_CONFIG_REG address (0x210).

When you configure the PHY_CONFIG_REG (0x210) register via the AXI4-Lite interface, the new lane configuration takes effect when all the current active data lanes are in idle state, which ensures no data corruption occurs.

2.12.2. MIPI CSI/DSI Receiver Dynamic Lane Reconfiguration

To reconfigure the dynamic registers, follow these steps:

1. Write to PHY_CONFIG_REG 0x210[2:0] to change the number of active lanes and PHY_CONFIG_REG 0x210[3] to enable the update.
2. When *PHY Mode* is set to *Hard D-PHY*, configurable deskew calibration can be set by writing to PHY_CONFIG_REG 0x210[4] to enable or disable deskew calibration and PHY_CONFIG_REG 0x210[5] to enable the deskew setting.
3. After setting PHY_CONFIG_REG 0x210, wait for 2 ms before writing to CONFIG_REG 0x200, to indicate that all configuration registers are already set.

When you configure the PHY_CONFIG_REG (0x210) register via the AXI4-Lite interface, the IP loads the new settings when the active data lanes exit the stop state and begins the sequence to enter high-speed mode. This ensures no data corruption occurs.

2.12.3. MIPI CSI/DSI Transmitter Resolution Reconfiguration

In MBSI mode, you only generate the appropriate frame packets at the MBSI interface. To reconfigure the resolution in the UVS mode, follow these steps:

1. After sending the whole frame packet, wait for 2 ms to allow packet processing to complete. Skip Step 1 if you need to change the resolution after power up and before sending any packets.
2. Read PHY_CONFIG_REG 0x020[0] to check if CFG_READY is 1 to proceed to the next step.
3. Write the new number of vertical lines per active frame into CONFIG_REG2 0x244[31:16]. When the **Blanking Period Enable (BP_EN)** option is enabled, NUM_LINE_V_BLANKING, H_BLANKING_BYTES_TH (CONFIG_REG1), and NUM_BYTES_ACTIVE_H (CONFIG_REG2) must be configured accordingly.
4. Send a new frame packet with the new resolution.

3. IP Parameter Description

The configurable attributes of the MIPI CSI/DSI IP are shown in the following tables. You can configure the IP by setting the attributes accordingly in the IP Catalog Module/IP Block Wizard of the Lattice Radiant software.

Wherever applicable, default values are in bold.

3.1. General

Table 3.1. General Attributes

Attribute	Selectable Values	Description
PHY Configuration		
PHY Direction	TX, RX	Selects the operating direction of the PHY.
PHY Mode	Soft D-PHY , Hard D-PHY	Selects the physical layer implementation. Only CrossLink-NX devices support Hard D-PHY.
PHY Data Width	8 , 16	Defines the width of the deserialized data stream per lane in bits. <i>PHY Data Width</i> = 16 is available only when <i>PHY Mode</i> is set to <i>Hard D-PHY</i> . Not editable. For information only.
Number of D-PHY Lanes	1, 2, 4	Specifies the number of data lanes. D-PHY supports up to 4 lanes (1-wire differential signaling per lane).
Line Rate Per Lane (Mbps) ¹	160–1500, 1000 (Soft D-PHY) 320–2500, 1000 (Hard D-PHY)	Maximum bandwidth when <i>PHY Direction</i> is <i>TX</i> . Set the data rate per lane in Mbps for D-PHY. This determines the maximum bandwidth capability of each lane.
	80–1500, 1000 (Soft DPHY) 160–2500, 1000 (Hard DPHY)	Maximum bandwidth when <i>PHY Direction</i> is <i>RX</i> . Set the data rate per lane in Mbps for D-PHY. This determines the maximum bandwidth capability of each lane.
Actual Byte Clock Frequency (MHz)	10–187.5, 125	Operating byte clock frequency of the IP. The value is <i>Line Rate per Lane (Mbps)/ PHY Data Width</i> . Not editable. For information only.
Actual D-PHY Clock Frequency (MHz)	40–1250, 500	Operating frequency of the PHY layer. The value is <i>Line Rate per Lane (Mbps)/2</i> . Not editable. For information only.
PHY Clock Mode	Continuous , Non-Continuous	Configures the clock mode for Soft D-PHY. Continuous mode keeps the clock lane in high-speed clock. Non-continuous mode allows the clock to enter low power state between high-speed data transmissions.
Edge Clock Mode	Internal , External	Selects the source of the edge clock used by the Soft D-PHY TX. Edge clock is the high-speed per-bank clock network in Lattice FPGAs that drives DDR I/O elements and synchronizes PHY data transitions. Refer to the MIPI CSI/DSI Transmitter Edge Clock Sharing section. Internal: The Soft D-PHY TX generates its own edge clock using internal edge clock synchronizer and divider resources. Use this mode when the D-PHY operates independently and can use dedicated per-bank edge clock resources. External: The Soft D-PHY TX receives edge clock from another D-PHY TX instance, enabling multiple TX instances in the same bank to share a common edge clock source. Use this mode to optimize clock resources and maintain synchronization across multiple TX instances running at the same bit rate.

Attribute	Selectable Values	Description
		This setting is available only when <i>PHY Direction</i> is set to <i>TX</i> .
Reference Clock Frequency (MHz)	60 (Soft D-PHY) 60–200, 100 (Hard D-PHY)	Sets the operating frequency of the clock domain for soft PHY related components in Soft D-PHY mode. Sets the reference clock frequency of the internal PLL for hard PHY related components in Hard D-PHY mode.
Controller		
Protocol Mode	DSI-2, CSI-2	Selects the MIPI protocol type. CSI-2 is optimized for camera and imaging applications with support for image data types and timing. DSI-2 is designed for display applications with video-specific packet structures and flow control mechanisms. DSI-2 is available only when <i>PHY Direction</i> is set to <i>RX</i> .
ECC Insert (TX) / ECC Check (RX)	checked , unchecked	Enables or disables ECC insertion for TX and ECC checking for RX. Disabling ECC can reduce IP resource usage. <i>PHY Direction == TX:</i> When checked, the IP recalculates ECC and updates the packet header with VC and VCX value provided in the sideband signal. Refer to the MIPI Byte Streaming Interface (MBSI) section. This setting is available only when <i>PHY Direction == TX</i> and <i>MIPI BYTE Streaming Interface (MBSI)</i> is checked. <i>PHY Direction == RX:</i> By default, the IP enables ECC checking. The 6-bit packet header ECC can correct single-bit errors and detect two-bit errors in the D-PHY configuration. In Passthrough Mode, the IP transmits the original ECC received in <code>axis_byte_tdata_o</code> . This setting is available only when <i>PHY Direction == RX</i> and <i>Protocol Mode == CSI2</i> . When <i>Protocol Mode == DSI2</i> , this is always enabled to qualify valid header assertion.
CRC Insert (TX) / CRC Check (RX)	checked , unchecked	Enables or disables CRC insertion for TX and CRC checking for RX. Disabling CRC can reduce the IP resource usage. <i>PHY Direction == TX:</i> When checked, the IP recalculates CRC checksum and updates the packet footer with recalculated CRC checksum for each long packet. Refer to the MIPI Byte Streaming Interface (MBSI) section. This setting is available only when <i>PHY Direction == TX</i> and <i>MIPI BYTE Streaming Interface (MBSI)</i> is checked. <i>PHY Direction == RX:</i> By default, the IP enables CRC checking. A checksum is computed over the WC bytes of each long packet.
Enable AXI4-Lite Interface	checked, unchecked	Enables the AXI4-Lite bus used for accessing IP registers.
Dynamic Reconfiguration	checked, unchecked	When checked, the IP enables dynamic reconfiguration feature through the AXI4-Lite interface register access. Refer to the Dynamic Reconfiguration section for details. This setting is available only when <i>Enable AXI4-Lite Interface</i> is checked.
Streaming Interface		
Unified Video Streaming Interface (UVSI)	checked , unchecked	When checked, the IP enables transmission and reception of pixel data via the UVSI AXI4-Stream interface. Refer to the Unified Video Streaming Interface (UVSI) section for details.
MIPI Byte Streaming Interface (MBSI)	checked, unchecked	When checked, the IP enables transmission and reception of byte data via the MBSI AXI4-Stream interface. The IP maps internal high

Attribute	Selectable Values	Description
		speed packet signals (header and byte data) to AXI4-Stream (TVALID, TREADY, TDATA, TUSER, TLAST). Refer to the MIPI Byte Streaming Interface (MBSI) section for details.
Passthrough Mode	checked	Not editable. For information only when <i>MIPI Byte Streaming Interface == checked</i>. <i>PHY Direction == TX</i>: Externally formatted MIPI packets (byte format) are received via MBSI. TDATA carries short packets, header for long packets, payload, and footer while TUSER provides VCX and VC fields. <i>PHY Direction == RX</i>: Bytes are forwarded in the same order defined by the MIPI specification. TDATA carries header for long packets, payload, and footer while TUSER provides VCX and VC fields.
Video Format		
Line Start and Line End	checked, unchecked	Checked: Activates LS and LE short-packet generation. When enabled, the video timing FSM issues Line Start and Line End short packet markers at each line boundary to indicate the beginning and end of active video lines. Unchecked: Disables LS and LE short-packet generation. The video timing FSM suppresses line-boundary markers and does not emit LS or LE short packets. This setting is available only when <i>PHY Direction</i> is set to TX.
Line Number Insertion	checked, unchecked	Checked: The module inserts the line number into the Line Start (LS) and Line End (LE) short packets. Unchecked: The line number field in both LS and LE packets is forced to 0. This setting is available only when <i>PHY Direction</i> is set to TX and <i>Line Start and Line End</i> is checked.
Blanking Enable	checked, unchecked	Checked: Inserts additional horizontal line or vertical frame blanking period duration for the LP11 state after the end of long packet transmission. When enabled, the video timing FSM inserts additional blanking period duration as follows: For horizontal line blanking period: <i>Line Start and Line End</i> enabled: The horizontal line blanking period is inserted in the LP11 state between the LE packet to the next LS packet. <i>Line Start and Line End</i> disabled: The horizontal line blanking period is inserted in the LP11 state between the end of a long packet to the beginning of the next long packet. For vertical frame blanking period: The vertical frame blanking period is inserted in the LP11 state between the current FE packet to the subsequent FS packet. Unchecked: Only minimal duration for LP11 state after the end of a long packet transmission or the LP to HS transition. Refer to the MIPI CSI/DSI Transmitter Blanking Insertion section. This setting is available only when <i>PHY Direction</i> is set to TX.
Number of Horizontal	1–65535, 2400	Indicates the number of horizontal active bytes of the video frame

Attribute	Selectable Values	Description
Active Bytes in Byte Clock Domain		per lane. The video timing FSM uses this value to determine the total horizontal period of the video frame and calculate the vertical frame blanking period. For example, Horizontal Pixel or Width of 1920 for RAW10 format: $\text{Total Horizontal Pixel Bytes} = (\text{H_active} \times \text{BPC})/8$ $= (1920 \times 10)/8$ $= 2400 \text{ bytes}$ $\text{Number of Horizontal Active Bytes per lane} = \text{Total Horizontal Pixel Bytes} / \text{Number of D-PHY Lanes}$ This is equivalent to the number of byte clock cycles required for active packet transmission where each byte clock transfers 8 bits/byte per byte clock cycle for each lane at the PHY serial interface. This setting is available only when <i>PHY Direction</i> is set to TX and <i>Blanking Enable</i> is checked.
Number of Horizontal Blanking Bytes in Byte Clock Domain	0–65535	Indicates the number of horizontal blanking bytes of the video frame per lane. The video timing FSM uses this value to determine the total horizontal period of the video frame and calculate the vertical frame blanking period. For example, Horizontal Blanking Pixel or Width of 280 for RAW10 format: $\text{Number of Horizontal Blanking Bytes per lane} = (\text{H_blank} \times \text{BPC})/8$ $= (280 \times 10)/8$ $= 350 \text{ bytes}$ Set this to zero for no additional horizontal line blanking period insertion. A minimum horizontal blanking period value is imposed on the LP11 state by the video timing FSM based on a specific setting configured in the TX. This is equivalent to the number of byte clock cycles required for blanking period where each byte clock transfers 8 bits/byte per byte clock cycle at the PHY serial interface. This setting is available only when <i>PHY Direction</i> is set to TX and <i>Blanking Enable</i> is checked.
Number of Vertical Blanking Lines Per Frame	0–65535	Indicates how many vertical blanking lines compose a video frame. The video timing FSM uses this value to determine the total vertical frame blanking period of the video frame between the End of Frame (FE) and Start of Frame (FS) short packets. Set this to zero for no additional vertical frame blanking period insertion. A minimum vertical frame blanking period value is imposed on the LP11 state by the video timing FSM based on a specific setting configured in the TX. This setting is available only when <i>PHY Direction</i> is set to TX and <i>Blanking Enable</i> is checked.
Number of Active Lines Per Frame	1–65535, 48	Indicates how many active lines compose a video frame. The video timing FSM uses this value to track line progression and determine the frame boundary positions for generating Start of Frame (FS) and End of Frame (FE) short packets. This setting is available only when <i>PHY Direction</i> is set to TX.
Number of Pixel per Clock (PPC)	1, 2, 4	Defines how many pixels are processed per clock cycle in the Pixel-to-Byte and Byte-to-Pixel conversion blocks. Higher PPC values increase throughput but require more parallel processing resources. The

Attribute	Selectable Values	Description
		available options depend on the enabled color formats, PHY configuration, and timing requirements. Refer to the MIPI CSI/DSI Transmitter Pixel-to-Byte Block and MIPI CSI/DSI Receiver Byte-to-Pixel Block sections for more details.
Enable RGB888 Color Format	checked , unchecked	Enables support for RGB888 (24-bit) color format in the Pixel-to-Byte or Byte-to-Pixel conversion block. ²
Enable RGB565 Color Format	checked, unchecked	Enables support for RGB565 (16-bit) color format in the Pixel-to-Byte or Byte-to-Pixel conversion block. ²
Enable RAW10 Color Format	checked, unchecked	Enables support for RAW10 (10-bit) color format in the Pixel-to-Byte or Byte-to-Pixel conversion block. ²
Enable RAW12 Color Format	checked, unchecked	Enables support for RAW12 (12-bit) color format in the Pixel-to-Byte or Byte-to-Pixel conversion block. ²
Miscellaneous		
Line Buffer Depth	depthN , depthN×2, depthN×4	Defines the depth of the line-buffer FIFO used in the Pixel-to-Byte (P2B) block for TX. The FIFO has a fixed width of 256 bits. The required depth is determined by the number of active pixels per line and the selected bits-per-component. The minimum depth (depthN) is computed as the smallest power-of-two FIFO depth that can hold one full video line, using ceil() to ensure the FIFO is never undersized. You may select depthN×2 or depthN×4 to increase buffering margin when horizontal blanking is short or absent. This attribute is available only when <i>PHY Direction</i> is set to <i>TX</i>. $\text{FIFO}_{\text{DEPTH}} = \text{Ceil}((\text{Hactive} \times \text{BPC})/256)$ $\text{depthN} = 2^{\text{ceil}[\log_2(\text{FIFO}_{\text{DEPTH}})]}$ Where Hactive: Number of Active Pixels Per Line BPC: Bits Per Color
Pixel Buffer Depth	depthN×1 , depthN×2, depthN×4, depthN×8	Specifies the depth of the internal pixel data buffer on Byte-to-Pixel (B2P) block of RX. The default min setting, depthN, is optimized to use one EBR. Nexus EBR size = 18432 bits $\text{depthN} = 2^{\text{floor}[\log_2(\frac{\text{EBR size (bits)}}{\text{TDATA_WIDTH}+2})]}$ Example 1: TDATA_WIDTH = 96 depthN = 2^{floor[log2(18432/(96+2))]} depthN = 128 GUI Options: 128, 256, 512, 1024 Example 2: TDATA_WIDTH = 64 depthN = 2^{floor[log2(18432/(64+2))]} depthN = 256 GUI Options: 256, 512, 1024, 2048 This setting is available only when <i>PHY Direction</i> is set to <i>RX</i> and <i>Unified Video Streaming Interface (UVSI)</i> == <i>checked</i>.
Maximum Buffered Pixels	For <i>PHY Direction</i> == TX, [floor((line_buffer_depth / bits_per_pixel) × 256)] For <i>PHY Direction</i> == RX,	Indicates the total number of pixels that can be buffered in the FIFO based on <i>Line Buffer Depth</i> and selected pixel format when <i>PHY Direction</i> is set to TX or <i>Pixel Buffer Depth</i> when <i>PHY Direction</i> is set to RX. Not editable. For information only when <i>Unified Video Streaming</i>

Attribute	Selectable Values	Description
	[Pixel Buffer Depth × Number of Pixel per Clock (PPC)]	Interface (UVSI) == checked.
Byte Buffer Depth ³	For <i>PHY Direction</i> == TX, depthN×2, depthN×4, depthN×8, depthN×16, depthN×32, depthN×64 For <i>PHY Direction</i> == RX, depthN, depthN×2, depthN×4	<i>PHY Direction</i> == TX: Specifies the depth of the internal byte data buffer. The default minimum setting is depthN. If TDATA_WIDTH = 32 GUI Options: 512, 1024, 2048, 4096, 8192, 16384 If TDATA_WIDTH = 64 GUI Options: 512, 1024, 2048, 4096, 8192 For both TDATA_WIDTH = 32 and TDATA_WIDTH = 64, the byte buffer FIFO stores the complete MIPI packet, including the packet header, payload, and CRC bytes, along with the required sideband information. Therefore, the maximum supported payload size is equal to the total packet size in bytes minus 8 bytes. <i>PHY Direction</i> == RX: Specifies the depth of the internal pixel data buffer on the Byte-to-Pixel (B2P) block of RX. The default minimum setting, depthN, is optimized to use one EBR. Nexus EBR size = 18432 bits K = 9 If <i>PHY Mode</i> == <i>Soft D-PHY</i>, TDATA_WIDTH = 32. If <i>PHY Mode</i> == <i>Hard D-PHY</i> and <i>Number of D-PHY Lanes</i> == 4, TDATA_WIDTH width is 64, else TDATA_WIDTH width is 32. $\text{depthN} = 2^{\text{floor}\left[\log_2\left(\frac{\text{EBR size (bits)}}{\text{TDATA_WIDTH}+K}\right)\right]}$ Example: TDATA_WIDTH = 32 depthN = 2^{floor[log2(18432/(32+9))]} depthN = 256 GUI Options: 256, 512, 1024, 2048 This setting is available only when <i>MIPI Byte Streaming Interface</i> == checked.
Maximum Buffered Bytes	When <i>PHY Direction</i> == TX, <i>Maximum buffered bytes</i> is [4 × <i>Byte Buffer Depth</i>] in all cases, except for <i>PHY Data Width</i> = 16 with 4 lanes is [8 × <i>Byte Buffer Depth</i>] For <i>PHY Direction</i> == RX, [<i>Byte Buffer Depth</i> × 6]	Indicates the total number of bytes that can be buffered in the FIFO based on <i>Byte Buffer Depth</i>. Not editable. For information only when <i>MIPI Byte Streaming Interface</i> == checked.
Enable tINIT Counter	checked, unchecked	Enables the internal T_INIT counter that ensures the D-PHY TX remains in the LP-11 Stop State for the required initialization period before entering high-speed transmission. When disabled, the TX skips the enforced T_INIT wait time. This setting is available only when <i>PHY Direction</i> is set to TX.
tINIT Counter Value	1–65535, 1000	Specifies the number of byte clock cycles the transmitter must remain in the LP-11 Stop State to meet the D-PHY T_INIT initialization

Attribute	Selectable Values	Description
		requirement. This setting is available only when <i>PHY Direction</i> is set to <i>TX</i> and <i>Enable tINIT Counter</i> is checked.
tINIT Duration (ns)	8000	Displays the effective tINIT duration in ns computed from the user-programmed <i>tINIT counter value</i> . $tINIT_ns = tINIT\ counter\ value \times Byte_Clock_Period_ns$ This setting is available only when <i>PHY Direction</i> is set to <i>TX</i> . Not editable. For information only.
Enable Initial Skew Calibration	checked, unchecked	Enables initial skew calibration. This option is available only when <i>PHY Mode</i> is set to <i>Hard D-PHY</i> and <i>Line Rate</i> > 1500 Mbps.
Enable Periodic Skew Calibration	checked, unchecked	Enables periodic skew calibration and performs periodic skew calibration during video frame end blanking period. You can disable this feature at runtime through CSR access. This option is available only when <i>PHY Mode</i> is set to <i>Hard D-PHY</i> .

Notes:

1. The maximum data rate depends on the family, package, and performance grade of the device. Check the device data sheet for more information.
2. Only one-color format can be enabled at a time.
3. Device selection must account for the required TX Line Buffer/Byte Buffer FIFO depth. Smaller devices may have limited EBR resources. Select a device with sufficient EBR capacity to store the MIPI packet, payload, and sideband information.

3.2. MIPI CSI/DSI TX Protocol Timing Parameters

Table 3.2. MIPI CSI/DSI TX Protocol Timing Parameters – General Attributes

Attribute	Selectable Values	Description
Protocol Timing Parameters		
TX Global Operation Timing Parameters		
t_LPX	1–255 ⁴	Duration of any low-power state.
t_HS-PREPARE	1–255	Duration of the LP-00 line state before the HS-0 line state. Default value of <i>t_HS-PREPARE</i> = $\text{ceil}(\text{Minimum timing parameter value}/\text{ByteClk Period})$ If <i>t_HS-PREPARE_val</i> > 0, the default value of <i>t_HS-PREPARE</i> is <i>t_HS-PREPARE_val</i> + 1, otherwise, the default value is 1.
t_HS-ZERO	1–255	Delay from LP 00 entry to the point when the IP is ready to begin HS transmission. The actual HS-ZERO on the D-PHY data lanes depends on the serializer delay and internal packet processing. Default value of <i>t_HS_ZERO</i> = $\text{ceil}(\text{Minimum timing parameter value}/\text{ByteClk Period})$.
t_HS-TRAIL	3–255	Duration of the flipped bit after the last payload data bit of an HS transmission burst. Default value of <i>t_HS_TRAIL</i> = $\text{floor}(\text{Maximum T_EOT timing parameter value}/\text{ByteClk Period})$.
t_HS-EXIT	1–255	Duration of the data LP-11 state following an HS transmission burst. Default value of <i>t_HS_EXIT</i> = $\text{floor}((\text{Minimum timing parameter value}/\text{ByteClk Period}) + 1)$.
t_CLK-PREPARE	1–255	Duration of the LP-00 clock state immediately before the HS-0 clock state in the LP-to-HS sequence. Default value of <i>t_CLK-PREPARE</i> = $\text{ceil}(\text{Minimum timing parameter value}/\text{ByteClk Period})$.
t_CLK-ZERO	1–255	Duration of the clock HS-0 state prior to starting the toggling of the high-speed clock.

Attribute	Selectable Values	Description
		Default value of $t_CLK-ZERO = \text{ceil}(\text{Minimum timing parameter value}/\text{ByteClk Period})$.
$t_CLK-PRE$	2–255	Duration of the HS clock prior to the start of the LP-to-HS sequence of the data lanes. Default value of $t_CLK-PRE = \text{ceil}((\text{Minimum timing parameter value}/\text{ByteClk Period}) + 1)$.
$t_CLK-POST$	2–255 ⁴	Duration of the HS clock after the last associated data lane has transitioned to LP mode. The interval is defined as the period from the end of $t_{HS-TRAIL}$ to the beginning of $t_{CLK-TRAIL}$.
$t_CLK-TRAIL$	2–255	Duration of the HS-0 state after the last clock bit of an HS transmission burst. Default value of $t_CLK-TRAIL = \text{floor}((\text{Minimum timing parameter value}/\text{ByteClk Period}) + 2)$.
$t_CLK-EXIT$	1–255 ⁴	Duration of the clock LP-11 state following an HS transmission burst to the next HS transmission burst in non-continuous clock mode.
$t_SKEWCAL-INITIAL$	2 ¹⁵ –100 μs	Duration of the initial skew calibration. Default value is close to 2 ¹⁵ UI. This setting is available only when <i>Enable Initial Skew Calibration</i> is checked.
$t_SKEWCAL-PERIODIC$	2 ¹⁰ –10 μs	Duration of the periodic skew calibration. Default value is close to 2 ¹⁰ UI. This setting is available only when <i>Enable Periodic Skew Calibration</i> is checked.

Notes:

1. The maximum data rate depends on the family, package, and performance grade of the device. Check the device data sheet for more information.
2. Only one color format can be enabled at a time.
3. The duration of the timing parameter is equal to (byte clock period) $\times$ (attribute value).
4. The default value of this parameter is equal to $\text{floor}((\text{Minimum timing parameter value}/\text{ByteClk Period}) + 1)$.
5. The timing parameters are in number of byte clock cycles. The IP automatically calculates the timing parameters (in byte-clock cycles) to meet the required timing limits. You must verify that the selected values meet your overall system timing, as actual D-PHY lane timing may vary because of internal serialization and register delays.

4. Signal Description

This section describes the MIPI CSI/DSI IP ports.

4.1. MIPI CSI/DSI TX Signal Description

4.1.1. Clock Interface

Table 4.1. Clock and Reset Ports

Port	Type	Description
ref_clk_i	Input	Clock input for PHY fabric-based implementation. This clock drives all PHY fabric related logic and follows the frequency set in the <i>Reference Clock Frequency (MHz)</i> attribute. Available if <i>PHY Mode == Soft D-PHY</i> and <i>Edge Clock Mode == Internal</i> or when <i>PHY Mode == Hard D-PHY</i> .
ref_rst_n_i	Input	Asynchronous active-low reset input. This is synchronized internally with ref_clk_i clock. This resets all logic and blocks in the ref_clk_i domain. Available if <i>PHY Mode == Soft D-PHY</i> and <i>Edge Clock Mode == Internal</i> or when <i>PHY Mode == Hard D-PHY</i> .
pll_clkop_i	Input	External PLL clock input operating at the D-PHY high-speed clock frequency. This is the 0-degree phase PLL output used as the primary reference for generating the HS clock lane. Available if <i>PHY Mode == Soft D-PHY</i> and <i>Edge Clock Mode == Internal</i> .
pll_clkos_i	Input	External PLL clock input that is 90 degrees phase-shifted from pll_clkop_i. Provides the quadrature HS clock required for DDR edge alignment in the D-PHY clock lane. Available if <i>PHY Mode == Soft D-PHY</i> .
axis_vid_clk_i	Input	When <i>Unified Video Streaming Interface (UVSI)</i> is checked: Clock input to the pixel-to-byte conversion logic and line buffer. The clock frequency is recommended to be determined by the selected video parameters such as resolution, frame rate, and pixel format. The IP includes a configurable line buffer that operates using a store-and-forward mechanism. When <i>MIPI Byte Streaming Interface (MBSI)</i> is checked: Clock input to the byte buffer write operation to store incoming MIPI packets.
axis_vid_rst_n_i	Input	Active-low AXI4-Stream video reset input for the Pixel-to-Byte converter. This is synchronized internally with axis_vid_clk_i. Resets logic in pixel domain.
s_axi_aclk_i	Input	The AXI4-Lite interface clock that drives the control and status register (CSR) logic and operates at a maximum frequency of 60 MHz.
s_axi_aresetn_i	Input	Active-low AXI4-Lite reset input for control and status registers.
Edge Clock Sharing		
byte_clk_i	Input	Byte clock input. Connects to byte_clk_o of source instance with <i>Edge Clock Mode == Internal</i> . Available if <i>Edge Clock Mode == External</i> ³ .
eclk_syncclk_i	Input	Drives the Edge Clock pin of DDR modules of data path. Connects to eclk_syncclk_o of source instance with <i>Edge Clock Mode == Internal</i> . Available if <i>Edge Clock Mode == External</i> ³ .
eclk_reset_i	Input	Drives the reset port of DDR modules. Connects to eclk_reset_o of source instance with <i>Edge Clock Mode == Internal</i> . Available if <i>Edge Clock Mode == External</i> ³ .
eclk_ready_i	Input	Indicates the clock synchronization status to sink instance. Connects to eclk_ready_o of source instance with <i>Edge Clock Mode == Internal</i> . Available if <i>Edge Clock Mode == External</i> ³ .
byte_clk_o	Output	Byte clock output. Byte Clock is generated by gearing down pll_clkop_i and operates as a divided version of the PLL output clock. Connects to byte_clk_i of sink instance with <i>Edge Clock Mode == External</i> . Available if <i>Edge Clock Mode == Internal</i> ³ or when <i>PHY Mode == Hard D-PHY</i> .
eclk_syncclk_o	Output	Drives the Edge Clock pin of DDR modules of data path. Connects to eclk_syncclk_i of sink instance with <i>Edge Clock Mode == External</i> .

Port	Type	Description
		Available if <i>Edge Clock Mode == Internal</i> ³ .
eclk_reset_o	Output	Drives the reset port of DDR modules. Connects to eclk_reset_i of sink instance with <i>Edge Clock Mode == External</i> . Available if <i>Edge Clock Mode == Internal</i> ³ .
eclk_ready_o	Output	Indicates the clock synchronization status of source instance. Connects to eclk_ready_i of sink instance with <i>Edge Clock Mode == External</i> . Available if <i>Edge Clock Mode == Internal</i> ³ .

Notes:

1. Refer to the [Unified Video Streaming Interface \(UVSI\)](#) section for details.
2. The maximum supported clock frequency depends on the family, package, and performance grade of the device. Check the device data sheet for more information.
3. Refer to the [MIPI CSI/DSI Transmitter Edge Clock Sharing](#) section for details.

4.1.2. D-PHY Interface

Table 4.2. D-PHY Interface Ports

Port	Type	Description
clk_p_io	Inout	Positive differential clock I/O pad for D-PHY implementations.
clk_n_io	Inout	Negative differential clock I/O pad for D-PHY implementations.
d_p_io[NUMLANE ¹ -1:0]	Inout	Positive differential data I/O pad for D-PHY implementations.
d_n_io[NUMLANE ¹ -1:0]	Inout	Negative differential data I/O pad for D-PHY implementations.

Note:

1. Corresponds to the *Number of D-PHY Lanes* attribute.

4.1.3. Unified Video Streaming Interface

Table 4.3. Unified Video Streaming Interface Ports

Port	Type	Description
axis_vid_tready_o	Output	AXI4-Stream ready signal from the TX module. Indicates that the module is ready to accept new pixel data. When this signal is low, upstream video logic must pause data transmission.
axis_vid_tvalid_i	Input	AXI4-Stream valid signal from upstream video logic. Indicates that axis_vid_tdata_i contains valid pixel data for transmission.
axis_vid_tdata_i[TDATA_WIDTH-1:0]	Input	AXI4-Stream pixel data bus carrying incoming formatted pixel data. Pixel values are packed LSB-first. Multiple pixels may be concatenated when PPC > 1. Data is accepted only when axis_vid_tvalid_i and axis_vid_tready_o are both high. Refer to the TDATA Mapping and Unified Video Streaming Interface (UVSI) sections for details.
axis_vid_tuser_i[1:0]	Input	AXI4-Stream video user sideband signals. Provides frame and line timing information. Bit [0]: Start of Frame (SOF) – Asserts for first pixel of each frame. Bit [1]: Interlaced F1 – Currently unused and is tied to low.
axis_vid_tlast_i	Input	AXI4-Stream End-of-Line indicator. Asserts with the last pixel of each video line and marks completion of a line transfer.

4.1.4. MIPI Byte Streaming Interface

Table 4.4. MIPI Byte Streaming Interface Ports

Port	Type	Description
axis_byte_tready_o	Output	AXI4-Stream byte ready output to upstream video processing logic. When asserted, the IP is ready to accept byte data. When deasserted, the IP applies backpressure to the upstream source, indicating insufficient internal buffering or flow control conditions.

Port	Type	Description
axis_byte_tvalid_i	Input	AXI4-Stream byte valid input. Indicates availability of valid byte data on axis_byte_tdata_i. Used for standard AXI4-Stream handshaking protocol.
axis_byte_tdata_i[TDATA_WIDTH-1:0]	Input	AXI4-Stream byte data input containing raw MIPI byte data packed as specified in MIPI specifications. Valid only when axis_byte_tvalid_i is asserted. When <i>PHY Mode == Soft D-PHY</i> , TDATA_WIDTH is <i>PHY Data Width</i> × 4. When <i>PHY Mode == Hard D-PHY</i> and <i>Number of D-PHY Lanes == 4</i> , TDATA_WIDTH is <i>PHY Data Width</i> × 4, else TDATA_WIDTH is 32.
axis_byte_tuser_i[77:0]	Input	AXI4-Stream byte user sideband signals. axis_byte_tuser_i[5:2] carries the virtual channel information of Header 1 when <i>PHY Data Width</i> is 8. axis_byte_tuser_i[43:40] carries the virtual channel information of Header 2 when <i>PHY Data Width</i> is 16. All remaining bits of axis_byte_tuser_i are reserved and must be driven to 0.
axis_byte_tlast_i	Input	AXI4-Stream byte signal indicating the last byte transfer of a long packet. This signal coincides with the payload MSB footer (CRC[15:8]).

4.1.5. Miscellaneous

Table 4.5. Miscellaneous Ports

Port	Type	Description
tinit_done_o	Output	Indicates completion of the D-PHY LP-11 initialization period. When <i>Enable tINIT Counter</i> is checked, this signal asserts after (<i>tINIT counter value</i> – 1) byte-clock cycles, representing the programmed LP-11 duration. When the <i>Enable tINIT Counter</i> is unchecked, the signal asserts automatically when the internal soft PHY has entered and stabilized in LP-11 state, indicating that the link is ready for HS entry.
horizontal_blanking_cnt_en_o	Output	Indicates that the additional horizontal line blanking is inserted in the LP-11 state between HS burst of consecutive long packets (Line Start and Line End is disabled) or between HS burst of the current Line End to the next Line Start of short packets (Line Start and Line End is enabled). The duration of the assertion indicates the additional blanking duration period inserted in the LP-11 state. This signal is active only when the <i>Number of Horizontal Blanking Bytes in Byte Clock Domain</i> is set to a non-zero value. This port is available only when <i>Blanking Enable</i> is set.
vertical_blanking_cnt_en_o	Output	Indicates that the additional vertical frame blanking is inserted in the LP-11 state between HS burst of the current Frame End to the next Frame Start of short packets. The duration of the assertion indicates the additional blanking duration period inserted in the LP-11 state. This signal is active only when the <i>Number of Vertical Blanking Lines Per Frame</i> is set to a non-zero value. This port is available only when <i>Blanking Enable</i> is set.

4.1.6. AXI4-Lite Subordinate Interface

Table 4.6. AXI4-Lite Subordinate Interface Ports

Port	Type	Description
s_axi_awaddr_i[11:0]	Input	AXI4-Lite write address input
s_axi_awprot_i[2:0]	Input	AXI4-Lite write protection attributes
s_axi_awvalid_i	Input	AXI4-Lite write address valid
s_axi_awready_o	Output	AXI4-Lite write address ready
s_axi_wdata_i[31:0]	Input	AXI4-Lite write data input
s_axi_wstrb_i[3:0]	Input	AXI4-Lite write byte enable strobes
s_axi_wvalid_i	Input	AXI4-Lite write data valid

Port	Type	Description
s_axi_wready_o	Output	AXI4-Lite write data ready
s_axi_bresp_o[1:0]	Output	AXI4-Lite write response status
s_axi_bvalid_o	Output	AXI4-Lite write response valid
s_axi_bready_i	Input	AXI4-Lite write response ready
s_axi_araddr_i[11:0]	Input	AXI4-Lite read address input
s_axi_arprot_i[2:0]	Input	AXI4-Lite read protection attributes
s_axi_arvalid_i	Input	AXI4-Lite read address valid
s_axi_arready_o	Output	AXI4-Lite read address ready
s_axi_rdata_o[31:0]	Output	AXI4-Lite read data output
s_axi_rresp_o[1:0]	Output	AXI4-Lite read response status
s_axi_rvalid_o	Output	AXI4-Lite read data valid
s_axi_rready_i	Input	AXI4-Lite read data ready

4.2. MIPI CSI/DSI RX Signal Description

4.2.1. Clock Interface

Table 4.7. Clock and Reset Ports¹

Port	Type	Description
ref_clk_i	Input	This is the clock input for soft PHY fabric-based implementation, and Hard D-PHY automatically generated power-up sequence and escape clock. Available if <i>PHY Mode == Soft D-PHY</i> or when <i>PHY Mode == Hard D-PHY</i> .
ref_rst_n_i	Input	Asynchronous active-low reset input. This is synchronized internally with ref_clk_i clock. This resets all logic and blocks in the ref_clk_i domain. Available if <i>PHY Mode == Soft D-PHY</i> or when <i>PHY Mode == Hard D-PHY</i> .
byte_clk_o	Output	Byte clock output recovered from the MIPI data stream. Represents the rate at which payload bytes are extracted from received packets. This is active only when the clock lane is in high-speed mode. Frequency is displayed as <i>Actual Byte Clock Frequency (MHz)</i> .
fr_clk_i	Input	Free-running clock. Determines the rate at which received MIPI packets are processed. Drives the logic that detects the low power to high-speed transition of clock and data lanes.
fr_rst_n_i	Input	Asynchronous active-low reset input. This is synchronized internally with fr_clk_i clock. Resets all logic and blocks in the fr_clk_i domain.
axis_vid_clk_i	Input	AXI4-Stream video clock input for the Byte-to-Pixel converter output interface. Drives the pixel processing pipeline including format conversion, pixel packing, and AXI4-Stream protocol generation.
axis_vid_rst_n_i	Input	Active-low AXI4-Stream video reset input for the Byte-to-Pixel converter. This is synchronized internally with axis_vid_clk_i. Resets logic in pixel domain.
s_axi_aclk_i	Input	AXI4-Lite interface clock input.
s_axi_aresetn_i	Input	Active-low AXI4-Lite interface reset input.

Note:

1. The operating clock frequency and requirements information are available in the [MIPI CSI/DSI Receiver Clocking Overview](#) section.

4.2.2. D-PHY Interface

Table 4.8. D-PHY Interface Ports

Port	Type	Description
clk_p_io	Inout	Positive differential clock I/O pad for D-PHY implementations.

Port	Type	Description
clk_n_io	Inout	Negative differential clock I/O pad for D-PHY implementations.
d_p_io[NUMLANE ¹ -1:0]	Inout	Positive differential data I/O pad for D-PHY implementations.
d_n_io[NUMLANE ¹ -1:0]	Inout	Negative differential data I/O pad for D-PHY implementations.

Note:

1. Corresponds to the *Number of D-PHY Lanes* attribute.

4.2.3. Unified Video Streaming Interface

Table 4.9. Unified Video Streaming Interface Ports

Port	Type	Description
axis_vid_tready_i	Input	AXI4-Stream video ready input from downstream video processing logic. When low, causes the byte-to-pixel converter to pause pixel output and may trigger internal buffering or flow control mechanisms.
axis_vid_tvalid_o	Output	AXI4-Stream video valid output. Indicates availability of valid pixel data on axis_vid_tdata_o. Used for standard AXI4-Stream handshaking protocol.
axis_vid_tdata_o[TDATA_WIDTH-1:0]	Output	AXI4-Stream video data output containing converted pixel data in the selected format. Pixel data is packed LSB-first. Multiple pixels are concatenated for <i>Number of Pixel per Clock (PPC)</i> > 1. Valid only when axis_vid_tvalid_o is asserted. Minimum TDATA_WIDTH width is 1 byte and may include padding bits for alignment to byte boundaries. Refer to the Unified Video Streaming Interface (UVSI) section for details.
axis_vid_tuser_o[1:0]	Output	AXI4-Stream video user sideband signals. Provides frame and line timing information. axis_vid_tuser_o [0]: Start of Frame (SOF) – Asserts for first pixel of each frame. axis_vid_tuser_o [1]: Interlaced F1 – Currently unused and is tied to low.
axis_vid_tlast_o	Output	AXI4-Stream video signal indicating the last pixel transfer of a video line. This signal always coincides with the last pixel of the line.

4.2.4. MIPI Byte Streaming Interface

Table 4.10. MIPI Byte Streaming Interface Ports

Port	Type	Description
axis_byte_tready_i	Input	AXI4-Stream byte ready input from downstream video processing logic. When low, the byte streamer pauses byte data output and may trigger internal buffering or flow control mechanisms.
axis_byte_tvalid_o	Output	AXI4-Stream byte valid output. Indicates availability of valid byte data on axis_byte_tdata_o. Used for the standard AXI4-Stream handshaking protocol.
axis_byte_tdata_o[TDATA_WIDTH-1:0]	Output	AXI4-Stream byte data output containing raw MIPI byte data packed as specified in MIPI specifications. Valid only when axis_byte_tvalid_o is asserted. When <i>PHY Mode</i> == <i>Soft D-PHY</i> , TDATA_WIDTH is <i>PHY Data Width</i> × 4. When <i>PHY Mode</i> == <i>Hard D-PHY</i> and <i>Number of D-PHY Lanes</i> == 4, TDATA_WIDTH is <i>PHY Data Width</i> × 4, else TDATA_WIDTH is 32.
axis_byte_tuser_o[77:0]	Output	AXI4-Stream byte user sideband signals. axis_byte_tuser_o[5:2] carries the virtual channel information of Header 1 axis_byte_tuser_o[43:40] carries the virtual channel information of Header 2 when <i>PHY Data Width</i> is 16. When <i>Protocol Mode</i> is DSI and with MPPT in a single HS burst, the virtual channel may not align with the actual axis_byte_tdata_o. All other remaining bits, even if unused, are still available and driven low.
axis_byte_tlast_o	Output	AXI4-Stream byte signal indicating the last byte transfer of HS packet Asserted once per HS transaction, specifically on the final content beat.

4.2.5. AXI4-Lite Subordinate Interface

Table 4.11. AXI4-Lite Subordinate Interface Ports

Port	Type	Description
s_axi_awaddr_i[11:0]	Input	AXI4-Lite write address input
s_axi_awprot_i[2:0]	Input	AXI4-Lite write protection attributes
s_axi_awvalid_i	Input	AXI4-Lite write address valid
s_axi_awready_o	Output	AXI4-Lite write address ready
s_axi_wdata_i[31:0]	Input	AXI4-Lite write data input
s_axi_wstrb_i[3:0]	Input	AXI4-Lite write byte enable strobes
s_axi_wvalid_i	Input	AXI4-Lite write data valid
s_axi_wready_o	Output	AXI4-Lite write data ready
s_axi_bresp_o[1:0]	Output	AXI4-Lite write response status
s_axi_bvalid_o	Output	AXI4-Lite write response valid
s_axi_bready_i	Input	AXI4-Lite write response ready
s_axi_araddr_i[11:0]	Input	AXI4-Lite read address input
s_axi_arprot_i[2:0]	Input	AXI4-Lite read protection attributes
s_axi_arvalid_i	Input	AXI4-Lite read address valid
s_axi_arready_o	Output	AXI4-Lite read address ready
s_axi_rdata_o[31:0]	Output	AXI4-Lite read data output
s_axi_rresp_o[1:0]	Output	AXI4-Lite read response status
s_axi_rvalid_o	Output	AXI4-Lite read data valid
s_axi_rready_i	Input	AXI4-Lite read data ready

5. Register Description

The control and status register (CSR) interface is implemented using an AXI4-Lite subordinate interface and is used to configure, control, and monitor the IP.

5.1. MIPI CSI/DSI Transmitter Register Description

Table 5.1. Summary of MIPI CSI/DSI Transmitter Registers

Address Offset	Register Name	Description
0x000	STATUS_REG	Status register
0x020	CONTROLLER_STATUS_REG	Controller status register
0x044	CONVERTER_STATUS_REG1	Pixel-to-Byte converter status Register 1
0x210	PHY_CONFIG_REG	PHY configuration register
0x220	CONFIG_REG0	Configuration Register 0
0x240	CONFIG_REG1	Configuration Register 1
0x244	CONFIG_REG2	Configuration Register 2

5.1.1. STATUS_REG (0x000)

Table 5.2. STATUS_REG (0x000) Register

Field	Name	Description	Access	Default
[2:0]	MAX_SUPPORTED_LANES	Maximum number of lanes supported (1–4). 1: x1 lane 2: x2 lane 4: x4 lane	RO	Number of D-PHY Lanes ¹
[3]	PHY_READY	When the <i>PHY Mode == Soft D-PHY</i> , this signal indicates the completion of the synchronization sequence and signifies that the PHY is ready. When the <i>PHY Mode == Hard D-PHY</i> , this signal directly indicates that PHY is ready.	RO	0x0
[4]	INIT_SKEWCAL_DONE	Initial skew calibration is complete. This register is applicable only when <i>PHY Mode == Hard D-PHY</i> .	RO	0x0
[31:5]	RSVD	Reserved.	RO	0x0

Note:

- The default value is based on the configurations set for the corresponding attribute in the [IP Parameter Description](#) section.

5.1.2. CONTROLLER_STATUS_REG (0x020)

Table 5.3. STATUS_REG (0x000) Register

Field	Name	Description	Access	Default
[0]	CFG_READY	Configuration ready.	RO	0x0
[31:1]	RSVD	Reserved.	RO	0x0

5.1.3. CONVERTER_STATUS_REG1 (0x044)

Table 5.4. CONVERTER_STATUS_REG1 (0x044) Register

Field	Name	Description	Access	Default
[3:0]	PPC	Pixel per clock. 0x1 = 1 pixel per clock 0x2 = 2 pixels per clock	RO	Pixels Per Clock ¹

Field	Name	Description	Access	Default
		0x4 = 4 pixels per clock		
[7:4]	SUPPORTED_COLOUR_FORMAT	Supported colour format. 0x0 – RGB888 0x1 – RGB565 0x2 – RAW10 0x3 – RAW12	RO	Color Format ¹
[31:8]	RSVD	Reserved.	RO	0x0

Note:

- The default value is based on the configurations set for the corresponding attribute in the [IP Parameter Description](#) section.

5.1.4. PHY_CONFIG_REG (0x210)

Table 5.5. PHY_CONFIG_REG (0x210) Register

Field	Name	Description	Access	Default
[2:0]	NUM_LANES	Active lanes 1–4 (bounded by MAX_SUPPORTED_LANES). 1: x1 lane 2: x2 lane 4: x4 lane	RW	Number of D-PHY Lanes ¹
[3]	NUM_LANES_UPDATE	Write enable for NUM_LANES. Write a 1 to update the NUM_LANES bitfield.	WO	0x0
[31:4]	RSVD	Reserved.	RO	0x0

Note:

- The default value is based on the configurations set for the corresponding attribute in the [IP Parameter Description](#) section.

5.1.5. CONFIG_REG0 (0x220)

Table 5.6. CONFIG_REG0 (0x220) Register

Field	Name	Description	Access	Default
[14:0]	T_SKEW_INITIAL	Period of initial skew calibration in word clock cycle. This register is applicable only when <i>PHY Mode</i> == <i>Hard D-PHY</i> . Example: <i>PHY Mode</i> = <i>Hard D-PHY</i> <i>Line Rate Per Lane</i> = 2500.0 Mbps <i>PHY Data Width</i> = 16 <i>T_SKEW_INITIAL</i> = 2049 (0x0801) <i>Byte Clock Period</i> = <i>PHY Data Width</i> / <i>Line Rate Per Lane</i> × 1000 = 16 / 2500.0 × 1000 = 6.4 ns Programming T_SKEW_INITIAL with a value of 2049 (0x0801) sets a skew calibration period of 2049 × 6.4 ns. When triggered via the CAL_INITIAL_TRIGGER_EN register bit, this results in a calibration time of approximately 13113.6 ns across the serial links.	RW	Period of Initial Skew Calibration ¹
[15]	RSVD0	—	RO	0x0
[27:16]	T_SKEW_PERIODIC	Period of periodic skew calibration in word clock cycle. This register is applicable only when <i>PHY Mode</i> == <i>Hard D-PHY</i> . Example: <i>PHY Mode</i> = <i>Hard D-PHY</i> <i>Line Rate Per Lane</i> = 2500.0 Mbps <i>PHY Data Width</i> = 16	RW	Period of Periodic Skew Calibration ¹

Field	Name	Description	Access	Default
		$T_SKEW_INITIAL = 65$ (0x041) $Byte\ Clock\ Period = PHY\ Data\ Width / Line\ Rate\ Per\ Lane \times 1000 = 16 / 2500.0 \times 1000 = 6.4\ ns$ Programming $T_SKEW_INITIAL$ with a value of 65 (0x041) sets a skew calibration period of $65 \times 6.4\ ns$. When triggered via the $CAL_PERIODIC_EN$ register bit, this results in a calibration time of approximately 416.0 ns across the serial links.		
[28]	CAL_INITIAL_EN	Status of initial skew calibration enable. This register is applicable only when <i>PHY Mode == Hard D-PHY</i> .	RO	<i>Enable Manual Initial Skew Calibration</i> ¹
[29]	CAL_INITIAL_TRIGGER_EN	Triggers an initial skew calibration (manual). This register is applicable only when <i>PHY Mode == Hard D-PHY</i> .	RW	0x0
[30]	CAL_PERIODIC_EN	Enables the periodic skew calibration. This register is applicable only when <i>PHY Mode == Hard D-PHY</i> .	RW	<i>Enable Periodic Skew Calibration</i> ¹
[31]	RSVD	—	RO	0x0

Note:

- The default value is based on the configurations set for the corresponding attribute in the [IP Parameter Description](#) section.

5.1.6. CONFIG_REG1 (0x240)

Table 5.7. CONFIG_REG1 (0x240) Register

Field	Name	Description	Access	Default
[15:0]	NUM_LINE_V_BLANKING	Number of lines for vertical blanking. This register is applicable only when <i>Blanking Enable</i> is set.	RW	<i>Number of Vertical Blanking Lines Per Frame</i> ¹
[31:16]	H_BLANKING_BYTES_TH	Number of bytes for horizontal blanking. This register is applicable only when <i>Blanking Enable</i> is set.	RW	<i>Number of Horizontal Blanking Bytes in Byte Clock Domain</i> ¹

Note:

- The default value is based on the configurations set for the corresponding attribute in the [IP Parameter Description](#) section.

5.1.7. CONFIG_REG2 (0x244)

Table 5.8. CONFIG_REG2 (0x244) Register

Field	Name	Description	Access	Default
[15:0]	NUM_BYTES_ACTIVE_H	Number of bytes for active horizontal. This register is applicable only when <i>Blanking Enable</i> is set.	RW	<i>Number of Horizontal Active Bytes in Byte Clock Domain</i> ¹
[31:16]	NUM_LINE_ACTIVE_V	Number of lines for active vertical.	RW	<i>Number of Active Lines Per Frame</i> ¹

Note:

- The default value is based on the configurations set for the corresponding attribute in the [IP Parameter Description](#) section.

5.2. MIPI CSI/DSI Receiver Register Description

Table 5.9. Summary of MIPI CSI/DSI Receiver Registers

Offset Address	Register Name	Description
0x000	STATUS_REG	Soft or Hard D-PHY status register
0x020	CONTROLLER_STATUS_REG	D-PHY controller status register
0x040	CONVERTER_STATUS_REG	Byte-to-Pixel status register
0x200	CONFIG_REG	Generic configuration register
0x210	PHY_CONFIG_REG	Soft or Hard D-PHY configuration register

5.2.1. STATUS_REG (0x000)

Table 5.10. Soft D-PHY STATUS_REG (0x000) Register

Field	Name	Description	Access	Default
[2:0]	MAX_SUPPORTED_LANES	Maximum number of lanes supported. 0x01: x1 lane 0x02: x2 lane 0x04: x4 lane	RO	<i>Number of D-PHY Lanes</i>
[6:3]	RSVD	Reserved.	RO	0x0
[7]	DDR_READY	Ready output signal indicating the initial DDR synchronization sequence is complete and PHY is ready. When <i>PHY Clock Mode</i> is set to Non-Continuous, this signal deasserts when D-PHY clock lane enters LP mode and asserts only when DDR resynchronization is complete.	RO	0x0
[8]	RSVD	Reserved.	RO	0x0
[12:9]	SOT_SYNC_ERR	SOT synchronization error per lane. This is limited to error detection only and cannot correct 1-bit error detected in the SYNC code.	RW1C	0x0
[31:13]	RSVD	Reserved.	RO	0x0

Table 5.11. Hard D-PHY STATUS_REG (0x000) Register

Field	Name	Description	Access	Default
[2:0]	MAX_SUPPORTED_LANES	Maximum number of lanes supported (1–4). 0x01: x1 lane 0x02: x2 lane 0x04: x4 lane	RO	<i>Number of D-PHY Lanes</i>
[6:3]	SKEWCAL_DET	Indicates that skew calibration burst is detected. This is applicable only when <i>Line Rate per Lane</i> > 1.5 Gbps. For other cases, this register is tied to 0x0.	RO	0x0
[10:7]	SKEWCAL_DONE	Indicates that skew calibration is complete.	RW1C	0x0
[14:11]	SOT_SYNC_ERR	SoT synchronization error per lane. This is asserted when the high-speed SoT leader sequence is corrupted, and synchronization cannot be achieved.	RW1C	0x0
[18:15]	CONTROL_ERR	Control error per lane. This bit is asserted when an incorrect state sequence is detected in D-PHY lane.	RW1C	0x0
[22:19]	SOTHS_ERR	SoT error per lane. This is asserted when the high-speed SoT	RW1C	0x0

Field	Name	Description	Access	Default
		leader sequence is corrupted, but proper synchronization can still be achieved. This is classified as a soft error in the leader sequence, indicating reduced confidence in the payload data.		
[31:23]	RSVD	Reserved.	RO	0x0

5.2.2. CONTROLLER_STATUS_REG (0x020)

Table 5.12. CONTROLLER_STATUS_REG (0x020) Register

Field	Name	Description	Access	Default
[3:0]	LANE_ALIGNER_FIFO_FULL	Lane aligner FIFO full flag status per lane.	RW1C	0x0
[4]	MBSI_FIFO_FULL	Byte streaming source FIFO full flag. This is applicable only when <i>MIPI Byte Streaming Interface</i> is enabled.	RW1C	0x0
[5]	PAYLOAD_CRC_ERR	When asserted, indicates the packet footer CRC bytes do not match the computed payload CRC bytes.	RW1C	0x0
[6]	ECC_BYTE_ERR	When asserted, indicates that a flipped bit is detected in the packet header ECC byte. This flag asserts when the detected error is in the received ECC byte.	RW1C	0x0
[7]	ECC_1BIT_ERR	When asserted, indicates that one bit error is detected and corrected in the packet header.	RW1C	0x0
[8]	ECC_2BIT_ERR	When asserted, indicates that two or more errors are detected in the packet header.	RW1C	0x0
[31:9]	RSVD	Reserved.	RO	0x0

5.2.3. CONVERTER_STATUS_REG (0x040)

Table 5.13. CONVERTER_STATUS_REG (0x040) Register

Field	Name	Description	Access	Default
[0]	B2P_FIFO_FULL	Byte-to-Pixel FIFO full flag status	RW1C	0x0
[1]	UVSI_FIFO_FULL	UVSI source FIFO full flag status	RW1C	0x0
[31:2]	RSVD	Reserved	RW1C	0x0

5.2.4. CONFIG_REG (0x200)

Table 5.14. CONFIG_REG (0x200) Register

Field	Name	Description	Access	Default
[0]	CFG_READY	Specifies that the configuration registers are set. This register must be set together with other configuration registers. Refer to the MIPI CSI/DSI Receiver Dynamic Lane Reconfiguration section for more details.	RW	0x0
[31:1]	RSVD	Reserved.	RO	0x0

5.2.5. PHY_CONFIG_REG (0x210)

Table 5.15. Soft D-PHY PHY_CONFIG_REG (0x210) Register

Field	Name	Description	Access	Default
[2:0]	NUM_LANES	Active lanes 1–4 (bounded by MAX_SUPPORTED_LANES). 0x01: x1 lane 0x02: x2 lane 0x04: x4 lane	RW	<i>Number of D-PHY Lanes</i>
[3]	NUM_LANES_UPDATE	Write enable for NUM_LANES. Write a 1 to update the NUM_LANES bitfield.	WO	0x0
[31:4]	RSVD	Reserved.	RO	0x0

Table 5.16. Hard D-PHY PHY_CONFIG_REG (0x210) Register

Field	Name	Description	Access	Default
[2:0]	NUM_LANES	Active lanes 1–4 (bounded by MAX_SUPPORTED_LANES). 0x01: x1 lane 0x02: x2 lane 0x04: x4 lane	RW	<i>Number of D-PHY Lanes</i>
[3]	NUM_LANES_UPDATE	Write enable for NUM_LANES. Write a 1 to update the NUM_LANES bitfield.	WO	0x0
[4]	DESKEW_EN	Enables or disables deskew operation.	RW	<i>DESKEW_EN</i>
[5]	DESKEW_EN_UPDATE	Write enable for DESKEW_EN.	WO	0x0
[31:6]	RSVD	Reserved.	RO	0x0

6. Designing with the IP

This section provides information on how to generate the IP core using the Lattice Radiant software and how to run simulation and synthesis. For more details on the Lattice Radiant software, refer to the Lattice Radiant software user guide.

Note: The screenshots provided are for reference only. Details may vary depending on the version of the IP or software being used. If there have been no significant changes to the GUI, a screenshot may reflect an earlier version of the IP.

6.1. Generating and Instantiating the IP

You can use Lattice Radiant software to generate IP modules and integrate them into the device architecture. To generate the MIPI CSI/DSI IP in the Lattice Radiant software, follow these steps:

1. Create a new Lattice Radiant software project or open an existing project.
2. Click the **IP Catalog** button to view the **IP Catalog** pane.
3. On the **IP on Local** tab, double-click **MIPI CSI/DSI** under the **IP, Audio_Video_and_Image_Processing** category. The **Module/IP Block Wizard** opens.

Note: If the IP is not available on the **IP on Local** tab, download the IP from the **IP on Server** tab.

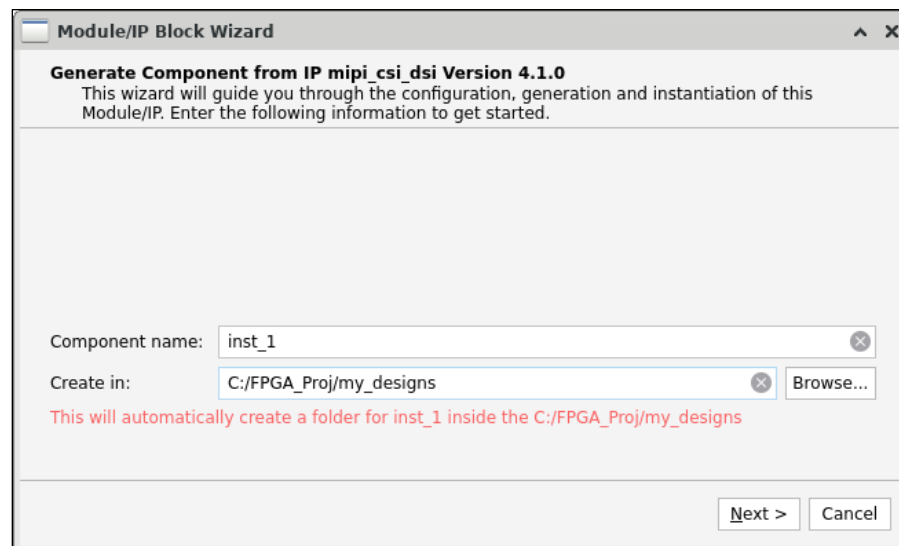


Figure 6.1. Module/IP Block Wizard

4. Enter values in the **Component name** and **Create in** fields, then click **Next**.
5. Customize the selected MIPI CSI/DSI IP using drop-down lists and check boxes. The figures below show the example configurations of the MIPI CSI/DSI IP. For details on the configuration options, refer to the [IP Parameter Description](#) section.

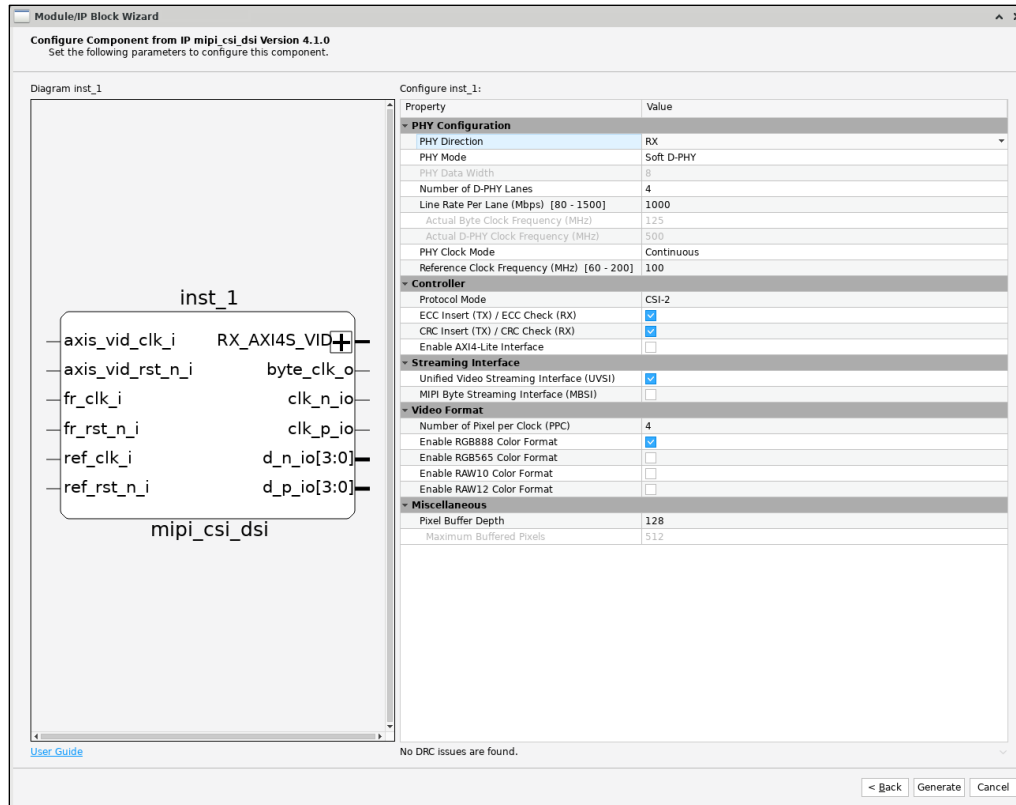


Figure 6.2. MIPI CSI/DSI IP Receiver Mode Configuration

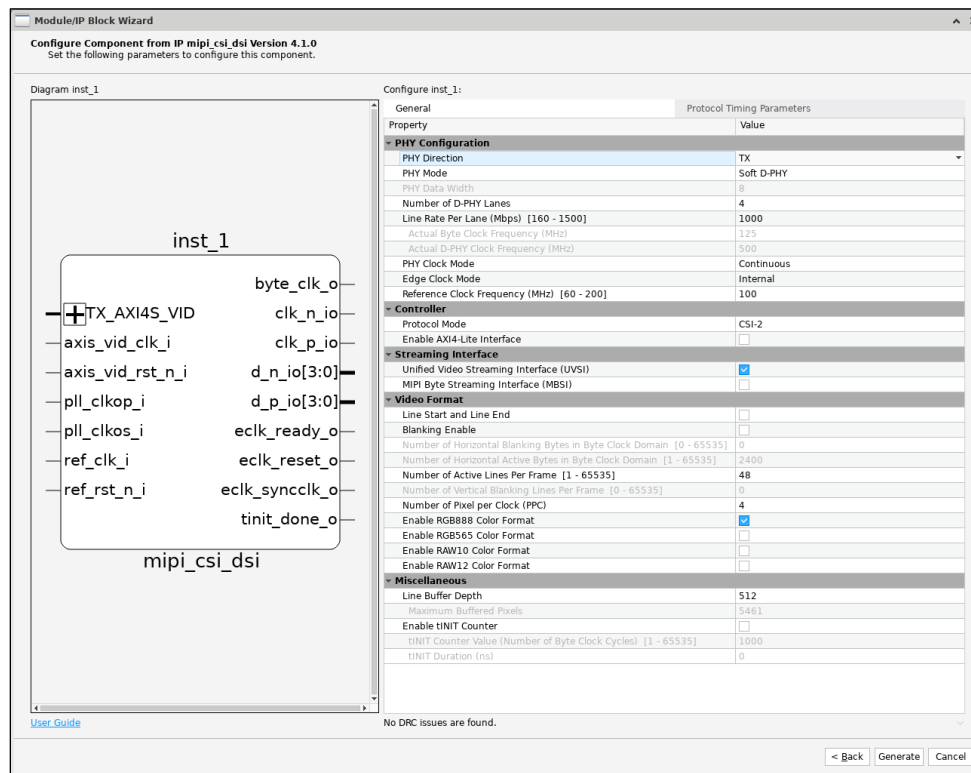


Figure 6.3. MIPI CSI/DSI IP Transmitter Mode Configuration

6. Click **Generate**. The **Check Generated Result** window opens. This window shows design block messages and results.

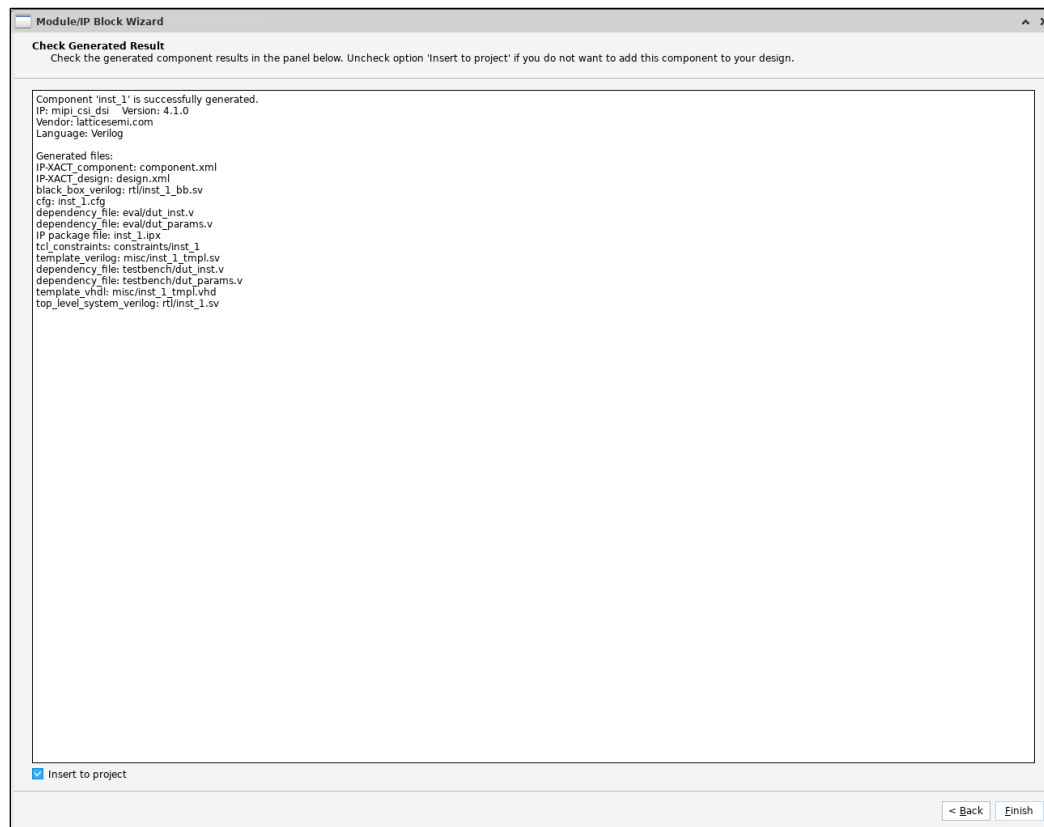


Figure 6.4. Check Generated Result

7. Click **Finish**. All generated files are placed in the directory specified by the **Component name** and **Create in** fields shown in [Figure 6.1](#).

6.1.1. Generated Files and File Structure

The generated MIPI CSI/DSI module package includes the closed-box (*<Component name>_bb.sv*) and instance templates (*<Component name>_tmpl.sv/vhd*) that can be used to instantiate the core in a top-level design. An example RTL top-level reference source file (*<Component name>.sv*) that can be used as an instantiation template for the module is also provided. You may also use this example as the starting template for your top-level design. The generated files are listed in [Table 6.1](#).

Table 6.1. Generated File List

Attribute	Description
<i><Component name>.ipx</i>	Contains the information on the files associated with the generated IP.
<i><Component name>.cfg</i>	Contains the parameter values used in IP configuration.
<i>component.xml</i>	Contains the ipxact: component information of the IP.
<i>design.xml</i>	Documents the configuration parameters of the IP in the IP-XACT 2014 format.
<i>eval/constraint.pdc</i>	Provides reference timing constraints and virtual I/O assignments for IP evaluation in the Lattice Radiant software.
<i>eval/dut_inst.v</i>	Provides a port connection template for instantiating the module.
<i>eval/dut_params.v</i>	Provides the parameter definitions used to configure the module.
<i>eval/eval_top.sv</i>	Provides an example evaluation top-level file for synthesis that instantiates the module.
<i>rtl/<Component name>.sv</i>	Provides an example RTL top file that instantiates the module.

Attribute	Description
rtl/<Component name>_bb.sv	Provides the synthesis closed-box.
misc/<Component name>_tpl.sv misc /<Component name>_tpl.vhd	Provides instance templates for the module.

6.2. Design Implementation

Completing your design includes additional steps to specify analog properties, pin assignments, and timing and physical constraints. You can add and edit the constraints using the Device Constraint Editor or by manually creating a .pdc file.

Post-Synthesis constraint files (.pdc) contain both timing and non-timing *constraint.pdc* source files for storing logical timing/physical constraints. Constraints that are added using the Device Constraint Editor are saved to the active .pdc file. The active post-synthesis design constraint file is then used as input for post-synthesis processes.

The IP contains an evaluation wrapper (<ip_instance_path>/eval/eval_top.sv) that you may use as a top-level wrapper for synthesis. Include this file only for synthesis and do not use this file for simulation.

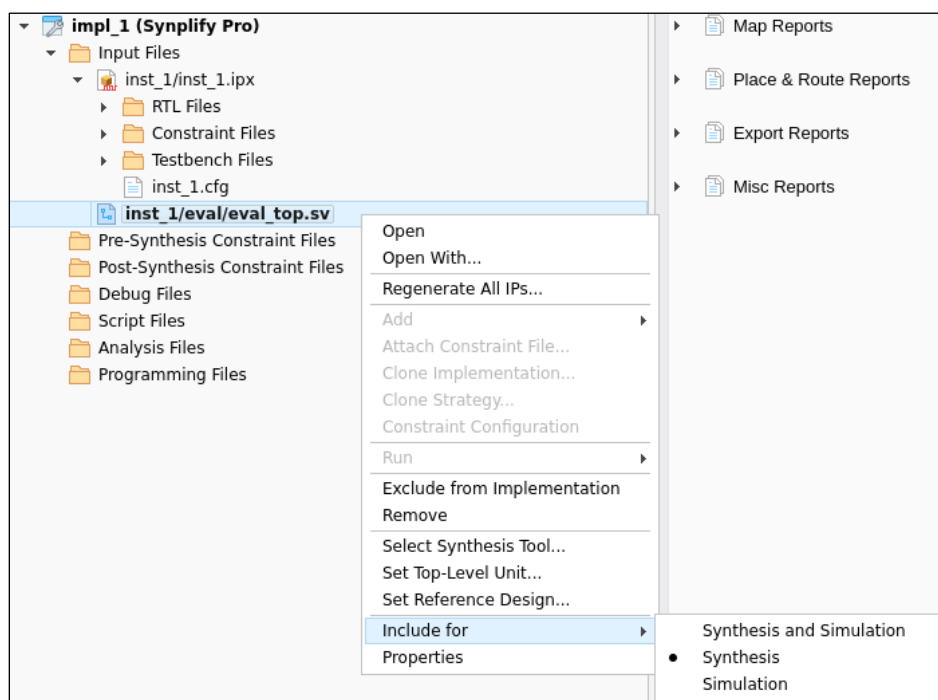


Figure 6.5. Example Evaluation Top Setup

Refer to the relevant sections in the Lattice Radiant software user guide for more information on how to create or edit constraints and how to use the Device Constraint Editor.

6.3. Timing Constraints

The MIPI CSI/DSI IP generates the following constraint files:

- A constraint file in SDC format (<ip_instance_path>/constraints/constraint.sdc) that contains both pre-synthesis and post-synthesis IP constraints. These constraints are automatically used and propagated by the software tool starting from the Lattice Radiant software version 2024.1. These constraints can be modified if you have a thorough understanding of the effect of each constraint.
- An evaluation post-synthesis constraint file in PDC format (<ip_instance_path>/eval/constraint.pdc).

To run the software implementation flow using the provided evaluation file after the IP is generated, follow these steps:

1. In the **Post-Synthesis Constraint Files** section, add `<ip_instance_path>/eval/constraint.pdc`.
2. Run the implementation flow.

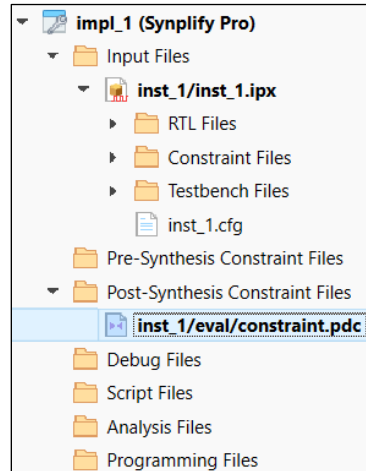


Figure 6.6. Example Evaluation Project Settings

Notes:

- You need to provide proper timing and physical design constraints to ensure that your design meets the desired performance goals for the FPGA.
- The constraint files have been verified during IP evaluation with the evaluation wrapper instantiated directly in the top-level module.
- During synthesis, you can ignore clock related warnings as the evaluation IP does not include clock-related constraints at pre-synthesis level.
- During post-synthesis, there may be warnings related to dropped constraints. As the IP supports many configurations and parameter combinations, some default constraints may not be applicable to the selected configuration.
- In transmitter mode, if *Edge Clock Mode* is set to Internal, you may encounter a Place and Route error as the edge clock synchronizer clock (eclk_syncclk_o) is illegally mapped to the FPGA I/O. This clock is intended to be connected to a DDR primitive or just left unconnected if unused. You may include the *eval_top.sv* as a top-level wrapper during IP synthesis stage to achieve a full compilation stage. The *eval_top.sv* must be set to *Include for – Synthesis* only and avoid using the file as top-level wrapper for simulation stage.

For more information on timing constraints, refer to the [Lattice Radiant Timing Constraints Methodology \(FPGA-AN-02059\)](#).

6.4. Physical Constraints

For MIPI CSI/DSI receiver Soft D-PHY mode, if I/O top-level ports are not automatically routed to MIPI I/O, define the MIPI pins (clk_p_io, d_p_io_*) with IO_TYPE of MIPI_DPHY in the .pdc file. An example is shown in [Figure 6.7](#).

```
ldc_set_port -iobuf {IO_TYPE=MIPI_DPHY} [get_ports {clk_p_io*}]
ldc_set_port -iobuf {IO_TYPE=MIPI_DPHY} [get_ports {d_p_io*}]
```

Figure 6.7. Defining MIPI D-PHY Port Pins Using MIPI_DPHY IO_TYPE

6.5. Specifying the Strategy

The Lattice Radiant software provides two predefined strategies: Area and Timing. The software also enables you to create customized strategies. For details on how to create a new strategy, refer to the Strategies section of the Lattice Radiant software user guide.

6.6. Running Functional Simulation

You can run functional simulation after the IP is generated.

To run functional simulation, follow these steps:

1. Click the  button located on the **Toolbar** to initiate the **Simulation Wizard** shown in [Figure 6.8](#).

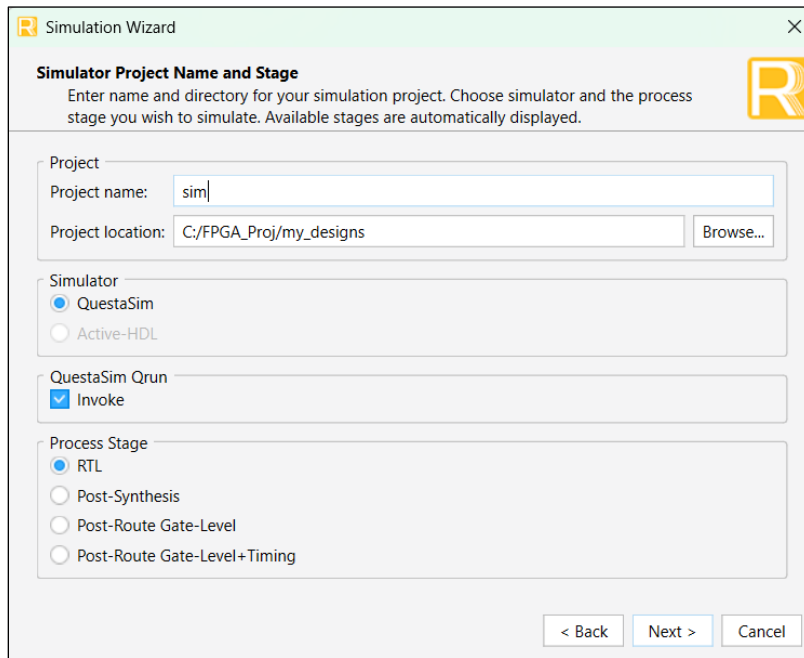


Figure 6.8. Simulation Wizard

2. Click **Next** to open the **Add and Reorder Source** window as shown in [Figure 6.9](#).

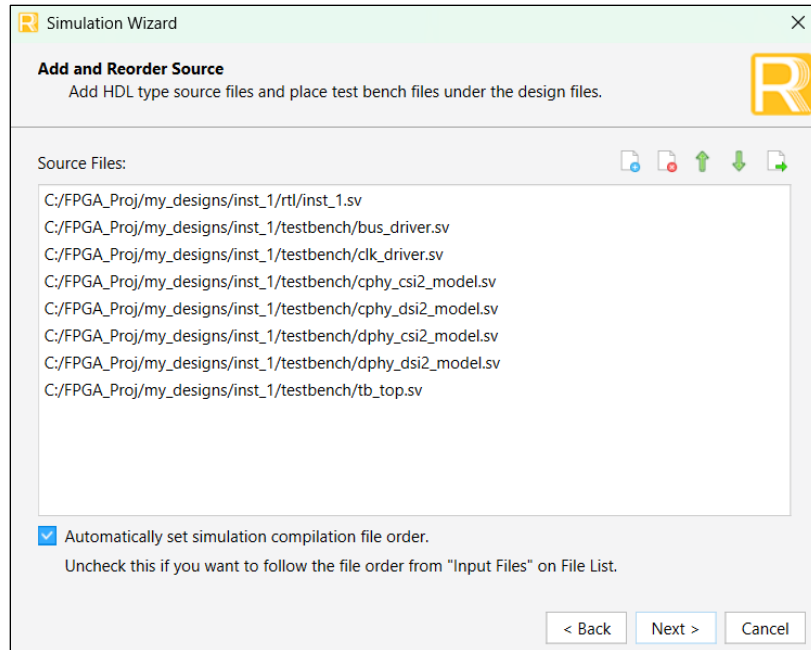


Figure 6.9. Add and Reorder Source

3. Click **Next**. The **Summary** window is shown.
 4. Click **Finish** to run the simulation.
- The waveform in Figure 6.10 shows an example simulation result.

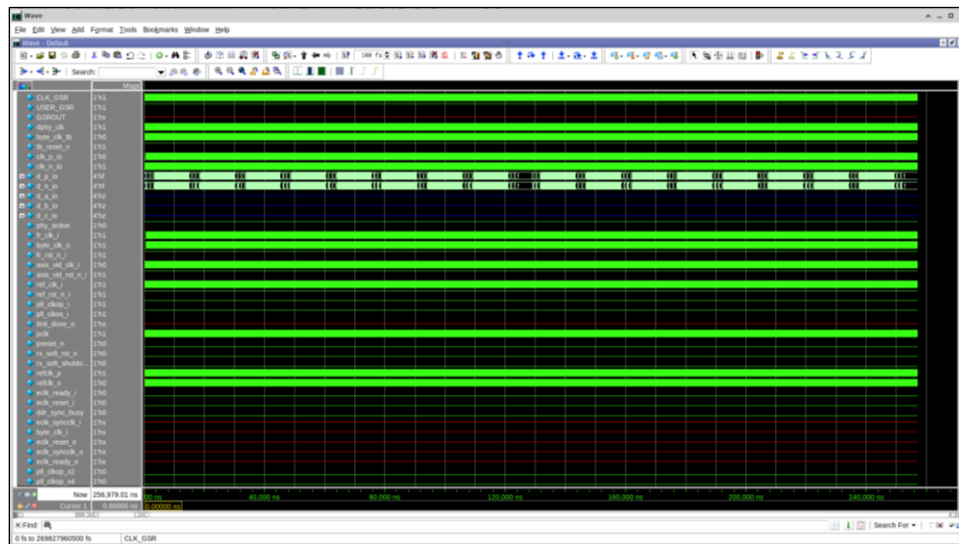


Figure 6.10. Simulation Waveform

6.6.1. Simulation Results

When the simulation is complete, the output in the Transcript window is shown in the following figure.

```
#
# 3577419080 CSI-2 Rx Model Data Reception Complete.
#
# =====
# [BC_ACTIVE] Estimated=1203.0 BC, Avg=1202.46 BC, Min=1202.46, Max=1202.46 (24 long pkts)
# =====
# [FRAME RATE] Avg = 8659.916 fps, Min = 8659.916 fps, Max = 8659.916 fps
# =====
# [LINE TIME] Avg = 14.343 us, Min = 14.134 us, Max = 16.536 us
# =====
# 3577519080 DATA MATCHED: 57600, DATA MISMATCHED: 0
#
# 3577519080 SIMULATION PASSED
# 3577519080 TEST FINISH
#
```

Figure 6.11. Simulation Result

For MIPI CSI/DSI Transmitter, when the simulation is complete, but a checker detects a packet integrity error, the output in the Transcript window is shown in the following figure.

```
# 520335000000 Data[2] : Receiving data = f0
#
# 520335000000 Data[3] : Receiving data = ff
#
# 520343000000 Data[0] : Receiving data = ff
#
# 520343000000 Data[1] : Receiving data = 0f
#
# 520343000000 Data[2] : Receiving data = 00
#
# 520445000000 CRC FAIL
# Errors 1/ 49
# ** Note: $stop :
# Time: 520444500 ps Iteration: 0 Instance: /tb_top/TX_TB/rx_model
```

Figure 6.12. Simulation Result with Error

Additional timing information from simulation is available in the *tb_rx_model_timing.txt* file from the simulation directory created after simulation is complete.

```
=====
BLANKING ANALYSIS SUMMARY
=====
Configuration:
Byte CLK Freq = 114.000 MHz
H_TOTAL = 1920 pixels
H_ACTIVE = 1920 pixels
V_TOTAL = 8 lines
V_ACTIVE = 8 lines
LS_LE_EN = 1
BPP = 10 bits
PIXELS_PER_CLOCK = 4
Byte Clock Domain:
H_TOTAL_BYTES_TH = 1200.0 bytes (BC_ACTIVE+BC_H_BLANK)
H_ACTIVE_BYTES_TH = 1200.0 bytes BC_ACTIVE
H_BLANK_BYTES = 0.0 bytes BC_H_BLANK
V_BLANK_LINES = 0 lines V_BLANK_LINES
V_BLANK_BYTES = 0.0 bytes (V_BLANK_LN * H_TOTAL_BYTES_TH)
=====
HS ACTIVE DATA (BC_ACTIVE) SUMMARY
=====
Avg Measured BC = 1202.46 BC
Estimated BC = 1203.0 BC (PH=2 + Payload=1200 + CRC=1)
Estimated Time = 10552.6 ns
=====
FRAME RATE / FRAME TIMING SUMMARY
=====
Avg Frame Rate = 8659.916 fps
Avg Frame Period = 0.115 ms
Ideal Frame Period(without overhead) = 0.084 ms (11875.000 fps)
=====
LINE TIMING SUMMARY
=====
Avg Line Period = 14.343 us
Ideal Line Period(without overhead) = 10.526 us
```

Figure 6.13. Additional Timing Information

7. Example Design

The MIPI CSI/DSI example design demonstrates the MIPI video streaming receiving and transmitting from the MIPI CSI IP in the FPGA with external MIPI CSI components.

This example design supports the following modes which are selectable using the DIP switch on the FPGA board:

- RX to TX retransmit mode: The external camera sensor (MIPI CSI source) transmitted to the MIPI RX IP (MIPI CSI sink), retransmitted to the MIPI TX IP (MIPI CSI source) before transmitting to the external Raspberry Pi 5 board (MIPI CSI sink).
- Pattern subsystem to TX mode: The internal pattern generator in the FPGA transmitted to the MIPI TX IP (MIPI CSI source) before transmitting to the external Raspberry Pi 5 board (MIPI CSI sink).

Refer to the [Setting Up the CertusPro-NX PCIe Vision Board](#) section for details on the DIP switch settings.

The MIPI CSI/DSI example design validates the MIPI CSI/DSI IP in CertusPro-NX devices using the following hardware:

- CertusPro-NX PCIe Vision Board
- Sony® IMX258 camera sensor
- Raspberry Pi 5 board

7.1. Example Design Supported Configuration

Table 7.1. MIPI CSI/DSI IP Configuration Supported by the Example Design

Attribute	Selectable Values
Common Settings	
PHY Configuration	
PHY Mode	Soft D-PHY
PHY Data Width	8
Reference Clock Frequency	60
Controller	
Protocol Mode	CSI-2
Enable AXI4-Lite Interface	Checked
Dynamic Lane Reconfiguration	Unchecked
Streaming Interface	
Unified Video Streaming Interface (UVSI)	Checked
Byte Streaming Interface	Unchecked
Video Format	
Number of Pixel per Clock (PPC)	4
Enable RAW10 Color Format	Checked
MIPI RX Settings	
Controller	
ECC Check(RX)/Insert(TX)	Checked
CRC Check(RX)/Insert(TX)	Checked
PHY Configuration	
PHY Direction	RX
Number of D-PHY Lanes	4
Line Rate Per Lane (Mbps)	371.25
Actual Byte Clock Frequency (MHz)	46.40625
Actual D-PHY Clock Frequency (MHz)	185.625
PHY Clock Mode	Continuous
Miscellaneous	
Pixel Buffer Depth	512
Maximum Buffered Pixels	2048

Attribute	Selectable Values
MIPI TX Settings	
PHY Configuration	
PHY Direction	TX
Number of D-PHY Lanes	2
Line Rate Per Lane (Mbps)	912
Actual Byte Clock Frequency (MHz)	114
Actual D-PHY Clock Frequency (MHz)	456
PHY Clock Mode	Non-Continuous
Edge Clock Mode	Internal
Video Format	
Line Start and Line End	Checked
Line Number Insertion	Checked
Blanking Enable	Checked
Number of Horizontal Active Bytes in Byte Clock Domain	1200
Number of Horizontal Blanking Bytes in Byte Clock Domain	0
Number of Active Lines Per Frame	1080
Number of Vertical Blanking Lines Per Frame	0
Miscellaneous	
Line Buffer Depth	512
Maximum Buffered Pixels	8192
Enable tINIT Counter	Unchecked
tINIT Counter Value	1000
tINIT Duration (ns)	0
Protocol Timing Parameters	
TX Global Operation Timing Parameters	
t_LPX	6
t_HS-PREPARE	6
t_HS-ZERO	13
t_HS-TRAIL	13
t_HS-EXIT	12
t_CLK-PREPARE	5
t_CLK-ZERO	30
t_CLK-PRE	4
t_CLK-POST	14
t_CLK-TRAIL	8
t_CLK-EXIT	12

For details of each parameter, refer to the [General](#) and [MIPI CSI/DSI TX Protocol Timing Parameters](#) sections.

The Sony IMX258 camera sensor is configured to run in 4 lanes with line rate of 371.25 Mbps for fixed video resolution of 1920 × 1080 (FHD).

7.2. Overview of Example Design and Features

Key features of the example design are as follows:

- MIPI CSI-2 video retransmits in the following sequence:
 - a. External CSI-2 source (camera sensor)
 - b. CSI-2 sink (MIPI CSI RX)
 - c. CSI-2 source (MIPI CSI TX)

- d. External CSI-2 sink (Raspberry Pi 5)
- MIPI CSI-2 video can also retransmit in the following sequence:
 - a. Pattern generator
 - b. CSI-2 source (MIPI CSI TX)
 - c. External CSI-2 sink (Raspberry Pi 5)

7.3. Example Design Overview

7.3.1. Example Design System

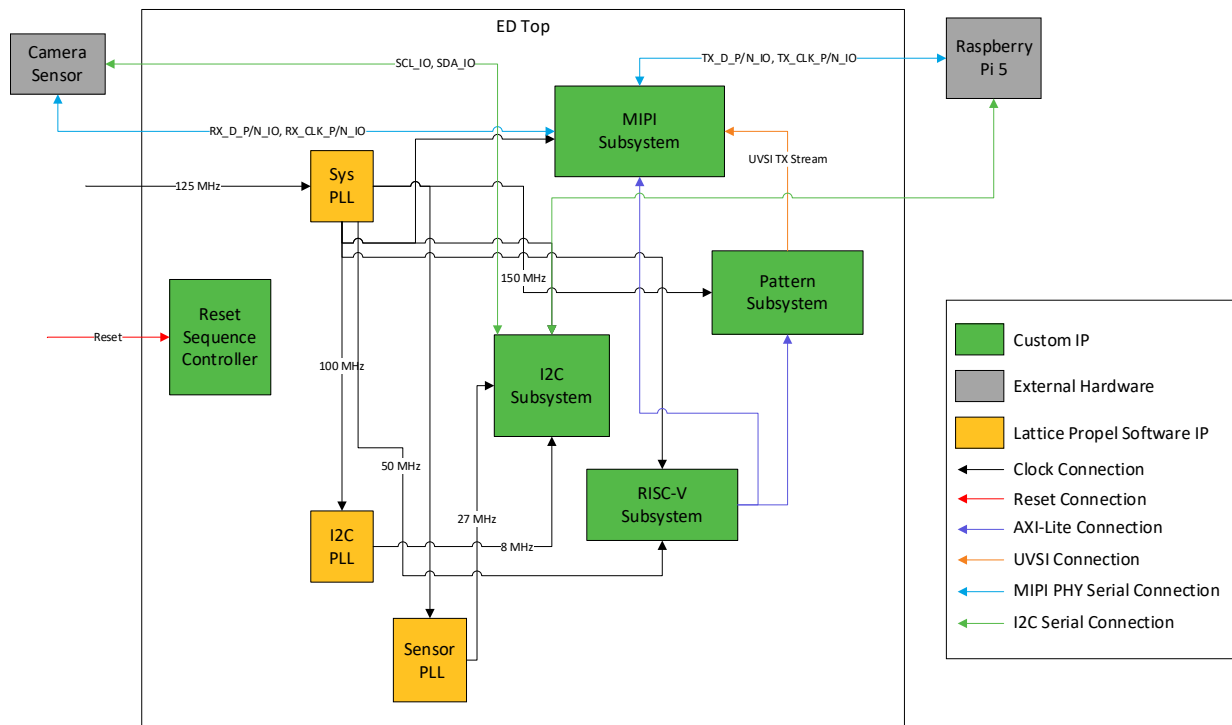


Figure 7.1. Example Design System Diagram

The example design system consists of the following IP and custom components:

- Reset sequence controller
- Sys PLL
- I2C PLL
- Sensor PLL
- MIPI subsystem
- I2C subsystem
- Pattern subsystem
- RISC-V subsystem

7.3.1.1. Reset Sequence Controller

The reset sequence controller handles the reset sequence of the IP and subsystem components inside the FPGA and the sensor camera which is an external component on device power up.

7.3.1.2. Sys PLL IP

The Sys PLL generates the following clock output:

- 150 MHz: Video clock input for the MIPI subsystem and pattern subsystem.
- 100 MHz: Reference clock input to the MIPI subsystem, system clock input to the I2C subsystem, and reference clock input to the I2C PLL.
- 60 MHz: Soft D-PHY reference clock for MIPI TX and RX IPs in the MIPI subsystem.
- 50 MHz: AXI-Lite clock input to the MIPI subsystem and system clock input to the RISC-V subsystem.

7.3.1.3. I2C PLL IP

The I2C PLL generates 8 MHz clock output for the I2C bridge component that resides within the I2C subsystem.

7.3.1.4. Sensor PLL IP

The sensor PLL generates 27 MHz clock output for the sensor LMMI driver component that resides within the I2C subsystem.

7.3.1.5. MIPI Subsystem

The MIPI subsystem consists of Lattice MIPI CSI IPs configured in transmitter and receiver modes and the TX PLL IP for PHY clock generation.

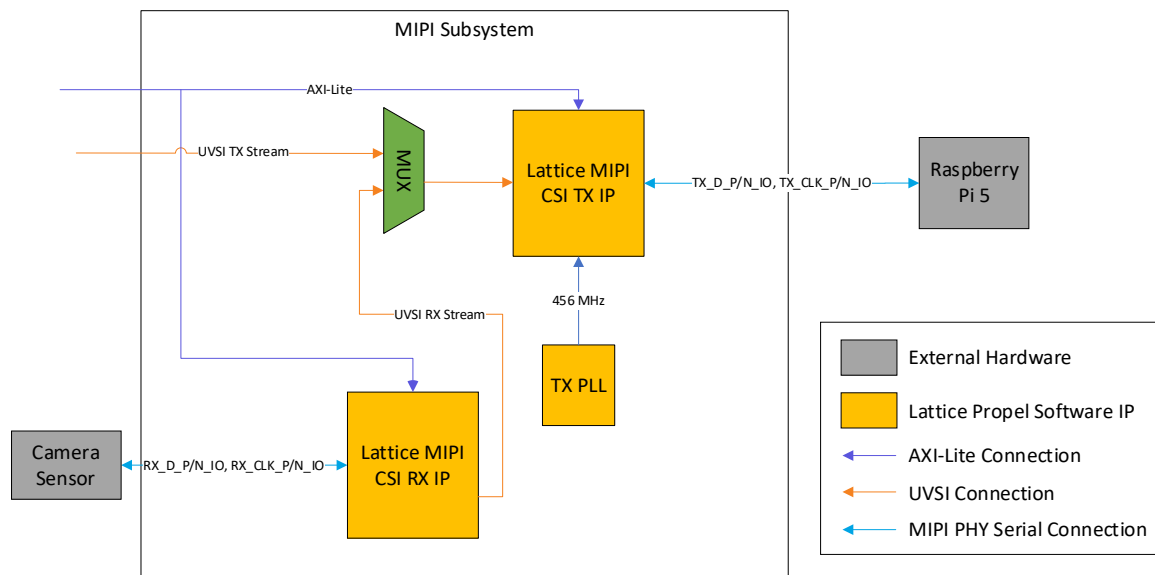


Figure 7.2. MIPI Subsystem Diagram

Lattice MIPI CSI TX IP

The MIPI CSI TX IP is the MIPI CSI IP configured in transmitter mode. For detailed configuration, refer to the [IP Parameter Description](#) and [Signal Description](#) sections. The AXI-Lite interface is connected to a RISC-V MC IP to allow you to read and write IP registers through the RISC-V software provided with the example design. For the UVSI interface input, this IP is selectable using DIP switches on the CertusPro-NX PCIe Vision Board for UVSI to retransmit from the MIPI CSI RX IP or direct video pattern from the pattern subsystem.

Lattice MIPI CSI RX IP

The MIPI CSI RX IP is the MIPI CSI IP configured in receiver mode. For detailed configuration, refer to the [IP Parameter Description](#) and [Signal Description](#) sections. The AXI-Lite interface is connected to a RISC-V MC IP to allow you to read and write IP registers through the RISC-V software provided with the example design.

TX PLL IP

The TX PLL generates 456 MHz clock output for the PHY clock of the MIPI CSI TX IP.

7.3.1.6. I2C Subsystem

The I2C subsystem serves as the I2C communication link between FPGA and external components such as sensor camera and the Raspberry Pi 5 board. The I2C subsystem consists of Lattice IPs such as I2C Controller, I2C Target, and custom components such as I2C bridge, sensor LMMI driver, and I2C APB driver. In this example design, the I2C subsystem initializes the camera sensor via the static ROM configuration setting from FPGA through the I2C communication. In addition, the I2C subsystem bypasses the Raspberry Pi 5 OS initialization to the camera sensor with a dummy ACK response during driver probing.

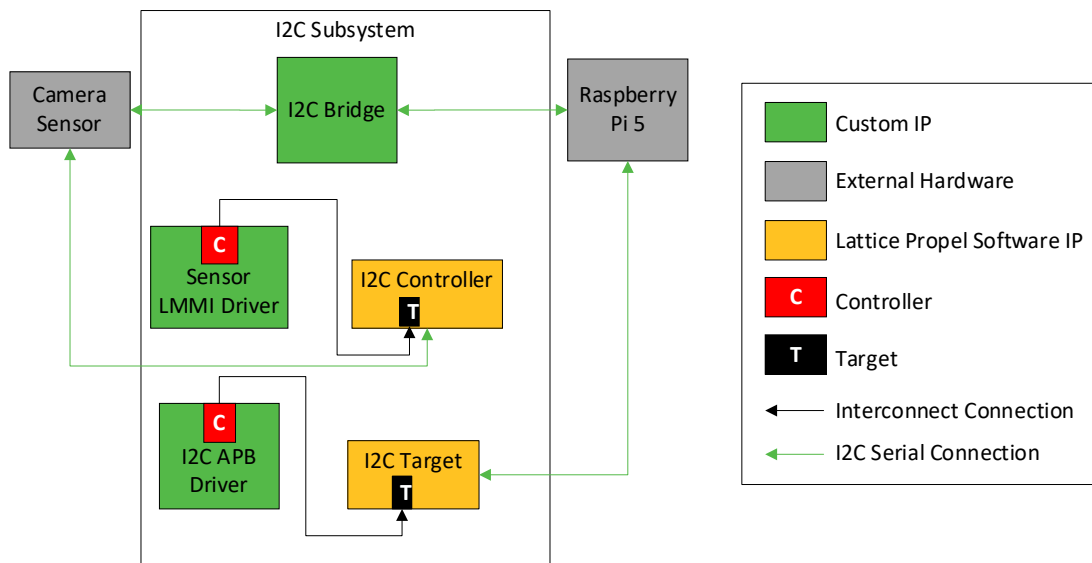


Figure 7.3. I2C Subsystem Diagram

I2C Target IP

The general purpose I2C Target IP provides device addressing, read and write operations, and an acknowledgement mechanism. The programmable nature of the FPGA provides flexibility of configuring the I2C target device to any legal target address, avoiding a potential target address collision on an I2C bus with multiple target devices. In this example design, the I2C Target IP is used for ACK response to the Raspberry Pi 5 OS during the I2C driver probing after power up.

For more information about the IP including register map information, refer to the [I2C Target IP User Guide \(FPGA-IPUG-02072\)](#).

I2C Controller IP

The general purpose I2C Controller IP controls an I2C bus. The programmable nature of the FPGA provides flexibility of configuring the I2C controller device, allowing you to customize the I2C controller to meet your specific design requirements. In this example design, the I2C Controller IP is used together with the sensor LMMI driver to configure the IMX258 sensor camera via I2C communication after power up.

For more information about the IP including register map information, refer to the [I2C Controller IP User Guide \(FPGA-IPUG-02071\)](#).

I2C Bridge

The I2C bridge is used for I2C communication handshake between the FPGA with external components such as sensor camera and the Raspberry Pi 5 OS. The I2C bridge is responsible for propagating any I2C commands through the FPGA

from a target controller (for example, Raspberry Pi 5) to an I2C target (sensor camera) using the SCL clock stretching mechanism which requires longer response time from the I2C target. In this example design, the I2C bridge is disabled as the sensor camera initialization is handled by the sensor LMMI driver and the I2C Controller IP while the ACK response comes from the I2C Target IP.

Sensor LMMI Driver

The sensor LMMI driver configures the I2C Controller IP to initiate the basic I2C transactions to initialize the sensor camera. In this example design, the sensor LMMI driver configures the I2C Controller IP in sequence via the predefined static ROM configuration setting.

I2C APB Driver

The I2C APB driver configures the I2C Target IP with the corresponding data (for example, CHIP_ID) in response to the I2C controller on read transaction. In this example design, the I2C APB driver configures the CHIP_ID value for the ACK response to the Raspberry Pi 5 OS during driver probing.

7.3.1.7. Pattern Subsystem

The pattern subsystem consists of the pattern generator which can generate color bar or checkerboard video pattern with a fixed video resolution of 1920 x 1080 (FHD) to the MIPI CSI TX IP when RX to TX retransmit mode is disabled.

7.3.1.8. RISC-V Subsystem

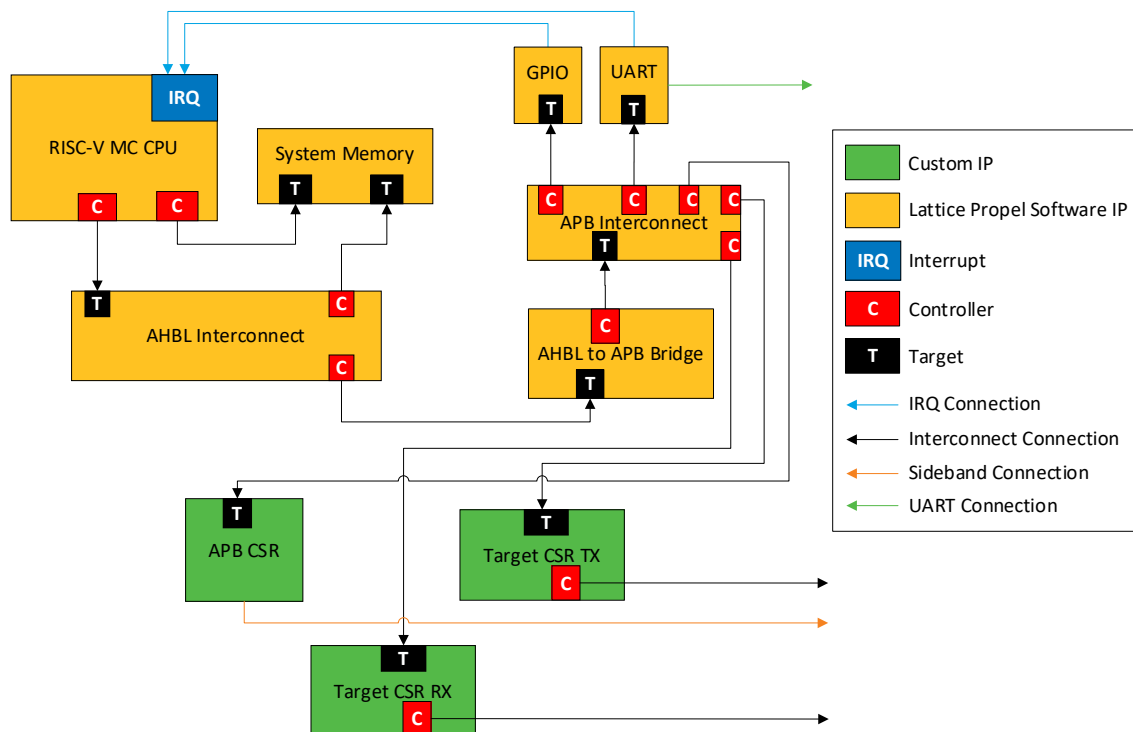


Figure 7.4. RISC-V Subsystem Diagram

RISC-V MC IP

The RISC-V MC CPU IP has advanced high-performance bus (AHB)-Lite based instruction and data ports. The instruction port is connected to the system memory that contains the system application for the central processing unit (CPU) to boot up when the FPGA is programmed. In this example design, the data port is connected to the system memory and other peripherals for control and status read. The CPU also receives user input commands from the demo GUI application via the universal asynchronous receiver/transmitter (UART) serial communication.

For more information about the IP including the register map information, refer to the [RISC-V MC CPU IP User Guide \(FPGA-IPUG-02300\)](#).

GPIO IP

The general purpose input/output (GPIO) Peripheral Soft IP is a simple IP designed to control GPIO via the Lattice memory mapped interface (LMMI) or APB interface. When configured as an input, the IP can detect the state of a GPIO by reading the state of the associated register. When configured as an output, the IP takes the value written into the associated register and controls the state of the controlled GPIO. In this example design, the IP receives input from DIP switches to determine the demo GUI display status.

For more information about the IP including register map information, refer to the [GPIO IP User Guide \(FPGA-IPUG-02076\)](#).

UART IP

The UART Transceiver IP performs serial-to-parallel conversion of data characters received from a peripheral UART device and parallel-to-serial conversion of data characters received from the host located inside the FPGA through an APB interface. In this example design, UART is used to interface with the demo GUI.

For more information about the IP including register map information, refer to the [UART IP User Guide \(FPGA-IPUG-02105\)](#).

Interconnects

The interconnects convert between protocols to match IP interfaces. Interconnects consist of AHB-Lite interconnect, AHB-Lite to APB bridge and APB interconnect.

System Memory

The system memory block stores the MEM initialization file of the software application for the RISC-V MC CPU IP.

APB CSR

The APB CSR is a custom APB target module with pre-defined control registers which is accessible by the RISC-V MC CPU IP. The APB CSR can be used to configure the pattern generator in the pattern subsystem for non-UVSI loopback mode.

Target CSR TX

The Target CSR TX is a custom APB target module that is responsible for the APB to AXI4-Lite conversion. The RISC-V MC CPU IP can perform read or write transactions to the AXI4-Lite interface of the MIPI CSI TX IP through the Target CSR TX module.

Target CSR RX

The Target CSR RX is a custom APB target module that is responsible for the APB to AXI4-Lite conversion. The RISC-V MC CPU IP can perform read or write transactions to the AXI4-Lite interface of the MIPI CSI RX IP through the Target CSR RX module.

7.4. Data Flow

7.4.1. Main Example Design System

When the FPGA device is programmed and the example design system is out of reset, the I2C subsystem initializes and enables streaming from the external camera sensor.

For RX to TX retransmit mode, the sensor image is transmitted from the camera sensor to the MIPI CSI RX IP through the PHY serial interface. The video stream received is decoded before retransmitted to the UVSI interface input of the MIPI CSI TX IP.

For pattern subsystem to TX mode, the video pattern generated from the pattern subsystem is sent to the UVSI interface input of the MIPI CSI TX IP.

The video stream is transmitted out from the MIPI CSI TX IP through the PHY serial interface and displayed in the Raspberry Pi 5 OS.

7.4.2. RISC-V Subsystem

The RISC-V MC CPU streams the application from the system memory and initializes all peripherals in the RISC-V subsystem. After the initialization stage is complete, the RISC-V MC CPU application monitors for any incoming request you send using the demo GUI application.

7.5. Clocking Scheme

Table 7.2. Clocking Scheme in the Example Design

Attribute	Description
clk_125	125 MHz clock from an on-board oscillator to reference clock input of Sys PLL and reference clock input of the MIPI subsystem
ref_clk_out	100 MHz system clock input to the I2C subsystem, and reference clock input to the I2C PLL.
sensor_clk_out	27 MHz clock input for the sensor LMMI driver and I2C controller in the I2C subsystem.
axis_vid_clk_out	150 MHz video clock input to the MIPI subsystem and pattern subsystem.
riscv_clk_out	50 MHz AXI-Lite clock input to the MIPI subsystem and RISC-V subsystem.
phy_ref_clk_out	60 MHz Soft D-PHY reference clock input for MIPI TX and RX IPs.
i2c_bridge_clk_out	8 MHz as the clock input for the I2C bridge component in the I2C subsystem.
pll_clkop	456 MHz PHY clock to the MIPI CSI TX IP in the MIPI subsystem.
pll_clkos	90-degree phase shift 456 MHz PHY clock to the MIPI CSI TX IP in the MIPI subsystem.

7.6. Reset Scheme

Table 7.3. Reset Scheme in the Example Design

Attribute	Description
resetrn_i	Active low asynchronous reset input from the on-board push button. This input resets the entire example design system.
sensor_rst_n_o	Active low asynchronous reset output to the sensor camera.
i2c_rst_n_o	Active low asynchronous reset output to the I2C subsystem.
ref_rst_n_o	Active low asynchronous reset output to the MIPI subsystem.
axis_vid_rst_n_o	Active low asynchronous reset output for the UVSI interface of the MIPI subsystem and pattern subsystem.
sys_pll_lock	PLL locked output from Sys PLL.
i2c_pll_lock	PLL locked output from I2C PLL.
sensor_pll_lock	PLL locked output from sensor PLL.
tx_pll_lock	PLL locked output from TX PLL.

7.6.1. Reset Overview

The figure below shows the reset sequence of this example design system when you power up and program the FPGA device.

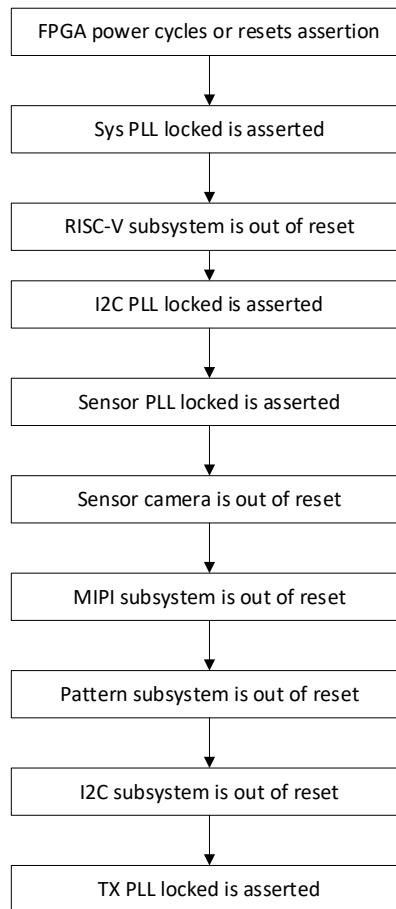


Figure 7.5. Reset Sequence of the Example Design

7.7. Software

The following software is provided in this example design:

- RISC-V bare metal program
- Demo GUI application

7.7.1. RISC-V Bare Metal Program

The bare metal program is built on-top of the RISC-V MC CPU Hello World template that is generated using the Lattice Propel software.

Table 7.4. Main API

API Function Description	API Function Description
int main (void)	When you power cycle the FPGA device or assert the system reset, the main function initializes all peripherals in the RISC-V subsystem. After that, this function waits for the user input commands received from the demo GUI via UART communication.
static int bsp_init(void)	Called within the main function which is responsible for initializing and configuring the interrupt and timer of the RISC-V MC CPU.

Note:

1. The RISC-V bare metal program is provided as reference only, you are not recommended to change the software.

7.8. Signal Description

Table 7.5. Primary I/O Interface Signals for the mipi_interopt_ed_top Module

Port Name	I/O	Width	Description
rx_clk_p_io	In/Out	1	RX serial clock differential positive input port for the MIPI PHY RX IP.
rx_clk_n_io	In/Out	1	RX serial clock differential negative input port for the MIPI PHY RX IP.
rx_d_p_io	In/Out	4	RX serial data differential positive input port for the MIPI PHY RX IP.
rx_d_n_io	In/Out	4	RX serial data differential negative input port for the MIPI PHY RX IP.
tx_clk_p_io	In/Out	1	TX serial clock differential positive input port for the MIPI PHY TX IP.
tx_clk_n_io	In/Out	1	TX serial clock differential negative input for the MIPI PHY TX IP.
tx_d_p_io	In/Out	2	TX serial data differential positive input port for the MIPI PHY TX IP.
tx_d_n_io	In/Out	2	TX serial data differential negative input port for the MIPI PHY TX IP.
m_scl_io	In/Out	1	SCL bidirectional inout port with the external I2C controller.
m_sda_io	In/Out	1	SDA bidirectional inout port with the external I2C controller.
s_scl_io	In/Out	1	SCL bidirectional inout port with the external I2C target.
s_sda_io	In/Out	1	SDA bidirectional inout port with the external I2C target.
uart_rxd_i	In	1	UART RX serial data input port.
uart_txd_o	Out	1	UART TX serial data output port.
dipsw	In	5	DIPSWITCH input port.
status_led	Out	8	LED output port. status_led[0]: sys_pll_lock status_led[1]: i2c_pll_lock status_led[2]: sensor_pll_lock status_led[3]: tx_pll_lock status_led[4]: i2c_rst_n_o status_led[5]: sensor_rst_n_o status_led[6]: ref_rst_n_o status_led[7]: ref_clk_out

7.9. Example Design Hardware Setup

This section describes how to set up and run the example design using the following Lattice software:

- Lattice Radiant software
- Lattice Propel Builder software

7.9.1. Locating the Example Design Files

To locate the example design files, follow these steps:

1. Generate the MIPI CSI IP through the Lattice Radiant software. The example design is in the *<Generated IP component directory>/eval* folder.
2. Unzip the .zip file in your generated IP component directory.

7.9.2. Opening the Lattice Propel Builder Project

The RISC-V subsystem is constructed using the Lattice Propel Builder software, follow these steps to open the RISC-V subsystem in the Lattice Propel Builder software:

1. Invoke the Lattice Propel Builder software.
2. Click **File > Open Design** and from the project database, open the Lattice Propel Builder Sbx file (.sbx) from the *<Generated IP component directory>/eval/mipi_vision_interopt/common/riscv_subsystem* folder.
3. Navigate to **Schematic** or the **Address** tab to view the connections of the RISC-V subsystem and the base address for the corresponding target and controller.

Note: The RISC-V subsystem is pre-generated. You are advised not to regenerate or change the IP configuration settings in the RISC-V subsystem.

7.9.3. Regenerating the IP and Recompiling the Lattice Radiant Software Project

To regenerate the IP and recompile the Lattice Radiant software project, follow these steps:

1. Invoke the Lattice Radiant software.
2. Click **File > Open Project** and from the project database, open the Lattice Radiant software project file (.rdf) from the *<Generated IP component directory>/eval/mipi_vision_interopt/hardware* folder.
3. From the left panel, double-click **mipi_tx_ip.ipx** and **mipi_rx_ip.ipx** to regenerate the MIPI CSI IP from the current Lattice Radiant software project.

Note: All other IP components other than the MIPI CSI IP are pre-generated, you are advised not to regenerate these IP components.

4. Click **Run all** to initiate full compilation.
5. The bitstream file is generated in this path:

<Generated IP component directory>/eval/mipi_vision_interopt/hardware/impl_1/mipi_interopt_ed_impl_1.bit

Note:

The bitstream file is pre-generated in the *<unzipped example design directory>/bitstream* folder, thus, compilation steps are optional.

If you recompile or regenerate IP components, ensure the *Read-only* attribute is unticked from properties of *<unzipped example design directory>* as follows:

1. Right-click on the *<unzipped example design directory>*.
2. Select **Properties**.
3. Locate the **Read-only** option under **Attributes type**.
4. Untick the **Read-only** option if it is enabled.
5. Click **OK**.

7.9.4. Hardware Board Setup

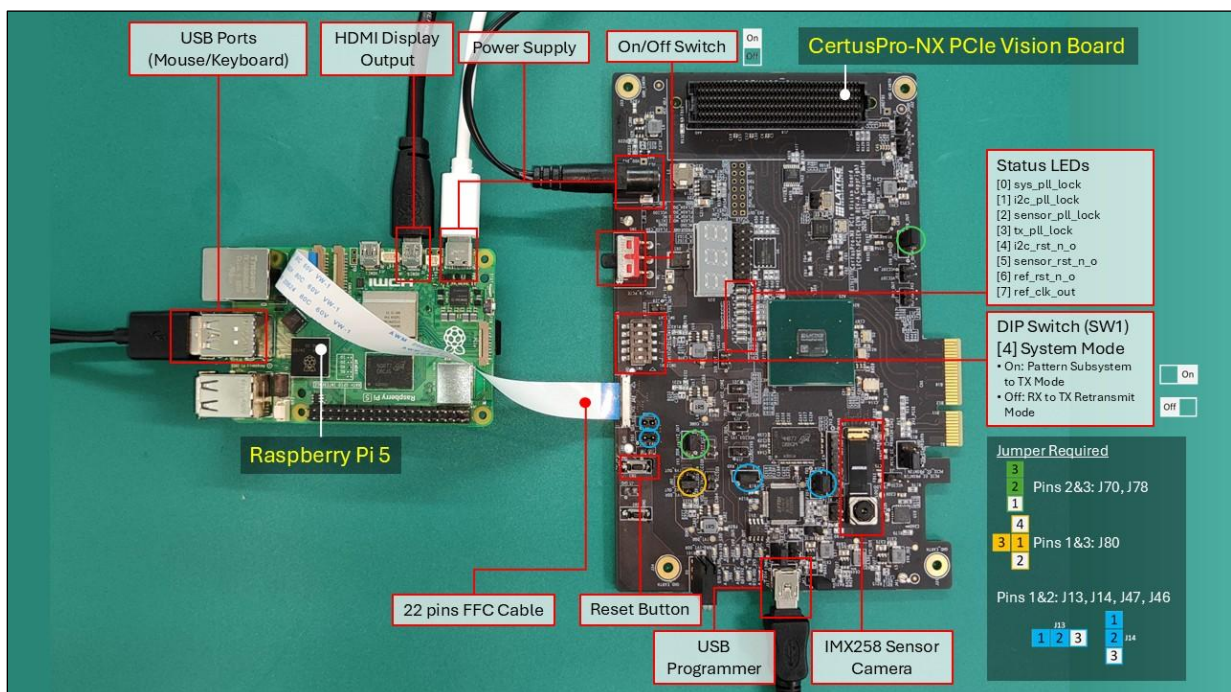


Figure 7.6. Hardware Setup

7.9.4.1. Setting Up the CertusPro-NX PCIe Vision Board

To set up the CertusPro-NX PCIe Vision Board, follow these steps:

1. Connect the 12 V power adaptor to J23.
2. Connect the mini-USB Type A cable from the PC to J17.
3. Connect the IMX258 sensor camera to CN2.
4. Connect the 22-pin FFC/FPC flexible flat cable to J42.
5. Connect the jumpers as follows:
 - a. Pin 2 and 3 at J70
 - b. Pin 2 and 3 at J78
 - c. Pin 1 and 3 at J80
 - d. Pin 1 and 2 at J13
 - e. Pin 1 and 2 at J14
 - f. Pin 1 and 2 at J47
 - g. Pin 1 and 2 at J46
6. Configure the SW1 DIP switch according to the table below.

Table 7.6. SW1 DIP Switch Settings

SW1	Settings		Default
	OFF == 1'b1 (Downward direction toward the CertusPro-NX Device)	ON == 1'b0 (Upward direction toward the edge of the board)	
1	Reserved. Set to ON.		Set to ON.
2			
3			
4	Pattern subsystem to TX mode.	RX to TX retransmit mode.	Set to OFF.
5	Default mode.	GUI mode.	

Note: You must perform system reset with the on-board push button (SW3) whenever you change the SW1 setting.

7.9.4.2. Setting Up the Raspberry Pi 5 Board

To set up the Raspberry Pi 5 board, follow these steps:

1. Connect the power adaptor to J1.
2. Connect the HDMI cable from J13 to the monitor.
3. Connect the FFC/FPC flexible flat cable to CAM/DISP0.

Notes:

- You must program the bitstream into the FPGA device before switching on the Raspberry Pi 5 board.
- You must perform system reset with the on-board push button or reprogram the FPGA board first if you want to restart the Raspberry Pi 5 board.

For first time setup of the Raspberry Pi 5 OS, refer to the [Setting Up the MIPI CSI Display in the Raspberry Pi 5 OS](#) section.

7.9.5. Running the Hardware

To run the example design, follow these steps:

1. Switch off the Raspberry Pi 5 board.
2. Set the SW1 DIP switch according to the default settings in [Table 7.6](#).
3. Open the Lattice Radiant software project. Refer to step 2 in the [Regenerating the IP and Recompiling the Lattice Radiant Software Project](#) section.
4. In the Lattice Radiant software, Click **Tools > Programmer** to invoke the Lattice Radiant Programmer.

- In the **Device Properties** window, click **Browse** to select the pre-generated bitstream `<unzipped example design directory>/bitstream/mipi_interopt_ed_impl_1.bit` for the programming file as shown in the figure below.

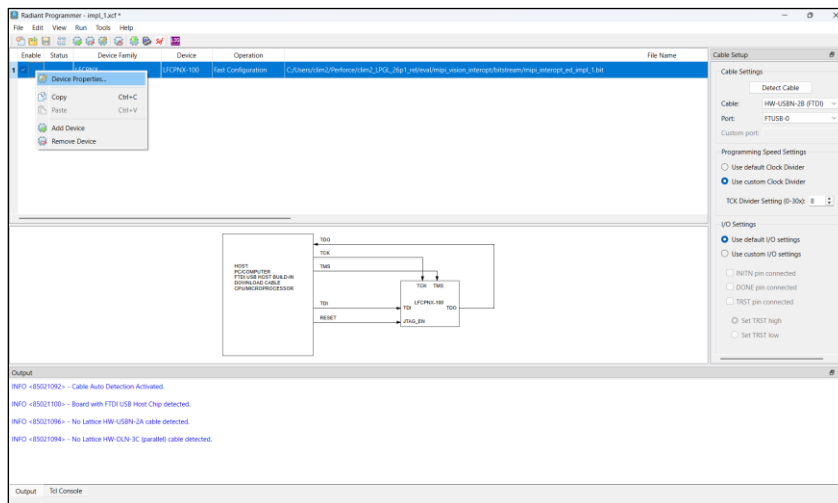


Figure 7.7. Selecting Bitstream in Radiant Programmer

7. Click **OK** to close the **Device Properties** window.
8. Click **Program Device**. The output console displays a successful operation message when the bitstream is programmed successfully.
9. The LED0—LED6 LEDs located on the CertusPro-NX PCIe Vision Board light up and LED07 blinks when the FPGA programming is successful.

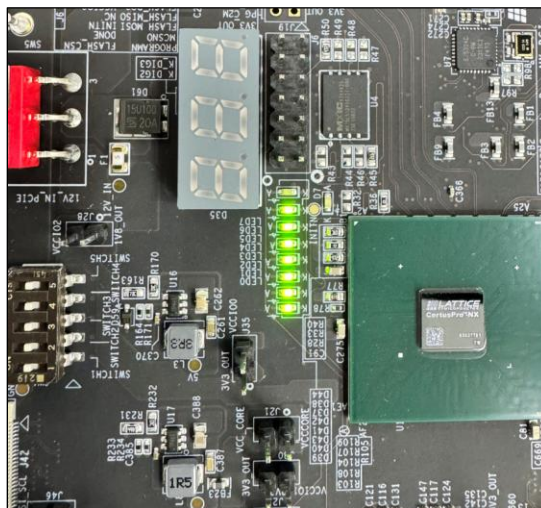


Figure 7.8. LEDs Light Up After Bitstream Programming

Note: If you encounter errors during bitstream programming, follow these steps:

- a. Disconnect the HW-USBN-2B cable from the PC.
 - b. Power cycle the board (switch off and on).
10. Switch on the Raspberry Pi 5 board.
11. Wait for the Raspberry Pi 5 OS to boot up successfully.

12. From Raspberry Pi 5 OS, invoke the command terminal window.

13. At the terminal, enter the following command:

```
rpicam-hello --mode 1920:1080 --framerate 47 --awbgains 1.1,1.1 --timeout <preview
duration in milliseconds>
```

Example:

```
rpicam-hello --mode 1920:1080 --framerate 47 --awbgains 1.1,1.1 --timeout 60000
```

The preview session terminates automatically when the session has reached the timeout value specified in the command.

The framerate option is set to 47 to force the Raspberry Pi 5 camera software to run in the 1920x1080 mode. The actual frame rate is based on the actual video transmission from the FPGA.

For more information on the Raspberry Pi 5 OS camera software, refer to the camera software documentation in the [Raspberry Pi](#) web page.

You must terminate the existing preview session in the Raspberry Pi 5 OS invoked from step 9 before changing the SW1 DIP switch setting and performing system reset. Refer to the [Setting Up the CertusPro-NX PCIe Vision Board](#) section for details on the SW1 DIP switch settings. After you have changed the SW1 setting and performed system reset, invoke a new preview session per step 9.

To terminate the existing preview session, navigate to the command terminal in the Raspberry Pi 5 OS where the current command from step 9 is invoked. On the keyboard connected to the Raspberry Pi 5 board, press the Ctrl+C keys to terminate the current session.

7.9.6. Setting Up the Demo GUI Application

The demo GUI application is supported only for Windows® Platform. For Linux Platform, you must ensure switch 5 from the [Setting Up the CertusPro-NX PCIe Vision Board](#) section is set to the default mode.

For default mode, the pattern subsystem is configured to run only in Color Bar Pattern mode and BGGR CFA format.

For Pattern Subsystem to TX mode, the pattern subsystem begins data transmission automatically after the ED system is out of reset when default mode is selected.

For RX to TX Retransmit mode, the camera sensor begins data transmission automatically after the ED system is out of reset regardless of the switch 5 setting from the SW1 DIP switch.

To set up the demo GUI application, follow these steps:

1. Launch the *video_control_gui.exe* located in the <Generated IP component directory>/eval/mipi_vision_interopt/gui/ folder.
2. Under **UART Configuration**, click **Refresh**.
3. Select **BaudRate: 115200**.
4. Select **ByteSize: 8**.
5. Under **UART Configuration**, click **Connect** to establish the UART connection. The **System Mode** and **Sensor Mode** parameters indicate the current DIP switch settings selected on the FPGA board. The **Pattern Subsystem** section is greyed out if **RX to TX Retransmit Mode** is selected current DIP switch setting.
6. Optionally, select **Logs** to display additional log sections in the GUI application. The **Sent Log** panel displays the command sent out from the GUI when a specific button is clicked. The **Command Log** panel displays the command received and processed in the RISC-V firmware. The **General Log** panel displays additional information in the RISC-V firmware.
7. Click **Read** to retrieve the status information from the MIPI CSI TX IP and MIPI CSI RX IP.

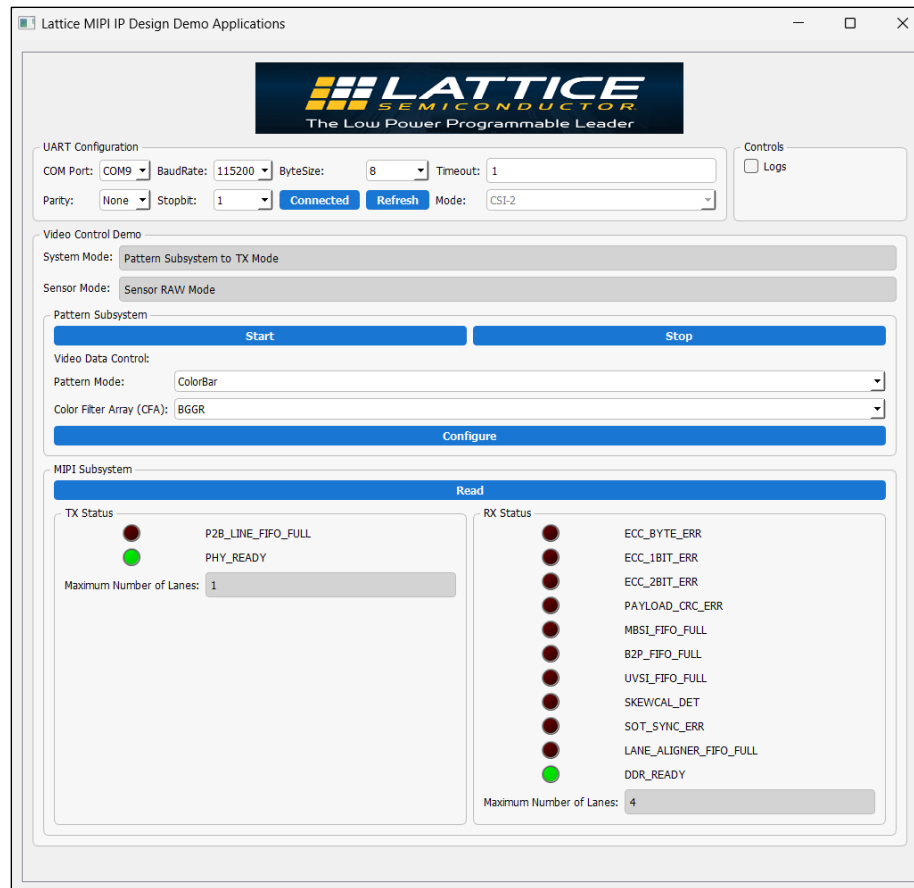


Figure 7.9. Demo GUI Setup

To configure and run the video pattern in the Pattern Subsystem to TX mode, follow these steps:

1. Ensure switch 4 from the [Setting Up the CertusPro-NX PCIe Vision Board](#) section is set to Pattern Subsystem to TX mode.
2. Terminate the existing preview session in the Raspberry Pi 5 OS. The session is invoked from step 9 in the [Running the Hardware](#) section.
3. Under **Video Data Control**, select the **Pattern Mode** and **Color Filter Array (CFA)** settings.
4. Click **Configure** to update the settings.
5. Click **Start** to enable video pattern transmission.
6. Invoke a new preview session in the Raspberry Pi 5 OS per step 9 in the [Running the Hardware](#) section.
7. Click **Stop** to disable video pattern transmission.
8. Repeat step 2 to step 7 if you switch to **Pattern Mode** or **Color Filter Array (CFA)**.

The following diagrams show the example configuration in the demo GUI application and the displayed video patterns for the specific mode selected for the Pattern Subsystem to TX mode or RX to TX Retransmit mode.

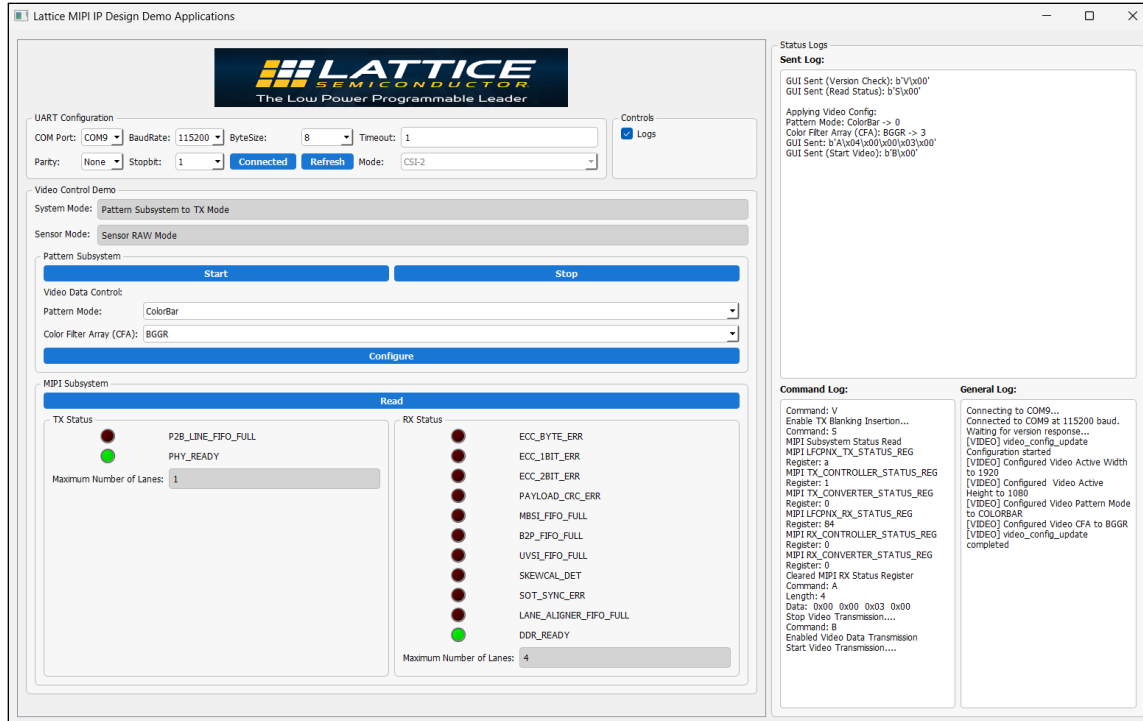


Figure 7.10. Color Bar CFA – BGGR for Pattern Subsystem to TX Mode Configuration Example

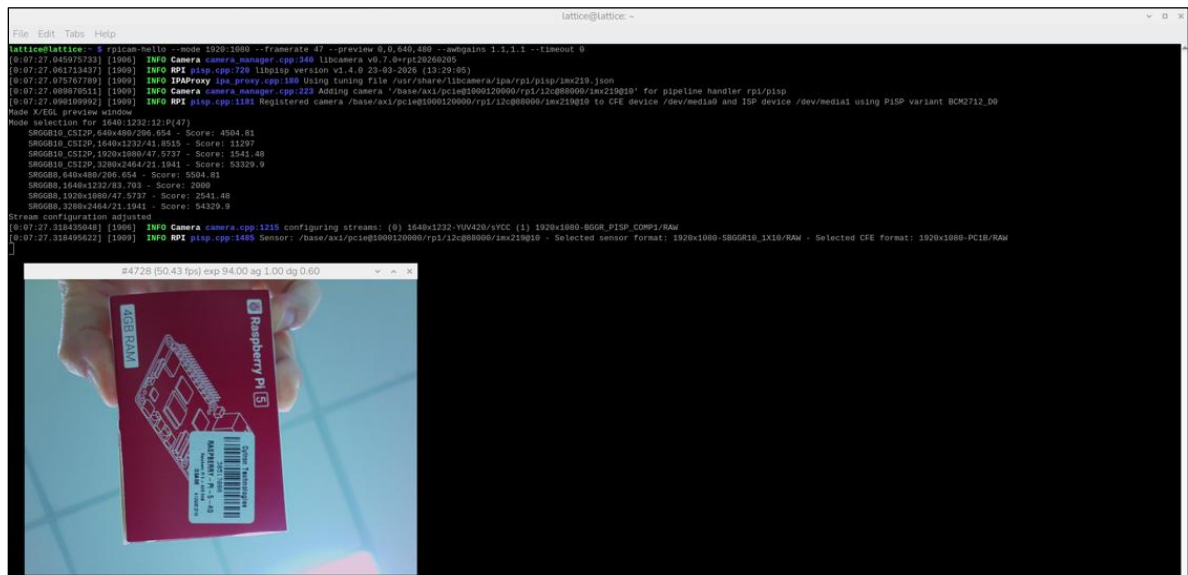


Figure 7.11. Sensor RAW Mode for RX to TX Retransmit Mode

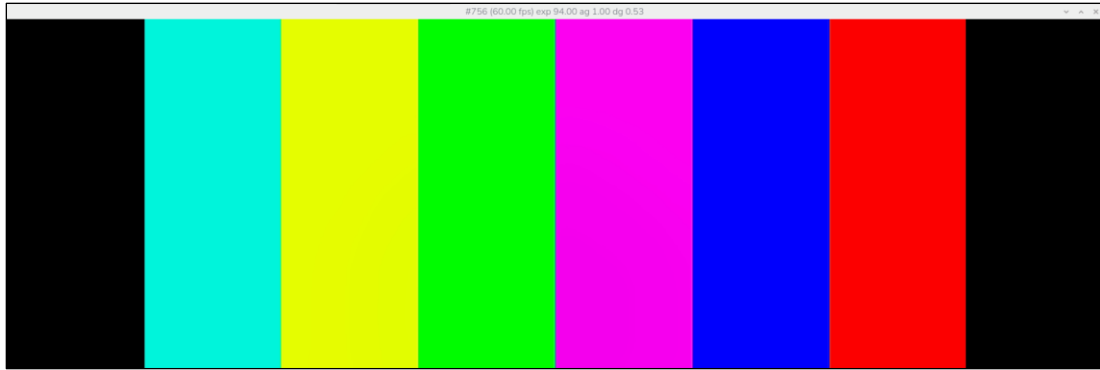


Figure 7.12. Color Bar, CFA – RGGB for Pattern Subsystem to TX Mode



Figure 7.13. Color Bar, CFA – BGGR for Pattern Subsystem to TX Mode

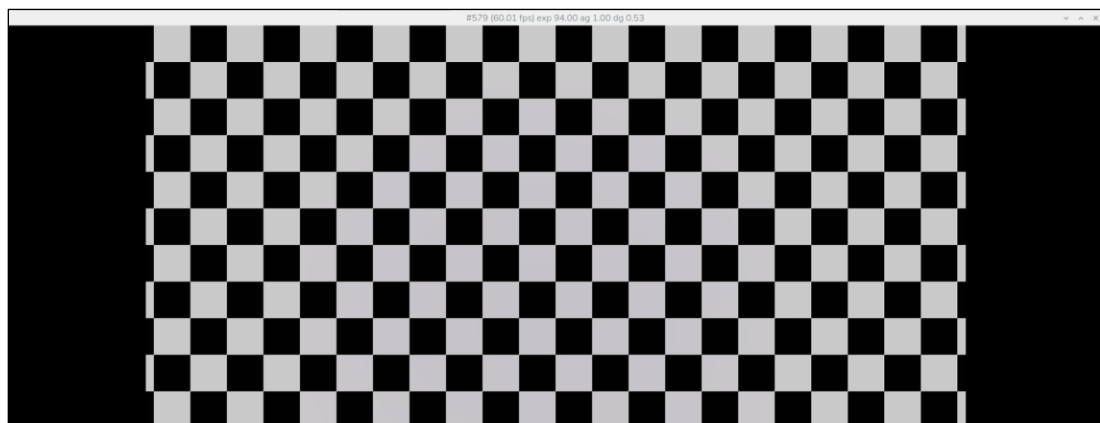


Figure 7.14. Checkerboard for Pattern Subsystem to TX Mode

8. Debugging

This section lists possible issues and suggested troubleshooting steps that you can follow.

8.1. Debug Methods

8.1.1. PHY Clock Status in Transmitter Mode

In transmitter mode, `tinit_done_o` is provided as an output status indicator to help you observe when the PHY has completed the initialization sequence and internal PHY reference clock has synchronized.

This allows you to ensure that the PHY is fully initialized and ready for normal operation before starting any LP or HS transactions. For additional information, refer to [Table 4.5](#).

8.1.2. Generated `byte_clk_o` Does Not Toggle in Receiver Mode

In non-continuous PHY clock mode, `byte_clk_o` toggles only during high-speed data transmission on the MIPI D-PHY lanes. In continuous PHY clock mode, `byte_clk_o` toggles all the time.

If this clock does not toggle as expected:

- Ensure the upstream source is active. For example, camera sensor or application processor.
- Ensure the upstream device configuration matches the IP settings, such as PHY clock mode and bit rate.
- Verify that all input clocks and reset signals to the IP module are correctly set up as recommended.

8.1.3. Data Appears Corrupted in Receiver Mode

If the IP output data appears corrupted when *Unified Video Streaming Interface (UVSI)* is enabled, check whether `axis_vid_tready_i` goes low during active packet transmission. If this signal remains low for more than $(\text{Pixel Buffer Depth}-1)$ `axis_vid_clk_i` cycles, the internal buffer may overflow, leading to functional issues. To prevent overflow, ensure `axis_vid_tready_i` does not stay low beyond the allowed limit. For details, refer to the [MIPI CSI/DSI Receiver UVSI](#) section.

If the IP output data appears corrupted when *MIPI Byte Streaming Interface* is enabled, do the same checking for `axis_byte_tready_i` deassertion with respect to $(\text{Byte Buffer Depth}-1)$ `fr_clk_i` cycles. For details, refer to the [MIPI CSI/DSI Receiver MBSI](#) section.

8.1.4. Missing Packets in Receiver Mode

If there are any missing packets in the IP output data, check whether all input clock frequencies and $t_{\text{HS-PREPARE}} + t_{\text{HS-ZERO}}$ timing requirements specified in the [Limitations for MIPI CSI/DSI Receiver](#) section are met.

8.2. Debug Tools

You can use various tools to debug MIPI CSI/DSI IP design issues.

8.2.1. Reveal Analyzer

The Lattice Reveal™ Analyzer continuously monitors signals within the FPGA for specific conditions that range from simple to complex conditions. When the trigger condition occurs, the Reveal Analyzer saves signal values preceding, during, and following the event for analysis, including a waveform presentation. The data can be saved in the following format:

- Value change dump file (.vcd) that can be used with tools such as QuestaSim®.
- ASCII tabular format that can be used with tools such as Microsoft® Excel.

Before running the Reveal Analyzer, use the Reveal Inserter to add Reveal modules to your design. In these modules, specify the signals to monitor, define the trigger conditions, and set other preferred options. The Reveal Analyzer supports multiple logic analyzer cores using hard/soft JTAG interface. You can have up to 15 modules, typically one for each clock region of interest. When the modules are set up, regenerate the bitstream data file to program the FPGA.

During debug cycles, this tool uses a divide and conquer method to narrow down the problem areas into many small functional blocks to control and monitor the status of each block.

Refer to the [Reveal User Guide for Radiant Software](#) for details on how to use the Reveal Analyzer.

9. Design Considerations

9.1. Limitations for MIPI CSI/DSI Transmitter

- Escape Mode, Ultra Low Power State (ULPS), and Bus Turnaround sequences are not yet supported.
- Minimum word count supported is dependent on the IP setting. Refer to [Table 2.4](#) for details.
- UVSI interlaced format is not supported.
- Trail duration may exceed maximum T_{EOT} because of limitations in design implementation when *Number of TX Lanes* > 1 for the following configurations:
 - *D-PHY TX IP* is Soft D-PHY. For this configuration, consider reducing the trail timing parameter by 1.
 - After reducing the trail timing parameter by 1, timing violations may still be observed at the lowest data rates because of the wider byte clock period, which further constrains the trail timing margin.
- Some IP configurations may have slower Fmax when used in devices with slow performance grade.
- When using the Unified Video Streaming Interface (UVSI) with word count is not a multiple of 32, ensure there is a minimum gap of 4 axis_vid_clk_i clock cycles between successive packet streams.
- When the **Blanking Period Enable (BP_EN)** option is enabled, observe the following timing constraint:
 - $T_{line_uvsi} < T_{line_byte_clock}$
 - T_{line_uvsi} — Line period as measured at the UVSI input interface.
 $T_{line_uvsi} = \text{clock period of axis_vid_clk_i} \times \text{number of pixels per horizontal line} / \text{PPC}$
 - $T_{line_byte_clock}$ — Line period as computed in the byte clock domain.
 $T_{line_byte_clock} = ((\text{NUM_BYTES_ACTIVE_H} / \text{number of lanes}) + \text{H_BLANKING_BYTES_TH}) \times T_{byte_clock}$
 - The UVSI line period must be strictly shorter than the total line budget allocated in the byte clock domain. This ensures that the MIPI TX has sufficient time to transmit both the active pixel payload (NUM_BYTES_ACTIVE_H) and the horizontal blanking period (H_BLANKING_BYTES_TH) within each line interval. If this constraint is not met, the horizontal blanking period may extend beyond the configured threshold because of slower traffic in the byte clock domain, causing the overall line period to exceed the expected value.

9.2. Limitations for MIPI CSI/DSI Receiver

- Escape Mode, Ultra Low Power State (ULPS), and Bus Turnaround sequences are not yet supported.
- Minimum word count supported is dependent on IP setting. Refer to [Table 2.7](#) for details.
- UVSI interlaced format is not supported.
- In DSI mode, MPPT parsing is continuously enabled regardless of the number of packets per HS burst, preventing the IP from distinguishing trailing bytes from valid data. Because of ECC correction, trailing bytes may be misinterpreted as valid headers. EoTP transmission is required for reliable data parsing and ECC reporting. If EoTP is disabled, link integrity must be assessed via ECC error flags.
- ECC error reporting may fail when a received header containing a 2-bit error is incorrectly corrected to either 0xFFFFF or 0x00000, causing the ECC logic to not flag the error correctly.
- When *PHY Mode* == Soft D-PHY, the minimum duration of $t_{HS-PREPARE} + t_{HS-ZERO}$ from D-PHY source must meet the following condition:

$$t_{HS-PREPARE} + t_{HS-ZERO} \geq 145 \text{ ns} + 10 \text{ UI} + (8 \times t_{CLK_SYNC})$$

where t_{CLK_SYNC} is the period of ref_clk_i.

- When *PHY Mode* == Soft D-PHY and the corresponding byte clock is ≤ 60 MHz, you must configure $t_{HS-TRAIL}$ duration closer to but not exceeding t_{EOT} limit as defined by the MIPI D-PHY specification. This configuration ensures reliable data capture and proper end-of-transmission detection under low-frequency conditions.
- The minimum idle duration required between HS transactions is:

$$t_{IDLE} \geq \max(t_{MIPI_REQ}, t_{IP_REQ})$$

where t_{MIPI_REQ} is the MIPI Global Operation Timing Parameter requirement and t_{IP_REQ} is as follows:

- For *PHY Mode* == Soft D-PHY

$$t_{IP_REQ} = t_{HS-EXIT} \rightarrow t_{LPX} \geq (8 \times t_{byte_clk_o})$$

- For *PHY Mode* == Hard D-PHY
 - Continuous Clock mode:

$$t_{IP_REQ} = t_{HS_EXIT} \rightarrow t_{LPX} \geq (16 \times t_{byte_clk_o}) + (3 \times t_{CLK_SYNC})$$
 where t_{CLK_SYNC} is the period of `ref_clk_i`.
 - Non-Continuous Clock mode:

$$t_{IP_REQ} = t_{CLK_POST} \rightarrow t_{LPX} \geq (16 \times t_{byte_clk_o}) + (3 \times t_{CLK_SYNC})$$
 where t_{CLK_POST} is at least 16 byte-clock cycles and t_{CLK_SYNC} is the period of `ref_clk_i`.
- For *PHY Mode* == Soft D-PHY, it is recommended to use the minimum duration of $t_{HS_PREPARE}$ from the transmitter, especially for lower line rates.
- Some IP configurations may have slower Fmax when used in devices with slow performance grade.

9.3. Horizontal and Vertical Blanking Period Insertion Tuning

As the overheads vary with different PHY configuration settings in TX, Lattice recommends simulating and analyzing using the IP simulation to determine the accurate horizontal and vertical blanking period to be inserted at TX to achieve the desired frame rate for specific resolution.

The general formula for Frame Period and Line Period are as follows:

Frame Period = 1/Frame Rate Per Second

Line Period = Line Start Short Packet HS burst duration (Optional) + LP-11 minimum duration + Long Packet HS burst duration + LP-11 minimum duration + Line End Short Packet HS burst duration (Optional) + Horizontal line blanking duration

Where:

- SOT duration is the summation of TLPX, T_HS-PREPARE, and T_HS-ZERO time which is required for LP to HS transition.
- EOT duration is the T_HS_TRAIL time which is required for HS to LP transition.
- LP-11 minimum duration is the minimum LP-11 state duration imposed in TX from Line Start Short Packet HS burst to Long Packet HS burst or from Long Packet HS burst to Line End Short Packet HS burst.
- The Line Start and Line End Short Packet HS bursts are optional packets depending on the TX configuration.

Example Calculation for Horizontal and Vertical Blanking Period

- Video Stream Resolution: 1920 × 1080
- Pixel format: RAW10
- Pixels Per Clock: 4
- Pixel Clock: 150 MHz
- Line Start and Line End: Enabled
- PHY Mode: Soft D-PHY
- PHY Clock Mode: Non-Continuous
- Number of lanes: 2
- Lane rate: 912 Mbps
- Byte Clock (Lane rate/PHY Data Width): 114 MHz

For frame rate of 60 per second, the theoretical frame period required is as follows:

Frame Period = 1/Frame Rate Per Second
= 16.666 ms

For Horizontal Line Blanking Insertion Only

With the vertical frame blanking period at a minimal value determined by the video FSM, adjust the LP-11 state duration between consecutive Line End and the next Line Start short packet to increase or decrease the Line Period.

The theoretical calculations are as follows:

Line Period (Per Lane) = Frame Period/Total Lines
= 16.666 ms/1080
= 15.43148 μ s

Total Horizontal Bytes = Line Period (Per Lane)/Byte Clock Period
= 15.43148 μ s/8.77193 ns
= 1759 bytes

Horizontal Blanking Bytes (Per Lane) = Total Horizontal Bytes – Active Horizontal Bytes
= 1759 – 1200
= 559 bytes

The horizontal blanking bytes calculated in the example is the theoretical value however the actual additional horizontal blanking line duration required is much less because of the overheads within a single active line transmission. The overheads are as follows:

- HS burst for short packets (Line Start and Line End)
- 4 Bytes of packet header for each short and long packet
- 2 Bytes of CRC for each long packet
- HS to LP state transition (SoT sequence overhead for each HS burst, EoT sequence overhead for each HS burst, and minimum duration in LP-11 state between each HS burst)
- TX data path logic delays

For details on the timing duration imposed during each HS to LP state transition, refer to the [MIPI CSI/DSI TX Protocol Timing Parameters](#) section based on the current configuration in TX.

To determine the actual horizontal blanking line duration required, configure the values as follows:

- **Number of Vertical Blanking Lines Per Frame** (IP Wizard GUI) or NUM_LINE_V_BLANKING field (CONFIG_REG1)—Set to 0.
- **Number of Horizontal Active Bytes in Byte Clock Domain** (IP Wizard GUI) or NUM_BYTES_ACTIVE_H field (CONFIG_REG2)—Set to 1200 bytes.
- **Number of Horizontal Blanking Bytes in Byte Clock Domain** (IP Wizard GUI) or H_BLANKING_BYTES_TH field (CONFIG_REG1)—Set to the theoretical value in the example calculation.

With the initial configuration settings in the example, trigger the IP simulation and analyze the horizontal line blanking duration in the LP-11 state between the Line End HS burst and next Line Start HS burst. For shorter simulation time, run the IP simulation with a small number of lines during initial blanking period calibration.

The following IP simulation waveform diagram shows the minimum vertical frame blanking period with LP-11 state between Frame End and the next Frame Start short packet.

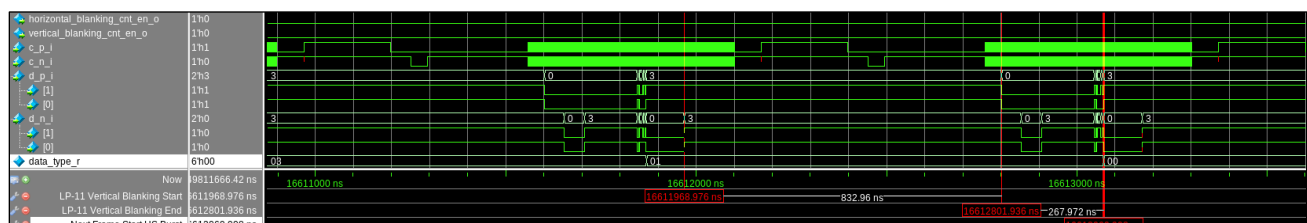


Figure 9.1. Minimum Vertical Frame Blanking between Frame End and the Next Frame Start HS Burst Short Packet (Number of Vertical Blanking Lines Per Frame = 0)

The following IP simulation waveform diagram shows the horizontal line blanking period of LP-11 state between Line End and the next Line Start short packet.

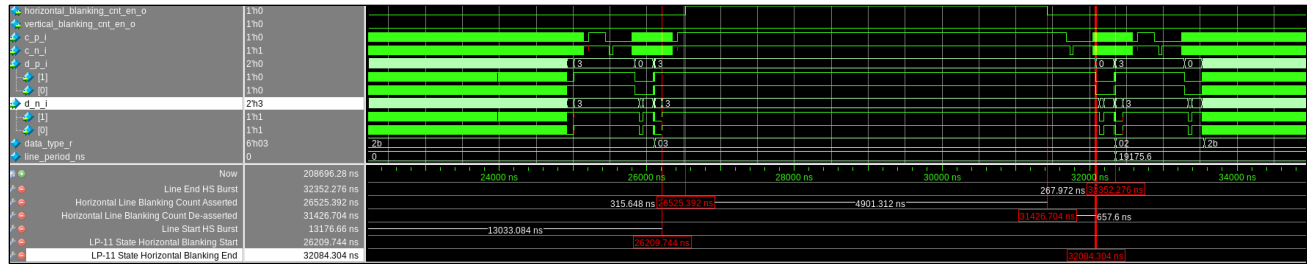


Figure 9.2. Horizontal Line Blanking between Line End and the Next Line Start HS Burst Short Packet (Number of Horizontal Blanking Bytes in Byte Clock Domain = 559)

From the IP simulation waveforms, the additional duration between the LP-11 state assertion to the horizontal_blanking_cnt_en_o assertion and the horizontal_blanking_cnt_en_o de-assertion to LP-11 state de-assertion are the overheads of data path logic delays and minimum LP-11 state duration imposed by TX. The LP-11 state for horizontal blanking begins when the d_p_io and d_n_io signals are asserted and ends when either the d_p_io or d_n_io signal is de-asserted. The data_type_r and line_period_ns signals are status variable from RX simulation model in the testbench.

$$\begin{aligned} \text{Additional Overheads} &= 315.548 \text{ ns} + 657.6 \text{ ns} \\ &= 973.148 \text{ ns} \end{aligned}$$

$$\begin{aligned} \text{Line Start Short Packet HS Burst/Line End Short Packet HS Burst Duration} &= \sim 245 \text{ ns (SOT)} + \sim 101 \text{ ns (EOT)} + (2 \text{ bytes} \times \\ &\text{Byte Clock Period}) \\ &= \sim 363.5438 \text{ ns} \end{aligned}$$

$$\begin{aligned} \text{Long Packet HS Burst Duration} &= ((1200 \text{ (Data)} + 2 \text{ (Packet Header)} + 1 \text{ (CRC)}) \times \text{Byte Clock Period}) + 245 \text{ ns (SOT)} + \\ &101 \text{ ns (EOT)} \\ &= \sim 10898.6318 \text{ ns} \end{aligned}$$

$$\text{LP-11 Minimum Duration} = \sim 831 \text{ ns (from simulation)}$$

$$\text{Vertical Frame Blanking Duration} = \text{Same as LP-11 minimum duration}$$

$$\text{Horizontal Line Blanking Duration} = \sim 5874.56 \text{ ns (from simulation)}$$

$$\text{Line Period} = (2 \times 363.5438 \text{ ns}) + (2 \times 831 \text{ ns}) + 10898.6317 \text{ ns} + 5874.56 \text{ ns} = 19.1622 \mu\text{s}$$

The actual required Horizontal Line Blanking Duration can be calculated as follows:

$$\begin{aligned} \text{Line Period Without Additional Horizontal Line Blanking Duration} &= 19.1622 \mu\text{s} - 5.8745 \mu\text{s} + 0.9731 \mu\text{s (Overhead)} \\ &= 14.2608 \mu\text{s} \end{aligned}$$

$$\begin{aligned} \text{Required Horizontal Line Blanking Duration} &= \text{Expected Line Period} - \text{Line Period Without Additional Horizontal Line} \\ &\text{Blanking Duration} \\ &= 15.4314 \mu\text{s} - 14.2608 \mu\text{s} \\ &= \sim 1170.60 \text{ ns} \end{aligned}$$

Number of Horizontal Blanking Bytes in Byte Clock Domain = Required Horizontal Line Blanking Duration/Byte Clock Period

$$= 1170.58 \text{ ns} / 8.77193 \text{ ns}$$

$$= \sim 133 \text{ Bytes}$$

The following IP simulation waveform diagram shows the horizontal line blanking period of LP-11 state between Line End and the next Line Start short packet with the latest Number of Horizontal Blanking Bytes in Byte Clock Domain value.

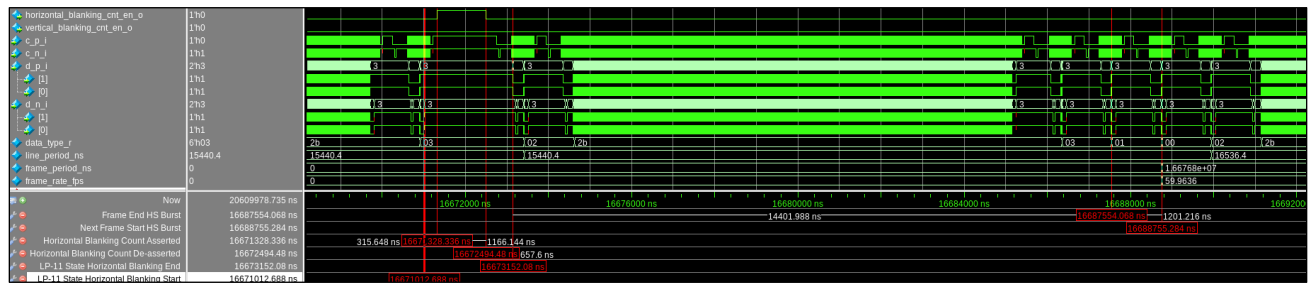


Figure 9.3. Horizontal Line Blanking between Line End and the Next Line Start HS Burst Short Packet (Number of Horizontal Blanking Bytes in Byte Clock Domain = 133)

The frame_period_ns and frame_rate_fps signals are status variable from the RX simulation model in the testbench which indicate the measured frame rate of ~59.96.

For Vertical Frame Blanking Insertion Only

With the horizontal line blanking period at a minimal value determined by the video FSM, adjust the LP-11 state between consecutive Frame End the next Frame Start short packet to increase or decrease the Frame Period. In this case, the value configured in **Number of Horizontal Active Bytes in Byte Clock Domain** is **Actual Active Bytes** plus additional dummy bytes which is used by TX to insert only vertical frame blanking line duration. The **Number of Horizontal Blanking Bytes in Byte Clock Domain** parameter is set to zero. This does not impact the actual pixel data packet received by TX from the user interface.

Line Start Short Packet HS Burst/Line End Short Packet HS Burst Duration = ~245 ns (SOT) + ~101 ns (EOT) + (2 x Byte Clock Period)

$$= \sim 363.5438 \text{ ns}$$

$$\begin{aligned} \text{Long Packet HS Burst Duration} &= ((1200 (\text{Data}) + 2 (\text{Packet Header}) + 1 (\text{CRC})) \times \text{Byte Clock Period}) + 245 \text{ ns (SOT)} + 101 \text{ ns (EOT)} \\ &= \sim 10898.6317 \text{ ns} \end{aligned}$$

$$\text{LP-11 Minimum Duration} = \sim 831 \text{ ns (from simulation)}$$

$$\text{Horizontal Line Blanking Duration} = \text{Same as LP-11 Minimum Duration}$$

$$\text{Derived Line Period} = (2 \times 363.5438 \text{ ns}) + (3 \times 831 \text{ ns}) + 10898.6317 \text{ ns} = 14118.7193 \text{ ns}$$

$$\text{Frame Period} = (\text{Active Lines} \times \text{Line Period}) + \text{Vertical Frame Blanking Duration}$$

$$\begin{aligned}\text{Vertical Frame Blanking Duration} &= 16666 \mu\text{s} - (1080 \times 14.1187 \mu\text{s}) \\ &= 1417.804 \mu\text{s}\end{aligned}$$

$$\begin{aligned}\text{Number of Vertical Blanking Lines Per Frame} &= \text{Vertical Frame Blanking Duration} / (\text{Number of Horizontal Active Bytes in} \\ &\text{Byte Clock Domain} \times \text{Byte Clock Period}) \\ &= 1417.804 \mu\text{s} / (1200 \times 8.77193 \text{ ns}) \\ &= \sim 134 \text{ lines}\end{aligned}$$

To determine the actual vertical frame blanking duration required, configure the values as follows:

- **Number of Vertical Blanking Lines Per Frame** (IP Wizard GUI) or NUM_LINE_V_BLANKING field (CONFIG_REG1)—Set to the calculated value in the example
- **Number of Horizontal Active Bytes in Byte Clock Domain** (IP Wizard GUI) or NUM_BYTES_ACTIVE_H field (CONFIG_REG2)—Set to 1200
- **Number of Horizontal Blanking Bytes in Byte Clock Domain** (IP Wizard GUI) or H_BLANKING_BYTES_TH field (CONFIG_REG1)—Set to 0

With the initial configuration settings in the example, trigger the IP simulation and analyze the vertical frame blanking duration in the LP-11 state between the Frame End HS burst and next Frame Start HS burst.

The following IP simulation waveform diagram shows the minimum horizontal line blanking period with LP-11 state between Line End and the next Line Start short packet.

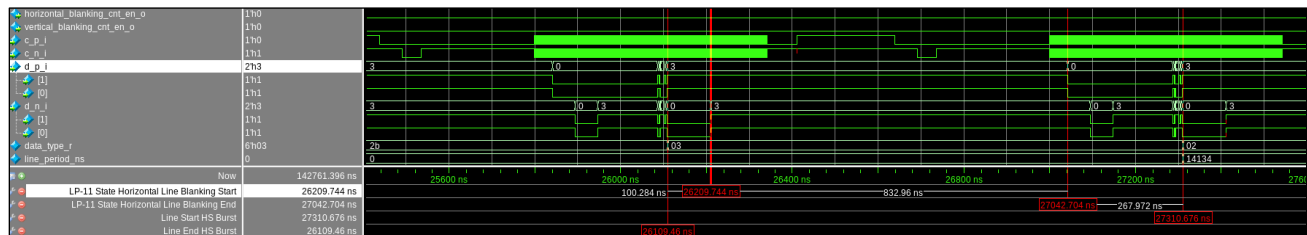


Figure 9.4. Minimum Horizontal Line Blanking between Line End and the Next Line Start HS Burst Short Packet (Number of Horizontal Blanking Bytes in Byte Clock Domain = 0)

$$\text{Measured Line Period} = 14134 \text{ ns (from simulation)}$$

$$\text{Expected Frame Period} = (\text{Active Lines} \times \text{Line Period}) + \text{Vertical Frame Blanking Duration}$$

$$\begin{aligned}\text{Vertical Frame Blanking Duration} &= 16666 \mu\text{s} - (1080 \times 14134 \text{ ns}) \\ &= 1401.28 \mu\text{s}\end{aligned}$$

$$\begin{aligned}\text{Number of Vertical Blanking Lines Per Frame} &= \text{Vertical Frame Blanking Duration} / (\text{Number of Horizontal Active Bytes in} \\ &\text{Byte Clock Domain} \times \text{Byte Clock Period}) \\ &= 1401.28 \mu\text{s} / (1200 \times 8.77193 \text{ ns}) \\ &= \sim 133 \text{ lines}\end{aligned}$$

The following IP simulation waveform diagram shows the vertical frame blanking period of LP-11 state between Frame End and the next Frame Start short packet with the latest Number of Vertical Blanking Lines Per Frame value.

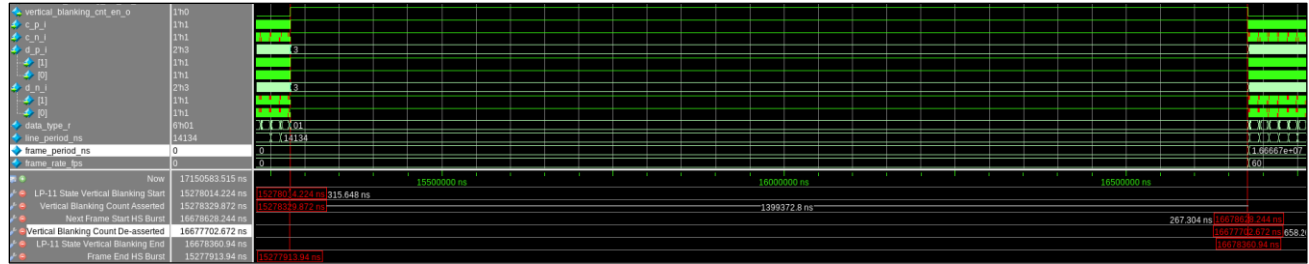


Figure 9.5. Vertical Frame Blanking between Frame End and the Next Frame Start HS Burst Short Packet (Number of Vertical Blanking Lines Per Frame = 133)

The LP-11 state for vertical blanking begins when the d_p_io and d_n_io signals are asserted and ends when either the d_p_io or d_n_io signal de-asserted.

The frame_period_ns and frame_rate_fps signals are status variable from the RX simulation model in the testbench which indicate the measured frame rate of ~60.

Appendix A. Resource Utilization

Note: Resource utilization values in this section are provided for reference only and may change based on the compilation strategy and selected tool options.

The tables below show the sample resource utilization of the MIPI CSI/DSI IP on FPGA devices using the Synplify Pro synthesis tool.

Table A.1. MIPI CSI/DSI Transmitter Resource Utilization for the LFCPNX-100-9LFG672C Device¹

IP Configuration	Slice	LUTs	Registers	EBR	High-Speed I/O Resources
<i>PHY Direction == TX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1500</i> <i>PHY Clock Mode == Continuous</i> <i>Streaming Interface == UVSI</i> <i>Line Buffer Depth == 512</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RGB888 Color Format == checked</i>	2335/79872	1769/79872	2335/80769	16/208	5 x ODDR4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == TX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 2</i> <i>Line Rate Per Lane == 1500</i> <i>PHY Clock Mode == Continuous</i> <i>Streaming Interface == UVSI</i> <i>Line Buffer Depth == 512</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RGB888 Color Format == checked</i>	2288/79872	1747/79872	2288/80769	16/208	3 x ODDR4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == TX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1500</i> <i>PHY Clock Mode == Continuous</i> <i>Streaming Interface == UVSI</i> <i>Line Buffer Depth == 512</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RGB565 Color Format == checked</i>	1677/79872	1071/79872	1677/80769	16/208	5 x ODDR4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == TX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1500</i> <i>PHY Clock Mode == Non-Continuous</i> <i>Streaming Interface == UVSI</i> <i>Line Buffer Depth == 2048</i> <i>Number of Pixel per Clock (PPC) == 1</i> <i>Enable RGB888 Color Format == checked</i>	2361/79872	1751/79872	2361/80769	64/208	5 x ODDR4, 1 x ECLKDIV, 1 x ECLKSYNC

IP Configuration	Slice	LUTs	Registers	EBR	High-Speed I/O Resources
<i>PHY Direction == TX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1500</i> <i>PHY Clock Mode == Continuous</i> <i>Streaming Interface == UVSI</i> <i>Line Buffer Depth == 512</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RAW10 Color Format == checked</i>	2760/79872	2760/79872	2760/80769	16/208	5 x ODDR4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == TX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1500</i> <i>PHY Clock Mode == Non-Continuous</i> <i>Protocol Mode == CSI-2</i> <i>Streaming Interface == MBSI</i> <i>ECC Insertion == checked</i> <i>CRC Insertion == checked</i> <i>Byte Buffer Depth == 4096</i> <i>Enable AXI4-Lite Interface == checked</i> <i>Dynamic Reconfiguration == checked</i> <i>Blanking Enable == checked</i>	1191/79872	1851/79872	1191/80769	8/208	5 x ODDR4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == TX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1500</i> <i>PHY Clock Mode == Continuous</i> <i>Streaming Interface == UVSI</i> <i>Line Buffer Depth == 512</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RAW12 Color Format == checked</i> <i>Enable AXI4-Lite Interface == checked</i> <i>Dynamic Reconfiguration == checked</i> <i>Blanking Enable == checked</i>	2546/79872	2744/79872	2546/80769	16/208	5 x ODDR4, 1 x ECLKDIV, 1 x ECLKSYNC

Note:

1. All other settings are default.

Table A.2. MIPI CSI/DSI Receiver Resource Utilization for the LFCPNX-100-9LFG672C Device¹

IP Configuration	Slices	LUTs	Registers	EBR	High-Speed I/O Resources
<i>PHY Direction == RX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1000</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == DSI-2</i> <i>Streaming Interface == UVSI</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RGB888 Color Format == checked</i>	1597/79872	1871/79872	1597/80769	8/208	4 x IDDR4, 1 x ECLKDIV, 1 x ECLKSYNC

IP Configuration	Slices	LUTs	Registers	EBR	High-Speed I/O Resources
<i>PHY Direction == RX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 2</i> <i>Line Rate Per Lane == 1000</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == DSI-2</i> <i>Streaming Interface == UVSI</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RGB888 Color Format == checked</i>	1386/79872	1464/79872	1386/80769	6/208	2 x IDDRX4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == RX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1000</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == CSI-2</i> <i>Streaming Interface == UVSI</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RGB888 Color Format == checked</i>	1600/79872	1804/79872	1600/80769	8/208	4 x IDDRX4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == RX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1000</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == CSI-2</i> <i>Streaming Interface == UVSI</i> <i>Number of Pixel per Clock (PPC) == 1</i> <i>Enable RGB888 Color Format == checked</i>	1345/79872	1840/79872	1345/80769	6/208	4 x IDDRX4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == RX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1000</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == CSI-2</i> <i>Streaming Interface == UVSI</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RAW10 Color Format == checked</i>	1395/79872	1808/79872	1395/80769	7/208	4 x IDDRX4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == RX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1000</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == DSI-2</i> <i>Streaming Interface == MBSI</i>	1193/79872	1646/79872	1193/80769	6/208	4 x IDDRX4, 1 x ECLKDIV, 1 x ECLKSYNC

IP Configuration	Slices	LUTs	Registers	EBR	High-Speed I/O Resources
<i>PHY Direction == RX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1000</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == CSI-2</i> <i>Streaming Interface == MBSI</i>	1096/79872	1410/79872	1096/80769	6/208	4 x IDDRX4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == RX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1000</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == DSI-2</i> <i>Streaming Interface == UVSI</i> <i>Enable AXI4-Lite Interface == checked</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RGB888 Color Format == checked</i>	1867/79872	2348/79872	1867/80769	8/208	4 x IDDRX4, 1 x ECLKDIV, 1 x ECLKSYNC
<i>PHY Direction == RX</i> <i>PHY Mode == Soft D-PHY</i> <i>PHY Data Width == 8</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 1000</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == CSI-2</i> <i>Enable AXI4-Lite Interface == checked</i> <i>Dynamic Reconfiguration == checked</i> <i>Streaming Interface == MBSI</i>	1322/79872	1897/79872	1322/80769	6/208	4 x IDDRX4, 1 x ECLKDIV, 1 x ECLKSYNC

Note:

1. All other settings are default.

Table A.3. MIPI CSI/DSI Transmitter Resource Utilization for the LIFCL-40-CABGA400 Device¹

IP Configuration	Slices	LUTs	Registers	EBR	High-Speed I/O Resources
<i>PHY Direction == TX</i> <i>PHY Mode == Hard D-PHY</i> <i>PHY Data Width == 16</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 2500</i> <i>PHY Clock Mode == Continuous</i> <i>Streaming Interface == UVSI</i> <i>Line Buffer Depth == 512</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RGB888 Color Format == checked</i>	2659/32811	2443/32256	2659/32811	16/84	1 D-PHY
<i>PHY Direction == TX</i> <i>PHY Mode == Hard D-PHY</i> <i>PHY Data Width == 16</i> <i>Number of D-PHY Lanes == 2</i> <i>Line Rate Per Lane == 2500</i> <i>PHY Clock Mode == Continuous</i> <i>Streaming Interface == UVSI</i> <i>Line Buffer Depth == 512</i> <i>Number of Pixel per Clock (PPC) == 4</i>	2533/32811	2297/32256	2533/32811	16/84	1 D-PHY

IP Configuration	Slices	LUTs	Registers	EBR	High-Speed I/O Resources
<i>Enable RGB565 Color Format == checked</i> <i>Enable RAW12 Color Format == checked</i> <i>Enable AXI4-Lite Interface == checked</i> <i>Dynamic Reconfiguration == checked</i>					
<i>PHY Direction == TX</i> <i>PHY Mode == Hard D-PHY</i> <i>PHY Data Width == 16</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 2500</i> <i>PHY Clock Mode == Continuous</i> <i>Streaming Interface == UVS</i> <i>Line Buffer Depth == 512</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RGB888 Color Format == checked</i> <i>Enable AXI4-Lite Interface == checked</i> <i>Dynamic Reconfiguration == checked</i> <i>Blanking Enable == checked</i>	3215/32811	3330/32256	32811/32811	16/84	1 D-PHY
<i>PHY Direction == TX</i> <i>PHY Mode == Hard D-PHY</i> <i>PHY Data Width == 16</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 2500</i> <i>PHY Clock Mode == Continuous</i> <i>Streaming Interface == UVS</i> <i>Line Buffer Depth == 2048</i> <i>Number of Pixel per Clock (PPC) == 1</i> <i>Enable RAW10 Color Format == checked</i> <i>Enable AXI4-Lite Interface == checked</i> <i>Dynamic Reconfiguration == checked</i>	2790/32811	3288/32256	2790/32811	64/84	1 D-PHY
<i>PHY Direction == TX</i> <i>PHY Mode == Hard D-PHY</i> <i>PHY Data Width == 16</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 2500</i> <i>PHY Clock Mode == Continuous</i> <i>Streaming Interface == MBS</i> <i>Line Buffer Depth == 4096</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable AXI4-Lite Interface == checked</i> <i>Dynamic Reconfiguration == checked</i>	1755/32256	3101/79872	1755/32811	16/84	1 D-PHY

Note:

1. All other settings are default.

Table A.4. MIPI CSI/DSI Receiver Resource Utilization for the LIFCL-40-CABGA400 Device¹

IP Configuration	Slices	LUTs	Registers	EBR	High-Speed I/O Resources
<i>PHY Direction == RX</i> <i>PHY Mode == Hard D-PHY</i> <i>PHY Data Width == 16</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 2500</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == DSI-2</i> <i>Streaming Interface == UVS</i> <i>Number of Pixel per Clock (PPC) == 4</i>	2241/32256	2144/32256	2241/32811	9/84	1 D-PHY

IP Configuration	Slices	LUTs	Registers	EBR	High-Speed I/O Resources
<i>Enable RGB888 Color Format == checked</i>					
<i>PHY Direction == RX</i> <i>PHY Mode == Hard D-PHY</i> <i>PHY Data Width == 16</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 2500</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == CSI-2</i> <i>Streaming Interface == MBSI</i>	1652/32256	882/32256	1652/32811	9/84	1 D-PHY
<i>PHY Direction == RX</i> <i>PHY Mode == Hard D-PHY</i> <i>PHY Data Width == 16</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 2500</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == DSI-2</i> <i>Enable AXI4-Lite Interface == checked</i> <i>Streaming Interface == UVSI</i> <i>Number of Pixel per Clock (PPC) == 4</i> <i>Enable RGB888 Color Format == checked</i>	2768/32256	3180/32256	2768/32811	6/84	1 D-PHY
<i>PHY Direction == RX</i> <i>PHY Mode == Hard D-PHY</i> <i>PHY Data Width == 16</i> <i>Number of D-PHY Lanes == 4</i> <i>Line Rate Per Lane == 2500</i> <i>PHY Clock Mode == Continuous</i> <i>Protocol Mode == CSI-2</i> <i>Enable AXI4-Lite Interface == checked</i> <i>Dynamic Reconfiguration == checked</i> <i>Streaming Interface == MBSI</i>	1697/32256	1590/32256	1697/32811	4/84	1 D-PHY

Note:

1. All other settings are default.

For more information about specific configuration, generate the IP, run synthesis and MAP, and check the MAP reports for resource utilization. Numbers may vary when using a different software version, device density, synthesis tool, or performance grade.

Appendix B. Raspberry Pi OS Setup

Raspberry Pi 5 OS Software Setup

This example design uses the Raspberry Pi OS (64-bit) OS.

For more information on setting up the Raspberry Pi 5 OS, refer to the Raspberry Pi Imager software in the [Raspberry Pi](#) web page.

Setting Up the MIPI CSI Display in the Raspberry Pi 5 OS

For more information on desktop setup for the Raspberry Pi 5 OS, refer to the Getting Started documentation in the [Raspberry Pi](#) web page.

To set up the MIPI CSI display in the Raspberry Pi 5 OS, follow these steps:

1. Boot into the Raspberry Pi 5 OS and invoke the command terminal window.
2. At the command terminal, type the following command:
`sudo nano /boot/firmware/config.txt`
3. In the config.txt file, locate the following setting and modify the value as follows:
`camera_auto_detect=0`
4. Add the following command right after the line in step 3:
`dtoverlay=imx219,cam0`
5. After completing the changes above, press Ctrl+O.
6. Press Enter to save the changes.
7. Press Ctrl+X to close the config.txt file.
8. Reboot the Raspberry Pi 5 OS for the changes to take effect.

Note: The IMX258 camera sensor is not officially supported in the Raspberry Pi 5 OS camera software. To enable the display preview feature, the IMX219 driver is selected in the Raspberry Pi 5 OS. During driver probing at the Raspberry Pi 5 OS boot up stage, the I2C subsystem in the FPGA returns with an ACK response containing the IMX219 chip ID to the Raspberry Pi 5 OS.

The following figure shows the camera sensor detected in the Raspberry Pi 5 OS during boot up with the FPGA device programmed and connected to it.

```
lattice@lattice: ~
lattice@lattice:~$ dmesg | grep -i imx219
0.029853 /axi/pcie@1000120000/rp1/l2c@888000/imx219@10: Fixed dependency cycle(s) with /axi/pcie@1000120000/rp1/csi@110000
0.029851 /axi/pcie@1000120000/rp1/csi@110000: Fixed dependency cycle(s) with /axi/pcie@1000120000/rp1/l2c@888000/imx219@10
0.029902 /axi/pcie@1000120000/rp1/l2c@888000/imx219@10: Fixed dependency cycle(s) with /axi/pcie@1000120000/rp1/csi@110000
0.030017 /axi/pcie@1000120000/rp1/csi@110000: Fixed dependency cycle(s) with /axi/pcie@1000120000/rp1/l2c@888000/imx219@10
0.385161 /axi/pcie@1000120000/rp1/l2c@888000/imx219@10: Fixed dependency cycle(s) with /axi/pcie@1000120000/rp1/csi@110000
0.385426 /axi/pcie@1000120000/rp1/l2c@888000/imx219@10: Fixed dependency cycle(s) with /axi/pcie@1000120000/rp1/csi@110000
0.385440 /axi/pcie@1000120000/rp1/csi@110000: Fixed dependency cycle(s) with /axi/pcie@1000120000/rp1/l2c@888000/imx219@10
0.675789 /axi/pcie@1000120000/rp1/csi@110000: Fixed dependency cycle(s) with /axi/pcie@1000120000/rp1/l2c@888000/imx219@10
0.675817 /axi/pcie@1000120000/rp1/l2c@888000/imx219@10: Fixed dependency cycle(s) with /axi/pcie@1000120000/rp1/csi@110000
2.919963 rp1-cfe 1f00110000.csi: found subdevice /axi/pcie@1000120000/rp1/l2c@888000/imx219@10
3.027889 rp1-cfe 1f00110000.csi: Using sensor imx219 10-0010 for capture
lattice@lattice:~$
```

Figure B.1. Camera Sensor Detected in the Raspberry Pi 5 OS

Troubleshooting the Raspberry Pi 5 Camera Software Preview Issue

The following figure shows the example error message you may see in the Raspberry Pi 5 OS when running the preview command.

```
lattice@lattice: ~
File Edit Tabs Help

another cable and/or sensor.
ERROR: Device timeout detected, attempting a restart!!!
[0:01:42.277455438] [1703] WARN V4L2 v4l2_videodevice.cpp:2100 /dev/video4[13
.cap]: Dequeue timer of 1000000.00us has expired!
[0:01:42.277561142] [1703] ERROR RPI pipeline_base.cpp:1356 Camera frontend ha
s timed out!
[0:01:42.277566364] [1703] ERROR RPI pipeline_base.cpp:1357 Please check that
your camera sensor connector is attached securely.
[0:01:42.277570049] [1703] ERROR RPI pipeline_base.cpp:1358 Alternatively, try
another cable and/or sensor.
ERROR: Device timeout detected, attempting a restart!!!
[0:01:43.314703104] [1703] WARN V4L2 v4l2_videodevice.cpp:2100 /dev/video4[13
.cap]: Dequeue timer of 1000000.00us has expired!
[0:01:43.314737771] [1703] ERROR RPI pipeline_base.cpp:1356 Camera frontend ha
s timed out!
[0:01:43.314741808] [1703] ERROR RPI pipeline_base.cpp:1357 Please check that
your camera sensor connector is attached securely.
[0:01:43.314745364] [1703] ERROR RPI pipeline_base.cpp:1358 Alternatively, try
another cable and/or sensor.
ERROR: Device timeout detected, attempting a restart!!!

rpikam-app
```

Figure B.2. Timeout Error in the Raspberry Pi 5 OS

The error occurs because of no active data transmission detected in the Raspberry Pi 5 camera software from the MIPI connector on the Raspberry Pi 5 board.

To resolve this issue, perform the following steps:

1. Terminate the current preview session in Raspberry Pi 5 OS.
2. Switch off both the Raspberry Pi 5 board and the CertusPro-NX PCIe Vision Board.
3. Ensure the hardware setup is correct as described in the [Hardware Board Setup](#) section.
4. Ensure the correct 22-pin FFC/FPC flexible flat cable is used for the connection between the Raspberry Pi 5 board and the CertusPro-NX PCIe Vision Board.
5. Follow the [Hardware Board Setup](#) section to rerun the hardware.
6. Verify both Pattern Subsystem to TX mode and RX to TX Retransmit mode to identify whether the issue occurs in both modes or in only one mode.

References

- [MIPI CSI/DSI IP Release Notes \(FPGA-RN-02112\)](#)
- [Lattice Radiant Timing Constraints Methodology \(FPGA-AN-02059\)](#)
- [I2C Target IP User Guide \(FPGA-IPUG-02072\)](#)
- [I2C Controller IP User Guide \(FPGA-IPUG-02071\)](#)
- [RISC-V MC CPU IP User Guide \(FPGA-IPUG-02300\)](#)
- [GPIO IP User Guide \(FPGA-IPUG-02076\)](#)
- [UART IP User Guide \(FPGA-IPUG-02105\)](#)
- [Certus-NX](#) web page
- [CertusPro-NX](#) web page
- [CrossLink-NX](#) web page
- [MachXO5-NX](#) web page
- [Lattice Radiant Software](#) web page
- [Lattice Propel Design Environment](#) web page
- [MIPI CSI/DSI IP Core](#) web page
- [Lattice Solutions Reference Designs](#) web page
- [Lattice Insights](#) for Lattice Semiconductor training courses and learning plans

Technical Support Assistance

Submit a technical support case through www.latticesemi.com/techsupport.

For frequently asked questions, refer to the Lattice Answer Database at www.latticesemi.com/Support/AnswerDatabase.

Revision History

Note: In some instances, the IP may be updated without changes to the user guide. The user guide may reflect an earlier IP version but remains fully compatible with the later IP version. Refer to the IP Release Notes for the latest updates.

Revision 1.2, IP v4.2.0, August 2026

Section	Change Summary
Introduction	Updated IP version in Table 1.1. Summary of the MIPI CSI/DSI IP .
Functional Description	- Updated description for <code>ref_clk_i</code> in the following tables: Table 2.1. MIPI CSI/DSI Transmitter Operating Frequency Details and Requirements Table 2.2. MIPI CSI/DSI Receiver Operating Frequency Details and Requirements - Updated the following figures: Figure 2.20. MBSI TX Timing Diagram for 32-bit Passthrough Mode Figure 2.21. MBSI TX Timing Diagram for 64-bit Passthrough Mode Figure 2.22. MBSI TX Timing Diagram for 32-bit Passthrough Mode with Backpressure Figure 2.23. MBSI RX Timing Diagram for 32-bit Passthrough Mode Figure 2.24. MBSI RX Timing Diagram for 64-bit Passthrough Mode Figure 2.25. MBSI RX Timing Diagram for 64-bit MPPT Passthrough Mode Figure 2.26. MBSI RX Timing Diagram for 32-bit Passthrough Mode with TREADY Deassertion - Added description on byte buffer FIFO in the CRC Insertion of the MIPI CSI/DSI Transmitter section. - Updated the description for 64-bit DSI Controller Mode in the MIPI CSI/DSI Receiver MBSI section. - Updated the initial skew calibration using CSR access in the MIPI CSI/DSI Transmitter Skew Calibration section. - Renamed section title from <i>MIPI CSI/DSI Transmitter Dynamic Reconfiguration</i> to MIPI CSI/DSI Transmitter Dynamic Lane Reconfiguration. - Renamed section title from <i>MIPI CSI/DSI Receiver Dynamic Reconfiguration</i> to MIPI CSI/DSI Receiver Dynamic Lane Reconfiguration. - Added the MIPI CSI/DSI Transmitter Resolution Reconfiguration section.
IP Parameter Description	Updated Table 3.1. General Attributes as follows: - Updated the default value for the <i>Line Rate Per Lane (Mbps)</i> attribute for Hard D-PHY when <i>PHY Direction</i> is TX. - Added Soft D-PHY value for the <i>Reference Clock Frequency (MHz)</i> attribute. - Updated the <i>Byte Buffer Depth</i> attribute.
Signal Description	- Updated description for <code>ref_clk_i</code> and <code>ref_rst_n_i</code> in Table 4.1. Clock and Reset Ports. - Updated description for <code>ref_rst_n_i</code> in Table 4.7. Clock and Reset Ports1. - Updated description for <code>axis_byte_tlast_o</code> in Table 4.10. MIPI Byte Streaming Interface Ports.
Register Description	- Added address offset 0x000 and 0x020 in Table 5.1. Summary of MIPI CSI/DSI Transmitter Registers. - Added the CONTROLLER_STATUS_REG (0x020) section. - Updated the access for <code>NUM_LINE_ACTIVE_V</code> in Table 5.8. CONFIG_REG2 (0x244) Register.
Example Design	- Updated the <i>Reference Clock Frequency</i> attribute in Table 7.1. MIPI CSI/DSI IP Configuration Supported by the Example Design. - Added 60 MHz Sys PLL clock output in the Sys PLL IP section. - Added the <code>phy_ref_clk_out</code> attribute in Table 7.2. Clocking Scheme in the Example Design. - Updated Table 7.6. SW1 DIP Switch Settings.

Section	Change Summary
	- Updated the following figures: Figure 7.9. Demo GUI Setup Figure 7.10. Color Bar CFA – BGGR for Pattern Subsystem to TX Mode Configuration Example
Design Considerations	Added guidelines when the Blanking Period Enable (BP_EN) option is enabled in the Limitations for MIPI CSI/DSI Transmitter section.
Resource Utilization	Updated EBR values in the following tables: Table A.1. MIPI CSI/DSI Transmitter Resource Utilization for the LFCPNX-100-9LFG672C Device1 Table A.3. MIPI CSI/DSI Transmitter Resource Utilization for the LIFCL-40-CABGA400 Device1

Revision 1.1, IP v4.1.0, June 2026

Section	Change Summary
All	Performed minor formatting and editorial edits.
Abbreviations in This Document	Updated list of abbreviations.
Introduction	- Updated the Overview of the IP section. - Updated Table 1.1. Summary of the MIPI CSI/DSI IP as follows: - Added the MBSI AXI4-Stream and AXI4-Lite user interface. - Updated IP version. - Updated Table 1.2. MIPI CSI/DSI IP Support Readiness. - Added description on design example in the IP Support Summary section. - Added support for Hard D-PHY, skew calibration, UVSI, MBSI, AXI4-Lite, and dynamic lane reconfiguration in the Features section. - Removed IP version in MIPI CSI/DSI IP on Transmitter Mode and MIPI CSI/DSI IP on Receiver Mode sections.
Functional Description	- Updated the description on blocks and layers in the IP Architecture Overview section and updated the following figures: Figure 2.1. MIPI CSI/DSI IP Transmitter Core Block Diagram Figure 2.2. MIPI CSI/DSI IP Receiver Core Block Diagram - Renamed <i>MIPI CSI/DSI IP Transmitter Clock Domain Block Diagram</i> to Figure 2.3. MIPI CSI/DSI IP Transmitter Clock Domain Block Diagram (UVSI) and updated figure. - Renamed <i>MIPI CSI/DSI IP Receiver Clock Domain Block Diagram</i> to Figure 2.5. MIPI CSI/DSI IP Receiver Clock Domain Block Diagram (UVSI) and updated figure. - Added the following figures: Figure 2.4. MIPI CSI/DSI IP Transmitter Clock Domain Block Diagram (MBSI) Figure 2.6. MIPI CSI/DSI IP Receiver Clock Domain Block Diagram (MBSI) Figure 2.8. MIPI CSI/DSI IP Transmitter Reset Sequence Waveform Figure 2.9. MIPI CSI/DSI IP Receiver Reset Sequence Waveform - Updated <code>axis_vid_clk_i</code> and added <code>s_axi_aclk_i</code> in Table 2.1. MIPI CSI/DSI Transmitter Operating Frequency Details and Requirements. - Reworked Section 2.9 <i>MIPI CSI/DSI Transmitter Edge Clock Sharing</i> and moved to Section 2.2.1.1 MIPI CSI/DSI Transmitter Edge Clock Sharing. - Added <code>s_axi_aclk_i</code> in Table 2.2. MIPI CSI/DSI Receiver Operating Frequency Details and Requirements. - Added MIPI Byte Streaming Interface in Table 2.3. User Interfaces and Supported Protocols. - Added the MIPI Byte Streaming Interface (MBSI) section. - Updated the MIPI CSI/DSI Transmitter PHY Block section as follows: - Added support for Hard D-PHY mode. - Added the skew calibration feature. - Updated the MIPI CSI/DSI Transmitter Controller Block section and added subsections.

Section	Change Summary
	- Added 64-bit TX controller mode in Table 2.5. MIPI CSI/DSI Transmitter Pixel-to-Byte Supported Configurations. - Removed the <i>MIPI CSI/DSI Transmitter Word Count Support Limitation</i> table in the MIPI CSI/DSI Transmitter Pixel-to-Byte Block section. - Added the MIPI CSI/DSI Transmitter Blanking Insertion section. - Added support for Hard D-PHY mode and deskew calibration in the MIPI CSI/DSI Receiver PHY Block section. - Updated the MIPI CSI/DSI Receiver Controller Block section as follows: - Updated lane merge and depacketization features. - Added the error detection and correction and byte streaming source features. - Added Table 2.7. MIPI CSI/DSI Receiver Word Count Support Limitation. - Updated the MIPI CSI/DSI Receiver Byte-to-Pixel Block section as follows: - Added 64-bit RX controller mode in Table 2.8. MIPI CSI/DSI Receiver Byte-to-Pixel Supported Configurations. - Removed the <i>MIPI CSI/DSI Receiver Word Count Support Limitation</i> table. - Added the Dynamic Reconfiguration section.
IP Parameter Description	- Updated Table 3.1. General Attributes as follows: - Updated attributes: <i>PHY Mode, PHY Data Width, Line Rate Per Lane (Mbps), Actual Byte Clock Frequency (MHz), Actual D-PHY Clock Frequency (MHz), Edge Clock Mode, Reference Clock Frequency (MHz), Line Buffer Depth, Pixel Buffer Depth, tINIT Duration (ns)</i> - Added attributes: <i>ECC Insert (TX) / ECC Check (RX), CRC Insert (TX) / CRC Check (RX), Enable AXI4-Lite Interface, Dynamic Reconfiguration, Unified Video Streaming Interface (UVSI), MIPI Byte Streaming Interface (MBSI), Passthrough Mode, Blanking Enable, Number of Horizontal Active Bytes in Byte Clock Domain, Number of Horizontal Blanking Bytes in Byte Clock Domain, Number of Vertical Blanking Lines Per Frame, Byte Buffer Depth, Maximum Buffered Bytes, Enable Initial Skew Calibration, Enable Periodic Skew Calibration</i> - Updated Table 3.2. MIPI CSI/DSI TX Protocol Timing Parameters – General Attributes as follows: - Added <i>t_SKEWCAL-INITIAL</i> and <i>t_SKEWCAL-PERIODIC</i> attributes. - Updated <i>t_CLK-ZERO</i> and <i>t_CLK-TRAIL</i> attributes.
Signal Description	- Added the following subsections under the MIPI CSI/DSI TX Signal Description section: - D-PHY Interface - MIPI Byte Streaming Interface - AXI4-Lite Subordinate Interface - Updated Table 4.1. Clock and Reset Ports as follows: - Update <i>ref_clk_i, ref_rst_n_i, axis_vid_clk_i, eclk_syncclk_i, byte_clk_o, eclk_syncclk_o, eclk_ready_o</i> ports. - Added <i>s_axi_aclk_i</i> and <i>s_axi_aresetn_i</i> ports. - Added <i>horizontal_blanking_cnt_en_o</i> and <i>vertical_blanking_cnt_en_o</i> ports in Table 4.5. Miscellaneous Ports. - Updated Table 4.7. Clock and Reset Ports¹ as follows: - Update <i>ref_clk_i</i> and <i>ref_rst_n_i</i> ports. - Added <i>s_axi_aclk_i</i> and <i>s_axi_aresetn_i</i> ports. - Added the following subsections under the MIPI CSI/DSI RX Signal Description section: - MIPI Byte Streaming Interface - AXI4 Lite Subordinate Interface - Updated the <i>axis_vid_tuser_o[1:0]</i> port in Table 4.9. Unified Video Streaming Interface Ports.
Register Description	Added this section.
Designing with the IP	- Updated the following figures: - Figure 6.1. Module/IP Block Wizard

Section	Change Summary
	- Figure 6.2. MIPI CSI/DSI IP Receiver Mode Configuration - Figure 6.3. MIPI CSI/DSI IP Transmitter Mode Configuration - Figure 6.4. Check Generated Result - Updated filename in the Generated Files and File Structure section and updated Table 6.1. Generated File List. - Added information about the evaluation wrapper and added Figure 6.5. Example Evaluation Top Setup in the Design Implementation section. - Added information about the top-level wrapper for synthesis in the Timing Constraints section. - Updated Figure 6.10. Simulation Waveform. - Added information about packet integrity error and added Figure 6.12. Simulation Result with Error in the Simulation Results section. - Added description about additional timing information and Figure 6.13. Additional Timing Information.
Example Design	Added this section.
Debugging	Updated the Data Appears Corrupted in Receiver Mode section.
Design Considerations	- Updated the following sections: - Limitations for MIPI CSI/DSI Transmitter - Limitations for MIPI CSI/DSI Receiver - Added the Horizontal and Vertical Blanking Period Insertion Tuning section.
Resource Utilization	- Added a note on resource utilization values. - Updated the following tables: - Table A.1. MIPI CSI/DSI Transmitter Resource Utilization for the LFCPNX-100-9LFG672C Device1 - Table A.2. MIPI CSI/DSI Receiver Resource Utilization for the LFCPNX-100-9LFG672C Device1 - Added the following tables: - Table A.3. MIPI CSI/DSI Transmitter Resource Utilization for the LIFCL-40-CABGA400 Device1 - Table A.4. MIPI CSI/DSI Receiver Resource Utilization for the LIFCL-40-CABGA400 Device1
Raspberry Pi OS Setup	Added this section.
References	Updated references.

Revision 1.0, IP v4.0.0, April 2026

Section	Change Summary
All	Initial release.



www.latticesemi.com